



معهد رسل الحضارة الدولي
للعلوم التقنية والفنية

معهد رسل الحضارة - قسم الحاسوب البرمجة بلغة C++ - الفصل الثاني

د. محمد عبد الرحمن

فصل الربيع 2025

المحاضرة الاولى

مقدمة عامة عن لغات البرمجة

البرمجة تشمل تحويل المهمة المعقدة إلى سلسلة من الأوامر والتعليمات البرمجية التي يمكن للحاسوب فهمها لتنفيذ المهمة المطلوبة. عندما نتحدث عن أنواع البرمجة، فإننا نشير إلى الأساليب المختلفة المستخدمة في كتابة الأكواد وتنفيذها. هناك أنواع مختلفة للبرمجة تشمل البرمجة الهيكلية والبرمجة الشيئية والبرمجة الموجهة للأحداث وغيرها

لغات البرمجة Programming Language:

هي اللغة تخاطب بين المستخدم (المبرمج) والحاسب لها قواعدها وأصولها وتنقسم إلى:

- لغات المستوى الأدنى (LLL) Low Level Language:

و هي اللغات التي تستخدم النظام الثنائي (1.0) الصفر و الواحد للتعبير عن الأوامر المختلفة التي يتكون منها البرنامج و هي لغات صعبة لا يحسن استخدامها إلا من صمم الحاسب نفسه (قلة قليلة من المبرمجين) و تسمى لغة الآلة (Machine Language).

- لغات المستوى المتوسط Middle Level Language:

لغات تميزت بأنها وسط بين لغة الآلة واللغات العالية وتستخدم خليط من الرموز والعلامات وتسمى لغة التجميع (Assembly Language):

- لغات المستوى العالي High Level Language:

اللغات الحديثة المستخدمة في أجهزة الحاسوب و هي قريبة من لغة الإنسان في قواعدها و تمتاز بسهولة الكتابة و سهولة اكتشاف الأخطاء البرمجية و من الأمثلة على هذه اللغات (لغة البيسك، الفورتران، الباسكال، الكوبل، السي و السي ++)

مقدمة عامة في لغة C++

لغة C++ هي لغة برمجة كائنية، متعددة الأغراض وهي من أقدم لغات البرمجة التي لا زالت تُستخدم في أيامنا هذه. وهي المهيمنة على تطبيقات سطح المكتب بجانب لغات المتوفرة من شركة مايكروسوفت. تُستخدم على نطاق واسع في بناء أنظمة التشغيل، التعامل مع البنية الصلبة للحاسوب، وحتى في برامج التسلية كألعاب الفيديو. تُعتبر إحدى اللغات الأكثر شيوعاً وقدرةً. وهي لغة برمجة متعددة الأغراض وتتميز بسرعة تنفيذ الأوامر والقدرة على التعامل مع الذاكرة بشكل مباشر. وتستخدم لغة C++ في مجالات عديدة كبرمجة أنظمة التشغيل، تطوير البرمجيات تطبيقات الحاسوب الذكاء الاصطناعي، الألعاب الإلكترونية، وغيرها من المجالات.

(البرمجة الكائنية أو Object-Oriented Programming (OOP) هي نهج برمجي يهدف إلى تنظيم البرنامج في إطار مفهوم الكائنات أو الأشياء بدلاً من الدوال في البرمجة الإجرائية. يتمثل الهدف من ذلك في تسهيل إدارة البرنامج، تقليل التكرار في الشيفرة، وجعل البرنامج أكثر هيكلية وسهولة في الصيانة

في البرمجة الشيئية، يتم تقسيم البرنامج إلى وحدات تسمى الكائنات (Objects). كل كائن هو حزمة من البيانات (المتغيرات والثوابت) والطرق ووحدات التنظيم وواجهات الاستخدام. يتم بناء البرنامج باستخدام الكائنات وربطها معاً وتوفير واجهة البرنامج الخارجية باستخدام هيكلية البرنامج وواجهات الاستخدام الخاصة بكل كائن.

من مميزات البرمجة الشيئية أنها تسمح بإعادة استخدام الأكواد البرمجية التي تم اختبارها، دون الحاجة لإعادة برمجتها. هذا يسهل بناء البرامج بشكل سريع في وقت قصير.)

مميزاتها

- تعد لغة C++ واحدة من أكثر اللغات البرمجية استخداماً في العالم، وتحظى بعدد من المميزات والفوائد، من أبرزها
- 1- **سرعة التنفيذ:** تتيح لغة C++ سرعة تنفيذ عالية، حيث أنها تتميز بالقدرة على العمل مباشرة على المعالج والذاكرة العشوائية
 - 2- **قابلية الاستخدام:** تعتبر لغة C++ قابلة للاستخدام في العديد من المجالات، كما أنها تدعم العديد من الأنظمة البرمجية مما يجعلها مفيدة في كثير من الأوقات.
 - 3- **التوافق مع لغات أخرى:** يمكن استخدام لغة C++ بشكل متكامل مع لغات أخرى مثل Assembly مما يجعل البرمجة بلغة C++ أكثر مرونة
 - 4- **التحكم الكامل في البرنامج:** تتيح لغة C++ للمبرمجين التحكم الكامل في البرامج التي ينشئونها، حيث يمكنهم الوصول إلى كل الموارد والمكونات المتوفرة في النظام.
 - 5- **دعم البرمجة الشبكية:** يتميز لغة C++ بدعم البرمجة الشبكية، وهي أحدث التقنيات في عالم البرمجة، حيث تجعل البرامج أكثر تنظيماً وسهولة في الصيانة.
 - 6- **الأمان:** تعتبر لغة C++ من أكثر اللغات البرمجية أماناً، حيث يمكن للمبرمجين تحسين أداء البرامج وتوفير الحماية من الهجمات الإلكترونية

استخداماتها

- تعد لغة C++ من أهم اللغات البرمجية في العالم، وتستخدم بشكل شائع في العديد من المجالات منها:
- 1- **تطوير برامج الحاسوب:** حيث تعد لغة C++ من أهم اللغات البرمجية التي يتم استخدامها في تطوير برامج الحاسوب المتنوعة مثل برامج إدارة الملفات وبرامج الألعاب، وبرامج المحاسبة، وغيرها.
 - 2- **تطوير تطبيقات الويب:** يتم استخدام لغة C++ في تطوير تطبيقات الويب المتقدمة، مثل تطبيقات الويب الحية. والتطبيقات الثنائية والتطبيقات الجرافيكية المتقدمة.
 - 3- **تطوير برامج الذكاء الاصطناعي:** تستخدم لغة C++ في تطوير البرامج المتعلقة بالذكاء الاصطناعي والتعلم الآلي ومعالجة الصور.
 - 4- **تطوير برامج الأجهزة المدمجة:** تعد لغة C++ من أشهر اللغات البرمجية التي تستخدم في تطوير البرامج المضمنة في الأجهزة المدمجة، مثل الهواتف الذكية، وأجهزة التحكم الصناعية والروبوتات.
 - 5- **تطوير برامج النظام:** تستخدم لغة C++ في تطوير برامج النظام، مثل أنظمة التشغيل وبرامج إدارة الملفات والأدوات الأساسية للنظام.

الصيغة العامة للبرامج في لغة C++

كل لغة برمجة لديها (بناء جملة syntax) خاص بها، ولكن في الجملة هناك منطقاً برمجياً موحدًا لكل لغة برمجة والكود التالي يوضح الصيغة العامة للبرامج في لغة C++

```
#include <iostream>

using namespace std;

void main()

{

    cout << "Hello World";

}
```

هذا البرنامج يطبع الجملة Hello World على شاشة سوداء (سطر الأوامر)، وفيما يلي شرح لكل سطر:

#include <iostream>

هذه الجملة تقوم بتضمين المكتبة iostream داخل البرنامج، وهي مكتبة ضرورية ومهمة جدًا كي نستطيع طباعة المخرجات على الشاشة، أو حتى استقبال مدخلات من المستخدم، حيث يعتبر الحرف i اختصارًا لكلمة input وتعني "مدخل"، والحرف o اختصارًا لـ output وتعني "مخرج"، وأما كلمة stream فتشير إلى تدفق أو تيار، وبالتالي فإن معنى هذه المكتبة هو "تيار من المدخلات والمخرجات"؛ وتحتوي لغة C++ على العديد من المكتبات الأخرى غير iostream مثل المكتبة cmath والتي تُمكننا من استخدام الدوال الرياضية مثل الدالة abs() التي تعطينا القيمة المطلقة للعدد الذي نريده وغيرها... إذا فالمكتبة عبارة عن مجموعة الدوال والجمال التي تمكننا من القيام بوظائف معينة.

using namespace std;

في لغة البرمجة C++، عبارة "using namespace std" تستخدم لتبسيط استخدام المكتبة القياسية (Standard Library) المتعلقة بالإدخال والإخراج والعديد من الوظائف الأخرى.

فائدة استخدامها:

عند استخدام "using namespace std"، يصبح من الممكن استخدام جميع الرموز والدوال الموجودة في مكتبة الـ std دون الحاجة إلى كتابة البادئة std:: قبل كل رمز. على سبيل المثال، بدلاً من كتابة std::cout لاستخدام دالة الإخراج، يمكننا استخدام cout مباشرة. هذا يجعل الشفرة أكثر قراءة وأقل تكرارًا.

void main()

وهذه الجملة هي عبارة عن دالة، وتسمى الدالة الرئيسية، ويمكن أن تقول أنها المحرك الأساسي في لغة C++، حيث أنه بمجرد تشغيل البرنامج فإنه سيذهب فوراً أولاً إلى الدالة main باعتبارها قلب البرنامج، حيث من عندها يبدأ تنفيذ البرنامج، ولا شيء غيرها؛ وبالنسبة للكلمة void فإنها تشير إلى أن الدالة لا ترجع قيمة عند استدعائها

cout << "Hello World";

الكلمة cout وهي اختصار لـ console output وتعني اطبع مخرجات على الشاشة، متبوعة بالإشارتين << والتي لا بدّ من كتابتها دائماً وهي تسمى إشارة معامل الإخراج، متبوعة بالجملة التي نريد طباعتها، ولا بدّ من تواجد الجملة داخل علامتي تنصيص "" وعدا عن ذلك فسيرفض تنفيذ البرنامج وسيظهر خطأ، كما أن معظم جمل C++ يجب أن تنتهي بفاصلة متقطعة.

لتنفيذ هذا الكود نحتاج الى مترجم للغة C++ (c++ compiler)

المترجم هو أداة تسمح لك بكتابة وتشغيل برامج C++ سواء بتنزيله على جهازك أو استخدام المترجمات المتاحة على الشبكة

لتنزيل مترجم على جهازك اتبع الخطوات الآتية

- 1- في خانة البحث اكتب download codeblocks
- 2- بعد الدخول على الصفحة التي ستظهر أولاً اختر الخيار الأول Download the binary release
- 3- من قائمة windows اختر الخيار الرابع codeblocks-20.03mingw-setup.exe و من القائمة المجاورة اضغط على FossHUB
- 4- بعد انتهاء التحميل قم بتنصيب البرنامج على جهازك

يمكنك استخدام المترجمات المتاحة على الشبكة مثل ما هو موجود على هذا الرابط

[Online C++ Compiler \(programiz.com\)](https://www.programiz.com/online-compiler/)

كيف نبدأ بتنفيذ البرامج

- 1- عند فتح الواجهة الرئيسية للمترجم اختر create new project
- 2- اختر الايقونة console application
- 3- من ثم سوف يقوم المترجم بإنشاء مشروع جديد

الأسئلة الإرشادية

- 1- اذكر ثلاثة امثلة على لغات البرمجة من المستوى العالي
- 2- اذكر ثلاثة من ميزات لغات C++
- 3- اذكر ثلاثة من استخدامات لغة C++
- 4- اذكر الغرض من استخدام جملة `using namespace std` في بداية الكود البرمجي
- 5- الى ماذا تشير كلمة `void` في الدالة الرئيسة `void main()`

المحاضرة الثانية

أنواع البيانات في لغة C++

في لغة C++، نتعامل مع الذاكرة باستخدام أنواع البيانات. هذه الأنواع تحدد كيف يتم تخزين ومعالجة البيانات في البرنامج. دعوني أقدم لك نظرة عامة على بعض أنواع البيانات الشائعة في C++:

1. الأنواع الأساسية (الأولية):

- **int** يُستخدم لتمثيل الأعداد الصحيحة، وحجمه 4 بايت.
`int myNumber = 5;`
- **short** يُستخدم للأعداد الصحيحة القصيرة، وحجمه 2 بايت.
`short myShort = 42;`
- **long** يُستخدم للأعداد الصحيحة الطويلة، وحجمه 4 بايت.
`long myLong = 1234567890;`
- **bool** يُستخدم لتمثيل القيم المنطقية (صحيح/خطأ)، وحجمه 1 بايت.
`bool isTrue = true;`
- **float** يُستخدم لتمثيل الأعداد العشرية، وحجمه 4 بايت.
`float myFloat = 3.14;`
- **double** يُستخدم للأعداد العشرية المزدوجة، وحجمه 8 بايت.
`double myDouble = 2.71828;`

في لغة C++، الأعداد العشرية (floats) والأعداد العشرية المزدوجة (doubles) هما نوعان من أنواع البيانات التي تُستخدم لتمثيل الأعداد الحقيقية، ولكنهما يختلفان في الدقة والمدى:

- **الأعداد العشرية (float):** تستخدم 32 بت في الذاكرة، وتوفر دقة تصل إلى 6-7 أرقام عشرية. هذا يعني أنها يمكن أن تمثل الأعداد بدقة معقولة حتى 7 أرقام بعد الفاصلة العشرية.
- **الأعداد العشرية المزدوجة (double):** تستخدم 64 بت في الذاكرة، وتوفر دقة تصل إلى 15-16 أرقام عشرية. هذا يعني أنها يمكن أن تمثل الأعداد بدقة أعلى بكثير من الأعداد العشرية العادية.

بشكل عام، إذا كنت بحاجة إلى دقة عالية في الحسابات الرياضية، فمن الأفضل استخدام double. ومع ذلك، إذا كانت الدقة المطلوبة ليست عالية جداً وتريد توفير المساحة في الذاكرة، فقد تكون float كافية.

- **char** يُستخدم لتمثيل الأحرف من جدول الأسكي، وحجمه 1 بايت.
`char myChar = 'A';`

(جدول الأسكي (ASCII) هو نظام ترميز يُستخدم في الحوسبة لتمثيل النصوص والرموز المستخدمة في اللغة الإنجليزية وبعض اللغات الأخرى. يُعرف جدول الأسكي بأنه معيار أمريكي لتبادل المعلومات ويُستخدم لتمثيل الحروف والأرقام والرموز الخاصة في الحاسوب، حيث يتم تخصيص رقم لكل رمز يُستخدم في النظام.

يُعرف معيار الأسكي بأنه يحتوي على 128 رمزًا، تُرمز كل منها برقم من 0 إلى 127. يشمل الجدول الأحرف اللاتينية الكبيرة والصغيرة، الأرقام من 0 إلى 9، علامات الترقيم، ورموز التحكم التي تُستخدم لتنسيق النصوص والتحكم في الأجهزة. على سبيل المثال، الرمز 65 يُمثل الحرف 'A'، والرمز 97 يُمثل الحرف 'a'.

- **wchar_t** يُستخدم لتمثيل الأحرف الواسعة (يشمل جدول اليونيكود)، وحجمه أكبر من الأسكي
wchar_t myWideChar = 'ب';

جدول اليونيكود هو معيار دولي يُستخدم لتمثيل النصوص والرموز في جميع أنظمة الكتابة تقريبًا. يُمكن اليونيكود الأجهزة الحاسوبية من تمثيل ومعالجة النصوص المكتوبة بطريقة متناسقة، ويشمل أكثر من 100,000 رمز تغطي مجموعة واسعة من الرموز والحروف، بما في ذلك الحروف اللاتينية، العربية، الهندية، الصينية، وغيرها.

2. الأنواع الإضافية:

- **signed int** يُستخدم لمتغير موجب أو سالب مع الإشارة، وحجمه 4 بايت.
signed int myNumber = -42;
- **unsigned int** يُستخدم للأعداد الصحيحة الموجبة بدون إشارة، وحجمه 4 بايت.
unsigned int positiveNumber = 123;
- **unsigned short** يُستخدم للأعداد الصحيحة الموجبة القصيرة، وحجمه 2 بايت.
unsigned short smallPositiveNumber = 42;
- **unsigned long** يُستخدم للأعداد الصحيحة الموجبة الطويلة، وحجمه 4 بايت.
unsigned long largePositiveNumber = 1234567890;

في لغة C++، **signed** و **unsigned** هما معدلان يُستخدمان لتحديد ما إذا كان نوع البيانات الصحيحة يمكن أن يحتوي على قيم سالبة أو لا.

Signed- يُستخدم لتحديد أن نوع البيانات يمكن أن يحتوي على قيم موجبة وسالبة. على سبيل المثال، **signed int** يمكن أن يحتوي على قيم تتراوح بين الحد الأدنى السالب والحد الأقصى الموجب للعدد الصحيح.

Unsigned- يُستخدم لتحديد أن نوع البيانات لا يمكن أن يحتوي على قيم سالبة ويحتوي فقط على قيم موجبة وصفر. على سبيل المثال، **unsigned int** يمكن أن يحتوي على قيم تبدأ من الصفر وحتى الحد الأقصى الموجب للعدد الصحيح.

الفرق الرئيسي بينهما يكمن في النطاق الذي يمكن لكل نوع تمثيله. الأنواع الموقعة (signed) تستخدم بت واحد لتمثيل الإشارة، مما يقلل من النطاق الموجب المتاح للقيمة، بينما الأنواع غير الموقعة (unsigned) تستخدم جميع البتات لتمثيل القيم الموجبة، مما يزيد من النطاق الموجب المتاح.

3. الأنواع المركبة:

• المصفوفات

المصفوفات في لغة C++ هي هياكل بيانات تُستخدم لتخزين مجموعات من العناصر التي تنتمي إلى نفس النوع. يتم تخزين هذه العناصر في مواقع متجاورة في الذاكرة، ويمكن الوصول إلى كل عنصر منها عبر فهرس. إليك بعض المفاهيم الأساسية حول المصفوفات في C++

تعريف المصفوفة: يمكن تعريف مصفوفة من نوع int بهذا الشكل:

```
int arrayOfInts[5];
```

هنا، arrayOfInts هي مصفوفة تحتوي على 5 عناصر من نوع int.

تهيئة المصفوفة: يمكن تهيئة المصفوفة بقيم محددة عند التعريف:

```
int arrayOfInts[5] = {10, 20, 30, 40, 50};
```

في هذا المثال، تم تعيين قيم لكل عنصر في المصفوفة.

الوصول إلى عناصر المصفوفة: يمكن الوصول إلى عناصر المصفوفة باستخدام الفهارس، مع العلم أن الفهرسة تبدأ من 0:

```
std::cout << arrayOfInts[0]; // سيطبع 10
```

```
std::cout << arrayOfInts[4]; // سيطبع 50
```

المصفوفات متعددة الأبعاد: يمكن أيضاً تعريف المصفوفات متعددة الأبعاد، مثل:

```
int matrix[3][3] = {
```

```
{1, 2, 3},
```

```
{4, 5, 6},
```

```
{7, 8, 9}
```

```
};
```

هنا، matrix هي مصفوفة ثنائية الأبعاد تحتوي على 3 صفوف و3 أعمدة.

المصفوفات مهمة في البرمجة لأنها تسمح بتجميع البيانات ذات الصلة والتعامل معها بكفاءة، خاصةً عند التعامل مع كميات كبيرة من البيانات أو عند تنفيذ العمليات الحسابية والمنطقية على مجموعات البيانات.

• الهياكل (Structures)

الهياكل في لغة C++ هي أنواع بيانات محددة من قبل المستخدم تسمح بتجميع عناصر البيانات المختلفة تحت اسم واحد . تُستخدم الهياكل لتمثيل البيانات التي لها علاقة منطقية ببعضها البعض، ولكن قد تكون من أنواع مختلفة. على سبيل المثال، يمكنك تعريف هيكل لتخزين معلومات عن كتاب كالتالي:

```
struct Book {
```

```
    std::string title;
```

```
    std::string author;
```

```
    int year;
```

```
    float price;
```

```
};
```

في هذا المثال، Book هو هيكل يحتوي على أربعة عناصر: عنوان الكتاب، اسم المؤلف، سنة النشر، وسعر الكتاب. يمكنك بعد ذلك إنشاء متغيرات من نوع Book وتعيين قيم لها:

```
Book myBook;
```

```
myBook.title = "المبتدئين C++";
```

```
myBook.author = "أحمد محمد";
```

```
myBook.year = 2021;
```

```
myBook.price = 29.99;
```

الهياكل مفيدة جداً في تنظيم البيانات وجعل الكود أكثر قابلية للقراءة والصيانة. للمزيد من المعلومات حول الهياكل في C++ وكيفية استخدامها، يمكنك الرجوع إلى الموارد التعليمية المتاحة على الإنترنت.

• الاتحادات (Unions)

الاتحادات في لغة C++، المعروفة بـ union، هي أنواع بيانات مركبة تسمح بتخزين عدة أنواع مختلفة من البيانات في نفس الموقع الذاكري. يمكنك تعريف اتحاد كالتالي:

union Data {

int intValue;

float floatValue;

char charValue;

};

في هذا المثال، Data هو اتحاد يمكن أن يحتوي على int، float، أو char، ولكن يمكن استخدام مساحة الذاكرة المخصصة لعضو واحد فقط في أي وقت. هذا يعني أن إذا قمت بتعيين قيمة إلى intValue ثم قمت بتعيين قيمة إلى floatValue، فإن القيمة الأولى ستُفقد لأن كلا العضوين يشغلان نفس الموقع في الذاكرة.

الاتحادات مفيدة عندما تحتاج إلى استخدام نفس الجزء من الذاكرة لأنواع مختلفة من البيانات، مما يساعد على إدارة الذاكرة بشكل أكثر كفاءة في بعض السيناريوهات.

• الفئات (Classes)

في لغة C++، الـ Class هو نوع بيانات محدد من قبل المستخدم يُستخدم لتعريف الهياكل التي تحتوي على كل من البيانات (المتغيرات) والوظائف (الدوال) التي تعمل على هذه البيانات ويمكنك تصور الـ Class كقالب يُستخدم لإنشاء كائنات (objects)، حيث يُحدد الـ Class الخصائص والسلوكيات التي ستكون لدى الكائنات التي تُنشأ منه.

إليك مثال بسيط على تعريف Class في C++:

class Car {

public:

std::string brand;

std::string model;

int year;

void startEngine() {

std::cout << "Engine started!" << std::endl;

}

};

في هذا المثال، Car هو Class يحتوي على ثلاث متغيرات عضوية (brand, model, year) ودالة عضوية (startEngine). يمكنك بعد ذلك إنشاء كائن من هذا الـ Class واستخدام خصائصه ودواله كالتالي:

```
Car myCar;  
  
myCar.brand = "Toyota";  
  
myCar.model = "Corolla";  
  
myCar.year = 2020;  
  
myCar.startEngine(); // سيطبع "Engine started!"
```

الفئات تعد أساسية في البرمجة الكائنية التوجه (OOP)، حيث يُمكنك من خلاله تجميع البيانات والوظائف المتعلقة بها في وحدة واحدة منظمة.

مقارنة بين الأنواع المركبة للبيانات في لغة C++

في لغة C++، المصفوفات والهياكل والاتحادات والفئات هي أنواع مختلفة من البيانات التي تُستخدم لتنظيم وتخزين البيانات بطرق مختلفة:

المصفوفات (Arrays): هي مجموعة من العناصر التي تنتمي إلى نفس النوع، وتُخزن في مواقع متجاورة في الذاكرة. يُمكن الوصول إلى العناصر عبر فهرس.

الهياكل (Structures): تُستخدم لتجميع عناصر البيانات المختلفة تحت اسم واحد. يمكن أن تحتوي الهياكل على أنواع بيانات متعددة وتُستخدم لتمثيل الكائنات ذات الخصائص المتعددة.

الاتحادات (Unions): تشبه الهياكل لكن بفارق رئيسي؛ جميع أعضاء الاتحاد يشغلون نفس المساحة في الذاكرة. يُمكن تخزين أنواع مختلفة من البيانات في نفس الموقع الذاكري ولكن ليس في نفس الوقت.

الفئات (Classes): هي توسع لمفهوم الهياكل حيث تحتوي على بيانات ووظائف (دوال) تعمل على هذه البيانات. الفئات هي الأساس في البرمجة الكائنية التوجه (OOP) وتُستخدم لإنشاء كائنات.

كل نوع من هذه الأنواع له استخداماته الخاصة ويُفضل بناءً على الحاجة إلى تنظيم البيانات والعمليات التي ستجرى عليها.

الدوال (Functions) في لغة C++

الدوال في لغة C++، المعروفة بـ Functions، هي بنى برمجية تُستخدم لتجميع مجموعة من الأوامر أو العمليات في وحدة واحدة يمكن استدعاؤها عند الحاجة. تُمكن الدوال المبرمجين من تقسيم البرامج إلى أجزاء صغيرة، مما يسهل فهم الكود وإعادة استخدامه وصيانته.

تتكون الدالة من أربعة أجزاء رئيسية:

- رأس الدالة (Function Header)

يحدد نوع القيمة التي سترجعها الدالة، اسم الدالة، والمعاملات (Parameters) التي تتلقاها.

- **جسم الدالة (Function Body)**

يحتوي على الأكواد البرمجية التي تُنفذ عند استدعاء الدالة.

- **استدعاء الدالة (Function Call)**

هو الكود الذي يُنفذ الدالة في البرنامج.

- **تعريف الدالة (Function Definition)**

يشمل رأس الدالة وجسمها ويُحدد كيفية عمل الدالة.

على سبيل المثال، إليك دالة بسيطة تُحسب مجموع عددين في C++:

```
int sum(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int result = sum(5, 3); // استدعاء الدالة  
    std::cout << "المجموع هو " << result << std::endl;  
    return 0;  
}
```

في هذا المثال، sum هي دالة تأخذ معامليْن a و b وترجع مجموعهما. يتم استدعاء الدالة sum داخل الدالة main ويتم طباعة النتيجة.

الهدف من استخدام الدوال في لغة C++ هو تنظيم الكود وجعله أكثر قابلية للقراءة والصيانة، بالإضافة إلى تسهيل إعادة استخدام الكود. الدوال تُمكن المبرمجين من تقسيم البرامج إلى وحدات أصغر يمكن اختبارها واستخدامها بشكل مستقل، مما يساعد في تبسيط التعقيد وتحسين إدارة المشروع.

على سبيل المثال، إذا كان لديك عملية حسابية تُستخدم في أماكن متعددة داخل البرنامج، يمكنك وضع هذه العملية داخل دالة واحدة واستدعائها كلما احتجت إليها، بدلاً من كتابة نفس الكود مرارًا وتكرارًا. هذا يجعل البرنامج أكثر تنظيمًا ويسهل تتبع الأخطاء وإصلاحها. كما أن الدوال تُسهل التعاون بين المبرمجين في المشاريع الكبيرة، حيث يمكن لكل مبرمج العمل على دوال مختلفة دون التأثير على بقية الكود.

ماهي دوال الإدخال في لغة C++

دوال الإدخال في C++ هي الوسائل التي تُستخدم لقراءة البيانات من المستخدم عبر واجهة السطر الأمر Command Line Interface (CLI) أشهر دالة إدخال في C++ هي cin، والتي تُستخدم مع عامل التشغيل >> لقراءة البيانات المُدخلة. بالإضافة إلى cin، هناك دوال أخرى مثل getline() التي تُستخدم لقراءة سطر كامل من النص.

إليك مثالاً على استخدام cin و getline():

```
#include <iostream>

#include <string>

int main() {

    int number;

    std::string text;

    // لقراءة عدد cin استخدام

    std::cout << "أدخل عدداً " << ";

    std::cin >> number;

    // تنظيف البافر

    std::cin.ignore();

    // لقراءة سطر نصي getline استخدام

    std::cout << "أدخل نصاً " << ";

    std::getline(std::cin, text);

    return 0;

}
```

في هذا المثال، cin يُستخدم لقراءة عدد صحيح، و getline() يُستخدم لقراءة سطر من النص بعد تنظيف البافر باستخدام cin.ignore()

البافر (Buffer) هو مساحة مؤقتة في الذاكرة تُستخدم لتخزين البيانات أو النصوص قبل أن يتم نقلها أو معالجتها. يتم استخدام البافر لتحسين أداء البرامج وتجنب تكرار الوصول المباشر إلى الذاكرة الرئيسية. عندما يتم قراءة أو كتابة البيانات، يتم نقلها أولاً إلى البافر، ثم يتم نقلها من البافر إلى الوجهة النهائية. مثلاً، عند قراءة ملف نصي، يتم قراءة البيانات من الملف إلى البافر، ثم يتم معالجتها من البافر. البافر يساعد في تقليل عدد عمليات الوصول المباشر إلى القرص الصلب أو الذاكرة، مما يحسن من أداء البرامج.

في البرمجة، يمكن استخدام البافر في العديد من السيناريوهات، مثل قراءة وكتابة الملفات، والاتصال بقواعد البيانات، والتفاعل مع الشبكات، والإدخال والإخراج من وإلى المستخدم.

عوامل التشغيل في C++

هي رموز تُستخدم لإجراء عمليات محددة على المتغيرات والقيم. على سبيل المثال، + هو عامل تشغيل للجمع، و - لل طرح. هناك أنواع مختلفة من عوامل التشغيل في C++، بما في ذلك:

- عوامل التشغيل الحسابية: مثل الجمع + و الطرح - و الضرب * و القسمة /
- عوامل التشغيل المنطقية: مثل && (العملية اللوجية AND) و || (العملية اللوجية OR) و ! (العملية اللوجية NOT)
- عوامل التشغيل العلاقية: مثل == (التساوي) و != (عدم التساوي) و < (أصغر من) و > (أكبر من).
- عوامل التشغيل البتية: مثل & (AND البتي) و | (OR البتي) و ^ (XOR البتي).
- عوامل التشغيل للتحكم في الوصول: مثل . (للوصول إلى أعضاء الكائن) و -> (للوصول إلى أعضاء الكائن عبر مؤشر)
- عوامل التشغيل الخاصة بالتحميل الزائد: يمكن تعريف عوامل التشغيل بشكل مخصص للأنواع المُعرّفة من قبل المستخدم.

(العملية اللوجية في البرمجة تشير إلى استخدام المنطق الرياضي لتحديد القيم الصحيحة أو الخاطئة. تُستخدم العمليات اللوجية لتقييم التعبيرات وإنتاج نتيجة منطقية، مثل true أو false، وتشمل عمليات مثل الـ (&&) AND، الـ (||) OR، و الـ (!) NOT. هذه العمليات تُستخدم بشكل واسع في الشروط والتحكم في تدفق البرامج)

(كلمة بتي في البرمجة تشير إلى العمليات التي تتعامل مع البتات، وهي أصغر وحدة للبيانات في الحوسبة وتمثل قيمة إما 0 أو 1. عوامل التشغيل البتية في لغات البرمجة مثل C++ تُستخدم للتعامل مع البيانات على مستوى البتات، وتشمل عمليات مثل الـ AND و OR و XOR البتية، والتي تُستخدم لإجراء عمليات منطقية على مستوى البتات الفردية للمتغيرات)

عوامل التشغيل تُسهل كتابة العمليات الحسابية والمنطقية بطريقة تقرأ بشكل مشابه للرياضيات التقليدية، مما يجعل الكود أكثر وضوحاً وسهولة في القراءة.

ماهي دوال الإخراج في لغة C++

دوال الإخراج في لغة C++ هي الدوال التي تُستخدم لعرض النتائج على المستخدم، سواء على شاشة الكمبيوتر أو في ملف. أشهر دالة إخراج في C++ هي cout، والتي تُستخدم مع عامل التشغيل << لطباعة النصوص والقيم المختلفة. هناك أيضاً دوال أخرى مثل cerr و clog لطباعة رسائل الخطأ والتسجيلات.

مثال على استخدام cout:

```
#include <iostream>

int main() {

    // طباعة نص cout استخدام

    std::cout << "C++! مرحباً بك في عالم" << std::endl;

    return 0;

}
```

في هذا المثال، يتم استخدام cout لطباعة رسالة ترحيبية على الشاشة. الدوال cerr و clog تعمل بطريقة مشابهة لكنها تُستخدم لأغراض مختلفة مثل التعامل مع الأخطاء والتسجيلات.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة أمثلة الأنواع الأساسية للبيانات في لغة C++
- 2- اذكر الغرض من استخدام نوع البيانات int
- 3- اذكر الغرض من استخدام نوع البيانات short
- 4- اذكر الغرض من استخدام نوع البيانات bool
- 5- اذكر ثلاثة أنواع من عوامل التشغيل في C++

المحاضرة الثالثة

قواعد البرمجة الأساسية في لغة C++

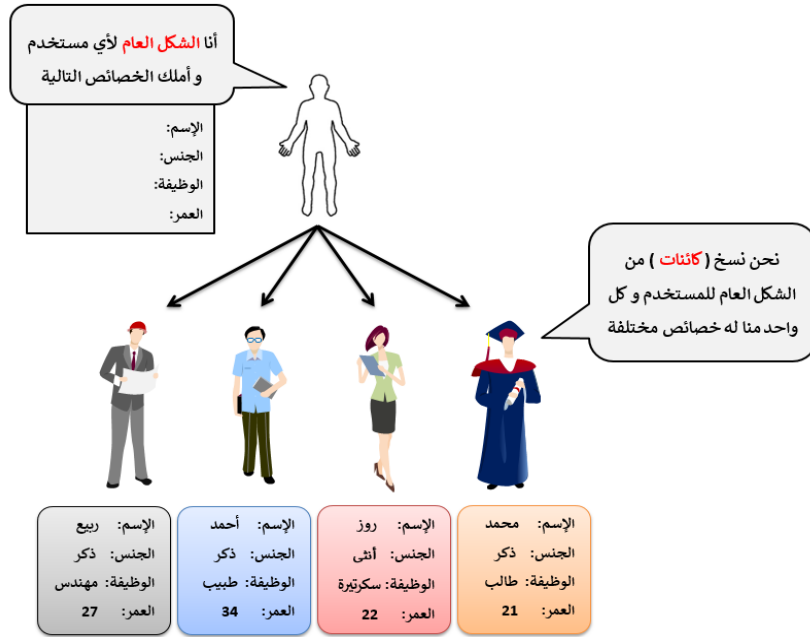
المفاهيم الأساسية في لغة C++

1. الكائنات (Objects):

البرمجة الكائنية (Object Oriented Programming) تختصر بكلمة OOP عبارة عن أسلوب نتبعه في كتابة الكود لجعل كتابته الكود أكثر سهولة. إذاً البرمجة الكائنية هي مجرد أسلوب في العمل لا أكثر وهي ليست خاصة بلغة C++ حيث أنها تطبق في باقي لغات البرمجة.

فكرة البرمجة الكائنية بشكل عام هي تجهيز الشكل الذي سيتم فيه حفظ المعلومات مما يجعل الوصول إليها والتعديل عليها سهل للغاية.

كمثال بسيط، إذا كنت تنوي بناء برنامج لحفظ معلومات المستخدمين، ستقوم بتجهيز الشكل العام للمعلومات التي تنوي حفظها لكل مستخدم. بعدها، أي مستخدم جديد تنوي إنشاؤه تجعله نسخة من الشكل العام لأي مستخدم وتجعله يدخل القيم الخاصة به كما في الصورة التالية.



عليك معرفة أن النوع الجديد أو الشكل الذي تقوم بتجهيزه بهدف إنشاء نسخ منه لاحقاً يقال له النسخة الخام (Blue Print). كلمة النسخة الخام يقصد بها النسخة الأصلية التي يتم تجهيزها بهدف إنشاء نسخ منها.

في C++ لديك خيارين لإنشاء نوع جديد وهما:

- أن تنشئه بواسطة النوع struct
- أن تنشئه بواسطة النوع class

ما هو الكائن؟

بشكل عام، الكائن (Object) عبارة عن نسخة من نوع محدد تم تعريفه بالأساس بواسطة الكلمة struct أو الكلمة class.

أمثلة واقعية عن دور الكائنات

من أكثر أنواع المشاريع التي قد تجد أنك تتعامل فيها مع عدد هائل من الكائنات هو برمجة الألعاب. كمثال بسيط، في أي لعبة تلعبها تجد أنه يوجد شرير ولكن هذا الشرير قد يظهر لك نفسه مئات المرات في اللعبة. أيضاً في بعض الألعاب تجد مطر يتساقط، والدقة أكثر فإنك تجد نفس قطرة المطر تنزل في أماكن مختلفة من الشاشة. إذاً في الألعاب، يتم تجهيز الشرير في الأصل مرة واحدة وكلما أرادوا أن يظهر لك نفس الشرير، يقوموا بإنشاء نسخة منه فقط. وحتى بالنسبة للمطر فإنه أيضاً يمكن تجهيز قطرة مطر واحدة وكلما أرادوا أن يظهر لك مطر، يقوموا بإنشاء عشرات النسخ منها وإظهارها في أماكن مختلفة.

النوع struct في C++

في الدروس السابقة، كنا نتعامل دائماً مع أنواع بيانات بسيطة فمثلاً كنا نقوم بتعريف متغير نوعه int وهذا المتغير كنا نضع فيه قيمة واحدة فقط. الأنواع البسيطة مفيدة جداً وسنتعامل معها دائماً ولكن في حالات معينة لا بد لنا من تعريف أنواع جديدة. كمثال بسيط، إذا كنا ننوي إرسال معلومات مجموعة من المنتجات وكل منتج يملك المعلومات التالية: اسم المنتج، تاريخ إنتاجه، سعره ومكوناته. هنا سيكون خيار ممتاز أن ننشئ نوع جديد يمثل المنتج، أي نوع فيه المعلومات الأساسية التي لا بد أن يمتلكها أي منتج. وعندها أي منتج جديد نريد تعريفه، نجعله نسخة منه.

الكلمة struct تستخدم لتعريف نوع جديد وهذا النوع يمكنه أن يحتوي على مجموعة من القيم من أي نوع كانت بشكل مرتب وسهل التعامل معها. على الأغلب، ستجد النوع الجديد الذي يتم تعريفه بواسطة الكلمة struct يستخدم لهذا الغرض فقط. ولكن عليك معرفة أنه يمكنه أن يحتوي على أي شيء آخر، مثل دوال خاصة به وحتى struct آخر وسترى ذلك لاحقاً في الأمثلة.

مصطلحات تقنية

أي نوع جديد تعرّفه بواسطة الكلمة struct يقال له Structure. أي نسخة تنشئها من النوع الجديد الذي قمت بتعريفه يقال لها كائن (Object) منه.

تعريف struct جديد في C++

إذا كنت ستقوم بتعريف النوع الجديد بواسطة الكلمة struct فيجب أن تتبع الأسلوب التالي.

```
struct struct_name {  
    member_definition;  
    member_definition;  
    member_definition;  
    ..  
} object_names;
```

- struct_name مكانها نضع الاسم الذي سنعطيه للنوع الجديد.
- member_definition هنا يمكنك تمرير اسم و نوع أي شيء تنوي جعل النوع الجديد يملكه.
- object_names إذا أردت إنشاء كائن (نسخة) من النوع الجديد مباشرةً عند تعريفه، فأبى اسم تضعه هنا سيتم إعتباره كائن منه.

الآن عليك معرفة أنه يمكنك تعريف struct في أي مكان تريده، فمثلاً يمكنك تعريفه في ملف خاص، خارج الدالة main() و حتى بداخلها إن أردت.

مثال على struct في لغة C++

```
#include <iostream>

using namespace std;

// لتمثيل معلومات الطالب struct تعريف
struct Student {
    string name;
    int age;
    double gpa;
};

int main() {
    // إنشاء مثيل من struct Student
    Student student1;
    student1.name = "أحمد";
    student1.age = 20;
    student1.gpa = 3.5;

    // عرض معلومات الطالب
    cout << student1.name << endl;
    cout << "اسم الطالب:" << endl;
```

```
cout << "سنة " << student1.age << " :عمر الطالب" << endl;
```

```
cout << "معدل الطالب" << student1.gpa << endl;
```

```
return 0;
```

```
}
```

في هذا المثال، قمنا بتعريف struct يُسمى Student يحتوي على ثلاثة أعضاء name و age و gpa ثم أنشأنا مثيلاً من هذا struct باسم student1 و قمنا بتعيين قيم لأعضائه. أخيراً، قمنا بعرض معلومات الطالب باستخدام cout.

النوع class في C++

الكلاس عبارة عن نوع جديد يتم تعريفه بواسطة الكلمة class و هذا النوع يمكنه أن يحتوي على دوال، متغيرات، مصفوفات إلخ..

النوع الذي نقوم بتعريفه بواسطة الكلمة class يشبه بشكل كبير النوع الذي نقوم بتعريفه بواسطة الكلمة struct التي تعرفنا عليها في الدرس السابق. لهذا السبب سنبدأ بذكر الفرق بينهما حتى لا ترتبك من شدة التشابه الذي ستلاحظه بينهما.

الفرق الأساسي بين النوع الذي يتم تعريفه بواسطة الكلمة class و النوع الذي يتم تعريفه بواسطة الكلمة struct هو أن هذا الأخير يمكن الوصول لأي شيء موجود فيه بشكل مباشر، بينما في النوع class أنت تحدد ما إن كان يمكن الوصول للأشياء التي تضعها فيه بشكل مباشر أم لا.

إمكانية تحديد الطريقة التي يمكن فيها الوصول للأشياء الموجودة في الكلاس تمكننا من تطبيق كل مبادئ البرمجة الكائنية (OOP) المتعارف عليها. لهذا سنركز بشكل كبير في الدروس القادمة على التعامل مع الكلاس بشكل خاص.

تعريف class جديد في C++

لتعريف كلاس جديد نكتب الكلمة class ثم نعطيها اسم، ثم نفتح أقواس تحدد بدايته و نهايته.

المثال التالي يوضح طريقة تعريف كلاس فارغ اسمه Book.

مثال

```
class Book {
```

```
// هنا يمكنك تعريف كل ما سيحتويه الكلاس
```

```
};
```

معلومة تقنية

في المشاريع الحقيقية، أي كلاس جديد نريد تعريفه نقوم في العادة بإنشاء ملف نوعه `cpp` خاص له و نضع الكود الخاص به بداخله. بعدها في أي مكان نريد أن نتعامل معه نقوم بتضمين الملف الذي يحتويه. الآن، بهدف جعل الشرح بسيط و سهل الفهم سنكتب الكود كله بداخل نفس الملف و في آخر الدرس سنعلمك كيف تنشئ الكلاس في ملف خاص أيضاً.

مثال عن الـ class في لغة C++

```
#include <iostream>

using namespace std;

// لتمثيل معلومات الطالب class تعريف

class Student {
public:
    // أعضاء الـ class
    string name;
    int age;
    double gpa;

    // دالة لعرض معلومات الطالب
    void displayInfo() {
        cout << "اسم الطالب" << name << endl;
        cout << "سنة" << age << " :عمر الطالب" << endl;
        cout << gpa << " :معدل الطالب" << endl;
    }
};

int main() {
```



```
// إنشاء مثيل من class Student
Student student1;
student1.name = "أحمد";
student1.age = 20;
student1.gpa = 3.5;
// عرض معلومات الطالب displayInfo استدعاء دالة
student1.displayInfo();
return 0;
}
```

في هذا المثال، قمنا بتعريف class يُسمى Student يحتوي على ثلاثة أعضاء name و age و gpa ثم أنشأنا مثيلاً من هذا class باسم student1 وقمنا بتعيين قيم لأعضائه. ثم استدعينا دالة displayInfo لعرض معلومات الطالب. (الـ void هو نوع بيانات في لغة C++ يُستخدم للإشارة إلى أن الدالة لا تعيد قيمة)

مفهوم الخصائص

الخصائص (Attributes) هي الأشياء (المتغيرات، المصفوفات و الكائنات) التي يتم تعريفها بداخل الكلاس و التي سيملك نسخة خاصة منها أي كائن ننشئه منه.

أي شيء تنوي تعريفه في الكلاس لا بد لك من تحديد كيفية الوصول إليه كما قلنا سابقاً. لتحديد كيفية الوصول للأشياء التي تضعها في الكلاس يمكنك استخدام الكلمات المخصصة لذلك و التي يقال لها Access Specifiers أي الكلمات التالية.

الكلمة	استخدامها
Public	تستخدم لتحديد أن الأشياء الموضوعة في الكلاس يمكن الوصول لها من أي مكان.
Private	تستخدم لتحديد أن الأشياء الموضوعة في الكلاس لا يمكن الوصول لها من خارجه.
Protected	تستخدم لتحديد أن الأشياء الموضوعة في الكلاس يمكن الوصول لها عند وراثتها. ملاحظة: سنتعلم الوراثة لاحقاً

الطرق (Methods):

في لغة البرمجة C++ ، الـ **Methods** هي الدوال التي تنتمي إلى الكلاس (Class). هناك طريقتين لتعريف الدوال التي تنتمي إلى صنف:

داخل تعريف الكلاس (Inside class definition)

يمكن تعريف الدوال داخل تعريف الكلاس نفسه. على سبيل المثال، يمكننا تعريف دالة داخل الكلاس وإطلاق عليها اسم "myMethod". يمكن الوصول إلى هذه الدوال بنفس الطريقة التي يتم الوصول بها إلى السمات (Attributes) ، أي بإنشاء كائن من الكلاس واستخدام النقطة (.) للوصول إليها. هذا مثال يوضح ذلك:

مثال على كيفية استخدام تعريف الـ class داخل الـ class في لغة C++:

```
#include <iostream>
```

```
using namespace std;
```

```
// لتمثيل معلومات الطالب class تعريف
```

```
class Student {
```

```
public:
```

```
    // class أعضاء الـ
```

```
    string name;
```

```
    int age;
```

```
    double gpa;
```

```
// داخلي لتمثيل معلومات العنوان class تعريف
```

```
class Address {
```

```
public:
```

```
    string street;
```

```
    string city;
```

```
    string country;
```

```
};
```

```
// دالة لعرض معلومات الطالب وعنوانه
```

```
void displayInfo() {
```

```
cout << "اسم الطالب" << name << endl;

cout << "سنة " << age << " :عمر الطالب" << endl;

cout << "معدل الطالب" << gpa << endl;

cout << "عنوان الطالب" << address.street << ", " << address.city << ", " <<
address.country << endl;

}

// مثل من class Address

Address address;

};

int main() {

    // إنشاء مثل من class Student

    Student student1;

    student1.name = "أحمد";

    student1.age = 20;

    student1.gpa = 3.5;

    student1.address.street = "شارع الحرية";

    student1.address.city = "طرابلس";

    student1.address.country = "ليبيا";

    // عرض معلومات الطالب وعنوانه displayInfo استدعاء دالة //

    student1.displayInfo();

    return 0;

}
```

في هذا المثال، قمنا بتعريف class يُسمى Student يحتوي على ثلاثة أعضاء name و age و gpa كما قمنا بتعريف class داخلي يُسمى Address لتمثيل معلومات العنوان. ثم أنشأنا مثيلاً من Student باسم student1 وقمنا بتعيين قيم لأعضائه وعنوانه. ثم استدعينا دالة displayInfo لعرض معلومات الطالب وعنوانه.

خارج تعريف الكلاس (Outside class definition)

يمكن تعريف الدوال خارج تعريف الكلاس، وذلك بتعريف الدالة داخل الكلاس ومن ثم تحديد تعريفها خارج الكلاس. يتم ذلك عن طريق تحديد اسم الكلاس، ثم استخدام مُعامل النطاق (::)، ثم اسم الدالة. هذا مثال يوضح ذلك:

مثال على كيفية استخدام تعريف الـ class خارج الـ class في لغة C++:

```
#include <iostream>

using namespace std;

// لتمثيل معلومات الطالب class تعريف
class Student {
public:
    // class أعضاء الـ
    string name;
    int age;
    double gpa;
    // دالة لعرض معلومات الطالب
    void displayInfo();
};

// class خارج الـ displayInfo تعريف دالة
void Student::displayInfo() {
    cout << "اسم الطالب" << endl;
    cout << "سنة " << age << " :عمر الطالب" << endl;
    cout << "معدل الطالب" << gpa << endl;
}

int main() {
    // class إنشاء مثل من
    Student student1;
    student1.name = "أحمد";
```

```
student1.age = 20;  
student1.gpa = 3.5;  
// لعرض معلومات الطالب displayInfo استدعاء دالة  
student1.displayInfo();  
return 0;  
}
```

في هذا المثال، قمنا بتعريف class يُسمى Student يحتوي على ثلاثة أعضاء name و age و gpa ثم قمنا بتعريف دالة displayInfo خارج الـ class لعرض معلومات الطالب. ثم أنشأنا مثيلاً من Student باسم student1 وقمنا بتعيين قيم لأعضائه. ثم استدعينا دالة displayInfo لعرض معلومات الطالب.

المتغيرات الفورية (Instant Variables):

في لغة C++، يُطلق مصطلح "المتغيرات الفورية" على المتغيرات المحلية التي تكون مخزنة في الذاكرة حتى نهاية البرنامج. على عكس المتغيرات المحلية التي ينتهي مداها عند انتهاء الدالة، تبقى المتغيرات الفورية مخزنة في الذاكرة طوال فترة تشغيل البرنامج. وهذا يعني أنها تجمع بين خصائص المتغيرات المحلية والمتغيرات العامة.

في السياق العام، يمكن استخدام المتغيرات الفورية لتخزين البيانات التي يمكن الوصول إليها من أي مكان في البرنامج. يمكن أن تكون هذه المتغيرات مفيدة في العديد من الحالات، مثل تخزين الإعدادات العامة أو الحالات المؤقتة. من الجدير بالذكر أن المتغيرات الفورية تحتفظ بقيمتها حتى نهاية تشغيل البرنامج، وهذا يجعلها مفيدة للتعامل مع البيانات على مستوى البرنامج بأكمله.

فيما يلي مثالاً على المتغيرين:

المتغير الفوري: (Instant Variable)

```
#include <iostream>  
  
int main() {  
    // المتغير الفوري  
    int instantVar = 42;  
    std::cout << "قيمة المتغير الفوري: " << instantVar << std::endl;  
    return 0;  
}
```

في هذا المثال، يتم تعريف المتغير instantVar داخل الدالة main()، وهو مخزن في الذاكرة طوال فترة تشغيل البرنامج.

المتغير المحلي: (Local Variable)

```
#include <iostream>

void myFunction() {
    // المتغير المحلي
    int localVar = 10;
    std::cout << "قيمة المتغير المحلي: " << localVar << std::endl;
}

int main() {
    myFunction();
    // هنا localVar لا يمكن الوصول إلى المتغير
    return 0;
}
```

في هذا المثال، يتم تعريف المتغير `localVar` داخل الدالة `myFunction()`، وهو محدود بنطاق الدالة وينتهي عند انتهاء الدالة.

لغة `C++` هي لغة برمجة قوية ومتعددة الاستخدامات. يمكنك استخدامها لتطوير تطبيقات متنوعة، من تطبيقات سطح المكتب إلى تطبيقات الويب وأكثر. إذا كنت مبتدئاً في `C++`، يمكنك البدء بكتابة أول برنامج باستخدام الصيغة الأساسية وتعلم المزيد عن مصطلحات اللغة وأنواع البيانات والمكتبات.

الجملة في C++

في لغة `C++`، يُطلق مصطلح "الجملة" على الأوامر التي تُنفذ في البرنامج. هذه الأوامر تستخدم لتحقيق مهام معينة أثناء تنفيذ البرنامج. هنا بعض الأمثلة على الجملة الشائعة في `C++`:

جملة الإسناد (Assignment Statement) :

```
int x = 10; // تعيين قيمة 10 للمتغير x
```

جملة الطباعة (Print Statement) :

```
cout << "Hello, World!"; // طباعة نص على الشاشة
```

جملة الشرط (Conditional Statement) :

```
if (x > 5) {  
    cout << "أكبر من 5 x";  
} else {  
    cout << "أقل من أو يساوي 5 x";  
}
```

جملة الحلقة (Loop Statement) :

```
for (int i = 0; i < 5; ++i) {  
    cout << i << " ";  
}  
// النتيجة: 0 1 2 3 4
```

جملة الوظيفة (Function Statement):

```
int add(int a, int b) {  
    return a + b;  
}  
// استدعاء الوظيفة:  
int result = add(3, 5); // النتيجة: 8
```

هذه مجرد أمثلة قليلة، وهناك العديد من الجمل الأخرى في C++ تستخدم للتحكم في تنفيذ البرنامج والتعامل مع البيانات.

العوامل الحسابية في C++

العامل (Assignment Operator) =

العامل = يستخدم لإعطاء قيمة للمتغير.

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 5; // أعطينا المتغير a القيمة 5
    int b = a; // أعطينا المتغير b قيمة المتغير a
    cout << "a = " << a << endl;
    cout << "b = " << b;
    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

a = 5

b = 5

في لغة C++، main.cpp هو اسم ملف يحتوي عادة على دالة main(). هذه الدالة تُعتبر نقطة البداية لتنفيذ أي برنامج مكتوب بلغة C++ في بيئة التنفيذ.

الدالة: main()

- تُعتبر الدالة الرئيسية في برنامج C++.
- يبدأ تنفيذ البرنامج من هذه الدالة.
- يجب أن تكون لديها إما توقيع int main() أو void main().

التوقيع الصحيح لـ: main()

- عادةً ما يُفضل استخدام int main() لأنه يُشير إلى أن البرنامج يُرجع قيمة صحيحة (0) عند الانتهاء بنجاح.
- يُفضل كتابة return 0; في البرنامج للدلالة على أن كل شيء تم على مايرام.

الملف: main.cpp

- يحتوي عادةً على دالة main() وربما على أكواد أخرى.
- يُترجم الملف إلى ملف ثنائي (object file) يحتوي على الأكواد المُعالجة من قبل المترجم.

- يُعتبر نقطة البداية لتنفيذ البرنامج.
- في الختام، يُعتبر `main.cpp` ملفًا مهمًا في برامج C++ حيث يبدأ تنفيذ البرنامج من هذا الملف .

العامل (Addition Operator) +

العامل + يستخدم لإضافة قيمة على قيمة، أي في عمليات الجمع.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 3;
    int b = 4;
    int c = a + b; // c = 3 + 4 = 7
    cout << "c = " << c;
    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

`c = 7`

العامل (Subtraction Operator) -

العامل - يستخدم لإنقاص قيمة من قيمة، أي في عمليات الطرح.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 3;
    int b = 4;
    int c = a - b; // c = 3 - 4 = -1
    cout << "c = " << c;
    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

c = -1

العامل (Unary-Plus Operator) +

يعني ضرب القيمة بالعامل +.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{
```

// هنا وضعنا في المتغير a قيمة أكبر من صفر، ثم وضعنا قيمة الـ Unary-Plus لها في المتغير b

```
int a = 10;

int b = +a; // b = +(10) = 10

cout << "b = " << b << endl;

// هنا وضعنا في المتغير a قيمة أصغر من صفر، ثم وضعنا قيمة الـ Unary-Plus لها في المتغير b

a = -10;

b = +a; // b = +(-10) = -10

cout << "b = " << b;

return 0;

}
```

سنحصل على النتيجة التالية عند التشغيل.

b = 10

b = -10

العامل (Unary-Minus Operator) -

يعني ضرب القيمة بالعامل -.

مثال

```
Main.cpp

#include <iostream>

using namespace std;

int main()

{

// هنا وضعنا في المتغير a قيمة أكبر من صفر، ثم وضعنا قيمة الـ Unary-Minus لها في المتغير b

int a = 10;

int b = -a; // b = -(10) = -10

cout << "b = " << b << endl;

// هنا وضعنا في المتغير a قيمة أصغر من صفر، ثم وضعنا قيمة الـ Unary-Minus لها في المتغير b
```



```
a = -10;  
b = -a; // b = -(-10) = 10  
cout << "b = " << b;  
return 0;  
}
```

سنحصل على النتيجة التالية عند التشغيل.

b = -10

b = 10

*** (Multiplication Operator) العامل**

العامل * يستخدم لضرب قيمة بقيمة، أي في عمليات الضرب.

مثال

```
Main.cpp  
#include <iostream>  
using namespace std;  
int main()  
{  
int a = 6;  
int b = 5;  
int c = a * b; // c = 6 * 5 = 30  
cout << "c = " << c;  
return 0;  
}
```

سنحصل على النتيجة التالية عند التشغيل.

c = 30

العامل / (Division Operator)

العامل / يستخدم لقسمة قيمة على قيمة، أي في عمليات القسمة.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 8;
    int b = 5;
    int c = a / b; // c = 8 / 5 = 1
    cout << "c = " << c;
    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

c = 1

ملاحظة: سبب عدم ظهور أي أرقام بعد الفاصلة هو أننا عرفنا المتغيرات كأعداد صحيحة int.

العامل % (Modulo Operator)

العامل % يقال له الـ Modulo و يسمى Remainder في الرياضيات و هو آخر رقم يبقى من عملية القسمة. إذاً نستخدم الـ Modulo للحصول على آخر رقم يبقى من عملية القسمة. و له فوائد كثيرة، فمثلاً يمكننا استخدامه لمعرفة ما إذا كان الرقم مفرد أو مزدوج (أي Even or Odd) و هذا شرحناه بتفصيل في مادة الخوارزميات.

في هذا المثال سنقوم بتخزين الرقم الذي يبقى من القسمة في المتغير c.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{int a = 8;

int b = 5;

int c = a % b; // c = 8 % 5 = 3

cout << "c = " << c;

return 0;

}
```

سنحصل على النتيجة التالية عند التشغيل.

c = 3

العامل ++ (Increment Operator)

العامل ++ يستخدم لزيادة قيمة المتغير واحداً، و هذا الأسلوب يستخدم كثيراً في الحلقات لزيادة قيمة العداد واحداً في كل دورة بكون أقل.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{

int a = 5;

a++; // a = 5 + 1 = 6

cout << "a = " << a;

return 0;

}
```

سنحصل على النتيجة التالية عند التشغيل.

a = 6

العامل (Decrement Operator) --

العامل -- يستخدم لإنقاص قيمة المتغير واحداً، و هذا الأسلوب يستخدم كثيراً في الحلقات لإنقاص قيمة العداد واحداً في كل دورة بكوند أقل.

مثال

Main.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 5;
    a--; // a = 5 - 1 = 4
    cout << "a = " << a;
    return 0;
}
```

الأسئلة الاسترشادية

- 1- اذكر الفرق الأساسي بين النوع الذي يتم تعريفه بواسطة الكلمة class و النوع الذي يتم تعريفه بواسطة struct الكلمة
- 2- اذكر مفهوم الخصائص (Attributes) في C++
- 3- اذكر الغرض من استخدام Public في ال Class
- 4- اذكر اذكر الغرض من استخدام Private في ال Class
- 5- اذكر الغرض من استخدام Protected في ال Class

المحاضرة الرابعة

جملة if

في لغة ++C، جملة if تُستخدم لتنفيذ كود معين بناءً على شرط معين. إليك مثال على جملة if في لغة ++C:

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    if (x > 5) {
        cout << "أكبر من 5" << endl;
    } else {
        cout << "أقل من أو يساوي 5" << endl;
    }
    return 0;
}
```

في هذا المثال، تستخدم جملة if لفحص ما إذا كانت قيمة المتغير x أكبر من 5. إذا كانت الشرط صحيحاً (يعني أن قيمة x تزيد عن 5)، فسيتم تنفيذ الكود داخل الجملة if، وسيتم طباعة "قيمة x أكبر من 5" باستخدام std::cout. إذا كان الشرط غير صحيح، فلن يتم تنفيذ الكود داخل الجملة if.

الحالات التي تستخدم فيها جملة if:

جملة if تستخدم في لغة ++C لتنفيذ كود معين بناءً على شرط معين. إليك بعض الحالات التي يُمكن استخدام جملة if فيها:

1- تنفيذ كود عندما يكون الشرط صحيحاً

يُستخدم if لتنفيذ كود معين إذا كان الشرط المحدد داخل الجملة صحيحاً

```
int x = 5;

if (x > 0) {
    أكبر من صفر x تنفيذ الكود عندما تكون //
}
```

2- تنفيذ كود بديل في حال عدم تحقق الشرط:

يُمكن استخدام جملة if مع else لتنفيذ كود بديل إذا لم يتحقق الشرط المحدد داخل الجملة if.

```
int x = 5;
if (x > 0) {
    أكبر من صفر x تنفيذ الكود عندما تكون //
} else
{
    أكبر من صفر x تنفيذ كود بديل عندما لا يكون //
}
```

3- تنفيذ كود معقد باستخدام المعاملات الشرطية الأخرى :

يمكن استخدام if مع المعاملات الشرطية الأخرى مثل &&(و) و || (أو) و ! (لا) لتحديد شروط معقدة لتنفيذ الكود

```
int x = 5;
int y = 10;
if (x > 0 && y < 20)
{
    أقل من 20 y أكبر من صفر و x تنفيذ الكود عندما تكون //
}
```

في لغة C++، جملة if تُستخدم لإجراء فحص شرطي. وهناك نوعان من هذه الجملة

جملة if البسيطة:

- تبدأ بكلمة مفتاحية if.
- بين الأقواس () يتم وضع الشرط الذي نريد فحصه.
- إذا كان الشرط يُعتبر صحيحًا (يُقيم إلى true)، سيتم تنفيذ الأوامر الموجودة داخل الأقواس {}.
- إذا كان الشرط غير صحيح (يُقيم إلى false) ، فإن الأوامر داخل الأقواس لن تُنفذ.

جملة: if-else

- تتكون من كلمة مفتاحية if، تليها الشرط.
- إذا كان الشرط صحيحاً، سيتم تنفيذ الأوامر داخل الأقواس {}.
- وإذا كان الشرط غير صحيح، سيتم تنفيذ الأوامر داخل الجزء الآخر من الجملة (الجزء بعد كلمة else)

جمل الشرط بشكل عام

الشكل العام لوضع الشروط هو التالي.

if (condition)

```
{  
    إذا كان الشرط صحيحاً نفذ هذا الكود //  
}
```

else if (condition)

```
{  
    إذا كان الشرط صحيحاً نفذ هذا الكود //  
}
```

else

```
{  
    نفذ هذا الكود في حال لم يتم التعرف على الكود في أي شرط //  
}
```

لست بحاجة إلى استخدام الجمل الثلاثة في كل شرط تضعه في البرنامج و لكنك مجبر على استخدام جملة الشرط if مع أي شرط.

جملة الشرط if

if في اللغة العربية تعني " إذا "، و هي تستخدم فقط في حال كنت تريد تنفيذ كود معين حسب شرط معين

المثال الأول

إذا كانت قيمة المتغير S أكبر من 5 سيتم طباعة الجملة: S is bigger than 5.

```
#include <iostream>

using namespace std;

int main()
{
    int S = 0;

    if( S > 5 )
    {
        cout << "S is bigger than 5";
    }

    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

هنا سأل نفسه التالي: هل قيمة المتغير S أكبر من 5؟
فكان جواب الشرط كلا (false) ، لذلك لم ينفذ أمر الطباعة الموجود في جملة الشرط.

المثال الثاني

إذا كانت قيمة المتغير S أكبر من 5 سيتم طباعة الجملة: S is bigger than 5.

```
#include <iostream>

using namespace std;

int main()
{
```



```
int S = 30;  
  
if( S > 5 )  
{  
    cout << "S is bigger than 5";  
}  
  
return 0;  
}
```

سنحصل على النتيجة التالية عند التشغيل.

S is bigger than 5

هنا سأل نفسه التالي: هل قيمة المتغير S أكبر من 5؟
فكان جواب الشرط نعم (true)، لذلك نفذ أمر الطباعة الموجود في جملة الشرط.

جملة الشرط else

else في اللغة العربية تعني "أي شيء آخر"، و هي تستخدم فقط في حال كنا نريد تنفيذ كود معين في حال كانت نتيجة جميع الشروط التي قبلها تساوي false.

يجب وضعها دائماً في الأخير، لأنها تستخدم في حال لم يتم تنفيذ أي جملة شرطية قبلها.

إذاً، إذا نفذ البرنامج الجملة if أو else if فإنه سيتجاهل الجملة else.
و إذا لم ينفذ أي جملة من الجمل if و else if فإنه سينفذ الجملة else.

المثال الأول

إذا كانت قيمة المتغير S تساوي 5 سيتم طباعة الجملة: S is equal 5.

إذا كانت قيمة المتغير S لا تساوي 5 سيتم طباعة الجملة: S is not equal 5.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
{
    int S = 5;
    if( S == 5 )
    { cout << "S is equal 5";
    }
    else
    {cout << "S is not equal 5";
    }
    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

S is equal 5

هنا سأل نفسه التالي: هل قيمة المتغير S تساوي 5؟
فكان جواب الشرط نعم (true) ، لذلك نفذ أمر الطباعة الموجود في الجملة if.

المثال الثاني

إذا كانت قيمة المتغير S تساوي 5 سيتم طباعة الجملة: S is equal 5.

إذا كانت قيمة المتغير S لا تساوي 5 سيتم طباعة الجملة: S is not equal 5.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int S = 20;
```

```
if( S == 5 )  
{  
    cout << "S is equal 5";  
}  
  
else  
{  
    cout << "S is not equal 5";  
}  
  
return 0;  
}
```

سنحصل على النتيجة التالية عند التشغيل.

S is not equal 5

هنا سأل نفسه التالي: هل قيمة المتغير S تساوي 5؟
فكان جواب الشرط كلا (false) ، لذلك نفذ أمر الطباعة الموجود في الجملة else.

جملة الشرط else if

جملة else if تستخدم إذا كنت تريد وضع أكثر من احتمال (أي أكثر من شرط).

جملة أو جمل الـ else if يوضعون في الوسط، أي بين الجملتين if و else.

مثال

إذا كانت قيمة المتغير number تساوي 1 سيتم طباعة الكلمة: one.
إذا كانت قيمة المتغير number تساوي 2 سيتم طباعة الكلمة: two.
إذا كانت قيمة المتغير number تساوي 3 سيتم طباعة الكلمة: three.
إذا كانت قيمة المتغير number أكبر أو تساوي 4 سيتم طباعة الجملة: four or greater.
إذا كانت قيمة المتغير number أصغر من 0 سيتم طباعة الجملة: negative number.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int number = 3;
```

```
if( number == 1 )
```

```
{
```

```
cout << "one";
```

```
}
```

```
else if( number == 2 )
```

```
{
```

```
cout << "two";
```

```
}
```

```
else if( number == 3 )
```

```
{
```

```
cout << "three";
```

```
}
```

```
else if( number >= 4 )
```

```
{
```

```
cout << "four or greater";
```

```
}
```

```
else
```

```
{
```

```
cout << "negative number";
```

```
}  
  
return 0;  
  
}
```

سنحصل على النتيجة التالية عند التشغيل.

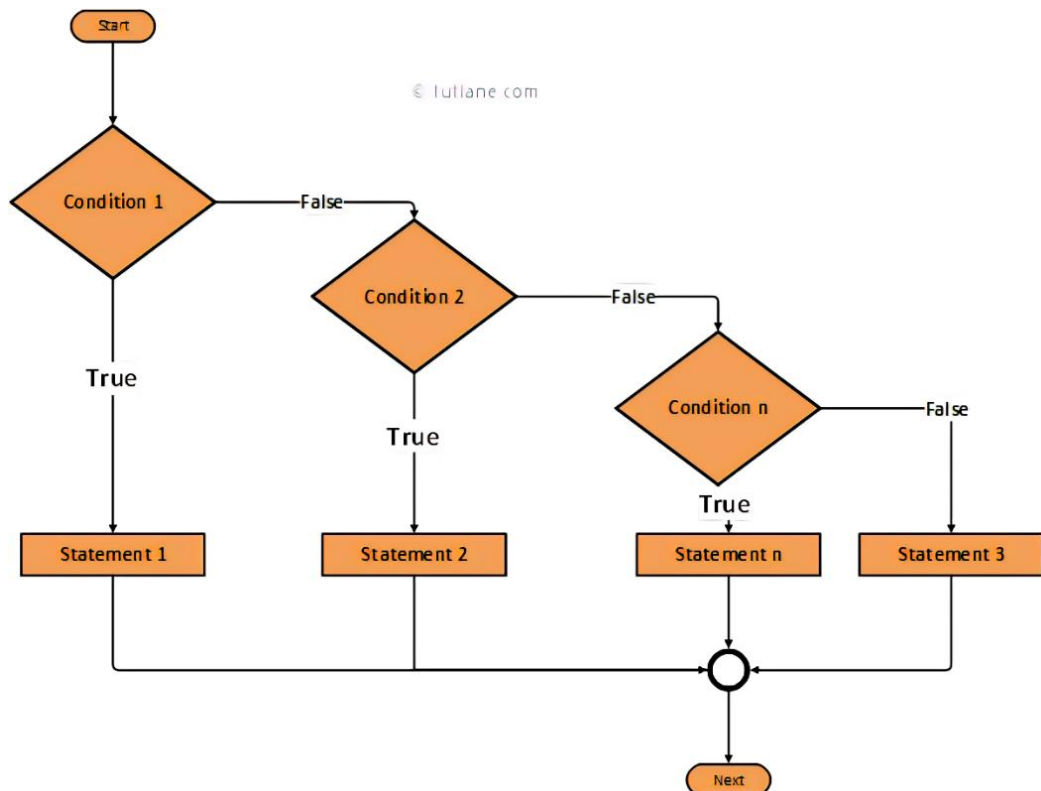
three

هنا سأل نفسه التالي: هل قيمة المتغير number تساوي 1؟
فكان جواب الشرط كلا (false)، فانتقل إلى الشرط الذي يليه.

ثم سأل نفسه التالي: هل قيمة المتغير number تساوي 2؟
فكان جواب الشرط كلا (false)، فانتقل إلى الشرط الذي يليه.

ثم سأل نفسه التالي: هل قيمة المتغير number تساوي 3؟
فكان جواب الشرط هذه المرة نعم (true)، فقام بتنفيذ أمر الطباعة الموجود في جملة الشرط الثالثة، ثم تجاوز جميع جمل الشرط التي أتت بعده.

توضيح if – else بواسطة مخطط التدفق flow chart



الأخطاء التي تحدث عند استخدام الجملة if

هناك بعض الأخطاء الشائعة التي قد تحدث عند استخدام جملة `if` في لغة C++. إليك بعضها:

نسيان القوسين: {}

عندما تكتب جملة `if`، يجب أن تحيط بالشرط المراد فحصه بقوسين. فمثلاً:

```
if (x > 5) {  
    افعل شيئاً هنا //  
}
```

استخدام `=` بدلاً من `==`

في الشروط، يجب استخدام `==` للمقارنة بين القيم، مثل:

```
if (x == 5) {  
    افعل شيئاً هنا //  
}
```

عدم وضع فاصلة بعد الشرط:

بعد كتابة الشرط في الجملة `if`، يجب وضع فاصلة منقوطة ; وإلا ستكون الأوامر التالية جزءاً من الشرط.

```
if (x > 5); {  
    x هذا الكود سينفذ بغض النظر عن قيمة //  
}
```

عدم استخدام `else` في حالاته المناسبة:

إذا كان لديك شرطان متضمنان، يمكن استخدام `else` للتحكم في تنفيذ الأوامر حسب نتيجة الشرط الأول، مثل:

```
if (x > 5) {  
    افعل شيئاً هنا //  
}  
else {  
    افعل شيئاً آخر هنا //  
}
```

عدم التأكد من الأقواس في الشروط المعقدة:

عند كتابة شروط معقدة تحتوي على العديد من المتغيرات والعمليات، يجب التأكد من وضع الأقواس بشكل صحيح لتحديد أولوية التشغيل، مثل:

```
if ((x > 5) && (y < 10)) {  
    افعل شيئاً هنا //  
}
```

ما هو الفرق بين if وغيرها من الجمل الشرطية في C++

في لغة C++ ، هناك عدة جمل شرطية يمكن استخدامها لتحديد تنفيذ الأوامر بناءً على قيمة معينة. من بين هذه الجمل الشرطية الشائعة هي if ، else if ، و switch . هنا الفرق بينها:

if

- تُستخدم if لتحديد تنفيذ كتلة من الأوامر إذا كان شرط معين صحيحًا (يُقدم لها قيمة true).
- إذا كان الشرط غير صحيح (يُقدم لها قيمة false) ولم يتم استخدام else ، فإن الكود بعد ال if لن يُنفذ.

else if

- تُستخدم else if لتحديد شرط جديد يتم فحصه في حالة عدم تحقق الشروط السابقة.
- إذا كان شرط ال if الأول صحيحًا، فإن else if لن يُفحص.
- يمكن استخدام else if بعد if أو else if لتحديد سلسلة من الشروط المتتالية.

switch

- switch يُستخدم لاختبار قيمة متغير معين وتحديد العملية التي يجب تنفيذها بناءً على القيمة.
- يُعد switch بديلاً عن if عندما تكون هناك قائمة من الحالات المحتملة، حيث يكون التنفيذ بناءً على القيمة التي تُقدم لل switch.

بشكل عام، تُستخدم `if` و `else if` لفحص شروط متفرقة، في حين تُستخدم `switch` عندما تكون هناك قائمة من الحالات الممكنة لقيمة معينة.

الأسئلة الإرشادية

- 1- اذكر ثلاثة من الحالات التي تستخدم فيها جملة if
- 2- اذكر ثلاثة من الأخطاء التي يمكن أن تحدث عند استخدام جملة if
- 3- اذكر الفرق بين جملة if و جملة else if

المحاضرة الخامسة

العامل (Conditional Operator) ?:

العامل ?: يقال له Conditional أو Ternary Operator لأنه يأخذ ثلاث عناصر ليعمل. يمكن إستعماله بدل جمل الشرط if و else في حال كنت تريد إعطاء قيمة للمتغير.

بناؤه

variable x = (expression) ? value if true : value if false

- **expression** يمثل الشرط الذي نضعه.
- **value if true** تمثل القيمة التي ستعطى للمتغير **x** إذا تحقق الشرط نضعها بعد الرمز **?**.
- **value if false** تمثل القيمة التي ستعطى للمتغير **x** إذا لم يتحقق الشرط نضعها بعد الرمز **:**.

المثال الأول

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a = 10;
```

```
// تساوي 1 سيتم b, إن لم تكن كذلك سيتم وضع القيمة 30 في المتغير b  
إذا كانت قيمة المتغير a وضع القيمة 20 في المتغير
```

```
int b = (a == 1) ? 20 : 30;
```

```
cout << "b = " << b;
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

b = 30

نلاحظ أنه تم وضع القيمة 30 في b لأن نتيجة الشرط كانت false.

المثال الثاني

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a = 10;
```

```
// أكبر من 1 b, إن لم تكن كذلك سيتم وضع القيمة 30 في المتغير b  
إذا كانت قيمة المتغير a سيتم وضع القيمة 20 في المتغير
```

```
int b = (a > 1) ? 20 : 30;
```

```
cout << "b = " << b;
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

b = 20

نلاحظ أنه تم وضع القيمة 20 في b لأن نتيجة الشرط كانت true.

متى يستخدم المشغل المشروط

المشغل المشروط في لغة البرمجة ++C يُستخدم عادة في حالات تحتاج فيها إلى إتخاذ قرار بناءً على شرط معين. يُمكن استخدام المشغل المشروط في العديد من السياقات، بما في ذلك:

1. الإسناد الشرطي:

يُستخدم المشغل المشروط لإسناد قيمة متغير اعتمادًا على شرط محدد. مثلاً، فيمكن استخدامه لتعيين قيمة متغير اعتمادًا على ما إذا كانت قيمة متغير آخر تفوق حدًا معينًا أو لا.

```
int x = 10؛
```

```
int y = (x > 5) ? 100 : 200 // إذا كان x أكبر من 5، اسند 100 لـ x؛ وإلا اسند 200 لـ y
```

2. طباعة القيم بناءً على الشرط:

يُمكن استخدام المشغل المشروط أيضًا لطباعة قيم مختلفة اعتمادًا على شرط معين.

```
int age = 25؛
```

```
std::cout << "You are " << (age >= 18 ? "adult" : "minor") << std::endl // إذا كان العمر أكبر من 18، فسيُطبع "adult"؛ وإلا سيُطبع "minor"
```

3. التفقد الشرطي في تعبيرات الإرجاع:

يُستخدم المشغل المشروط في بناء تعبيرات الإرجاع (return statements) لإرجاع قيم مختلفة اعتمادًا على شروط معينة.

```
int max(int a, int b){
```

```
    b، وإلا ارجع a، فارجع b أكبر من a؛ // إذا كان a > b،
```

```
{
```

4. التحكم في تنفيذ الشفرة:

يُمكن استخدام المشغل المشروط في تنفيذ أجزاء مختلفة من الشفرة اعتمادًا على شروط معينة.

```
int x = 10؛
```

```
"x"؛ // إذا كان x is greater than 5 : std::cout << "x is greater than 5" : std::cout << "x is not greater than 5" // إذا كان x أكبر من 5، اطبع "x is greater than 5"؛ وإلا اطبع "x is not greater than 5"
```

يُستخدم المشغل المشروط لتبسيط التعبيرات وجعل الشفرة أكثر قراءة وفهمًا، ويُمكن استخدامه في العديد من السياقات حسب الحاجة والمتطلبات البرمجية.

جملة Switch

تعريف الجملة switch

switch نستخدمها إذا كنا نريد إختبار قيمة متغير معين مع لائحة من الاحتمالات نقوم نحن بوضعها، و إذا تساوت هذه القيمة مع أي إحتمال وضعناه ستنفذ الأوامر التي وضعناها في هذا الإحتمال فقط.

كل إحتمال نضعه يسمى case.

أنواع المتغيرات التي يمكن إختبار قيمتها باستخدام هذه الجملة هي: enum - char - short - byte - int.

طريقة تعريفها

يمكننا تعريفها بعدة أشكال، الشكل الأساسي هو التالي:

```
switch (expression) {
```

```
    case value:
```

```
        // Statements
```

```
        break;
```

```
    case value:
```

```
        // Statements
```

```
        break;
```

```
    default:
```

```
        // Statements
```

```
        break;
```

```
}
```

- switch تعني اختبر قيمة المتغير الموضوع بين قوسين.
- expression هنا يقصد بها المتغير الذي نريد إختبار قيمته. نوع المتغير الذي يسمح لنا بإختباره. int - short - char - String :



- **case** تعني حالة value, تعني قيمية، و Statements تعني أوامر. و يقصد من هذا كله، أنه في حال كانت قيمة الـ expression تساوي هذه القيمة سيقوم بتنفيذ الأوامر الموضوعة بعد النقطة. الآن بعد تنفيذ جميع الأوامر الموضوعة بعد النقطتين، يجب وضع break لكي يخرج من الجملة switch مباشرةً بدل أن ينتقل للـ case التالية الموجودة في الجملة switch. نستطيع وضع العدد الذي نريده من الـ case بداخل الجملة switch. إنتبه: الـ expression و الـ value يجب أن يكونا من نفس النوع.
 - **default** تعني افتراضياً و هي نفس فكرة الجملة else, و يمكننا أن لا نضعها أيضاً. هذه الجملة تنفذ فقط في حال لم تنفذ أي case موجودة في الجملة switch و لذلك نضعها بالآخر.
- لا حاجة لوضع break للحالة الأخيرة لأن البرنامج سيخرج من الجملة switch في جميع الأحوال.

المثال الأول

سنقوم باختبار قيمة المتغير x و الذي نوعه int.
سنضع عدة حالات و كل حالة تطبع شيء معين.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int x = 2;
```

```
switch(x) // اختبار قيمة المتغير x
```

```
{
```

```
case 1: // في حال كانت تساوي 1 سيتم تنفيذ أمر الطباعة الموضوع فيها
```

```
cout << "x contain 1";
```

```
break;
```

```
case 2: // في حال كانت تساوي 2 سيتم تنفيذ أمر الطباعة الموضوع فيها
```

```
cout << "x contain 2";
```

```
break;
```

في حال كانت تساوي 3 سيتم تنفيذ أمر الطباعة الموضوع فيها // case 3:

```
cout << "x contain 3";
```

```
break;
```

في حال كانت لا تساوي أي قيمة من القيم الموضوعة سيتم // default:
تنفيذ أمر الطباعة الموضوع فيها

```
cout << "x contain a different value";
```

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

x contain 2

- نلاحظ أنه تم تنفيذ أمر الطباعة الموجود في الحالة الثانية لأن قيمة المتغير x تساوي 2.
- هنا سأل نفسه التالي: هل قيمة المتغير x تساوي 1؟
فكان جواب الشرط كلا (false)، فانتقل إلى الحالة التي تليه.
- ثم سأل نفسه التالي: هل قيمة المتغير x تساوي 2؟
فكان جواب الشرط نعم (true)، فقام بتنفيذ أمر الطباعة الموجود في هذه الحالة، و بعدها خرج من جملة الـ switch بأكملها بسبب الجملة break.

المثال الثاني

سنقوم باختبار قيمة المتغير x و الذي نوعه int.

سنضع عدة حالات و كل حالة تطبع شيء معين.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int x = 5;

switch(x) // اختبار قيمة المتغير x
{
    case 1: // في حال كانت تساوي 1 سيتم تنفيذ أمر الطباعة الموضوع فيها
        cout << "x contain 1";
        break;
    case 2: // في حال كانت تساوي 2 سيتم تنفيذ أمر الطباعة الموضوع فيها
        cout << "x contain 2";
        break;
    case 3: // في حال كانت تساوي 3 سيتم تنفيذ أمر الطباعة الموضوع فيها
        cout << "x contain 3";
        break;
    default: // في حال كانت لا تساوي أي قيمة من القيم الموضوعة سيتم
        تنفيذ أمر الطباعة الموضوع فيها
        cout << "x contain a different value";
}

return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

x contain a different value

- نلاحظ أن تم تنفيذ أمر الطباعة الموجود في الحالة الافتراضية لأن قيمة المتغير x لا تساوي أي قيمة من القيم الموضوعة في الحالات.
- هنا سأل نفسه التالي: هل قيمة المتغير x تساوي 1؟ فكان جواب الشرط كلا (false)، فانتقل إلى الحالة التي تليه.

- ثم سأل نفسه التالي: هل قيمة المتغير x تساوي 2؟ فكان جواب الشرط كلا (false)، فانتقل إلى الحالة التي تليه.
- ثم سأل نفسه التالي: هل قيمة المتغير x تساوي 3؟ فكان جواب الشرط كلا (false)، فانتقل إلى الحالة التي تليه.
- بما أنه لم يجد أي حالة تساوت فيها القيمة مع قيمة المتغير الذي يتم اختباره، قام بتنفيذ الأوامر الموجودة في الحالة الافتراضية default، وعندما إنتهى من تنفيذ الأوامر خرج من جملة الـ switch بأكملها.

وضع نفس الأوامر لأكثر من حالة

إذا أردت وضع نفس الأوامر لأكثر من حالة، عليك وضع الحالات تحت بعضها، ثم كتابة الأوامر، ثم وضع break.

مثال

سنقوم باختبار قيمة المتغير x و الذي نوعه int.

سنضع ثلاث حالات ينفذون نفس الأوامر.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int x = 3;
```

```
switch(x) // اختبار قيمة المتغير x
```

```
{
```

```
case 1: // في حال كانت تساوي 1 أو 2 أو 3 سيتم تنفيذ أمر الطباعة
```

```
case 2:
```

```
case 3:
```

```
cout << "x contain 1 or 2 or 3";
```

```
break;
```

```
default: // في حال كانت لا تساوي أي قيمة من القيم الموضوعة سيتم
```

```
تنفيذ أمر الطباعة الموضوع فيها
```



```
cout << "x contain a different value";  
  
}  
  
return 0;  
  
}
```

سنحصل على النتيجة التالية عند التشغيل.

x contain 1 or 2 or 3

- نلاحظ أنه تم تنفيذ أمر الطباعة الموضوع للحالات الثلاث الأولى لأن قيمة x تساوي 3.
- هنا سأل نفسه التالي: هل قيمة المتغير x تساوي 1 أو 2 أو 3؟

فكان جواب الشرط نعم (true)، فقام بتنفيذ أمر الطباعة الموضوع لهذه الحالات الثلاث، و بعدها خرج من جملة الـ switch بأكملها بسبب الجملة break.

وضع أكثر من احتمال باستخدام العامل ...

إذا كان لديك سلسلة من الاحتمالات تريد وضع نفس الأوامر عليها، يمكنك استخدام العامل

مثال

سنقوم باختبار قيمة المتغير x و الذي نوعه int.

سنضع 100 حالة ينفذون نفس الأوامر باستخدام case واحدة.

```
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
  
int x = 40;  
  
switch( x ) // اختبار قيمة المتغير x  
{
```

في حال كانت تساوي أي عدد صحيح موجود بين 1 و 100 سيتم // 100 ... 1 case
تنفيذ أمر الطباعة

```
cout << "x contain " << x;
```

```
break;
```

في حال كانت لا تساوي أي قيمة من القيم الموضوعة سيتم // default:
تنفيذ أمر الطباعة الموضوع فيها

```
cout << "x contain a different value";
```

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

x contain 40

- نلاحظ أنه تم تنفيذ أمر الطباعة الموضوع للحالة الأولى لأن قيمة x موجودة بين 1 و 100.
- هنا سأل نفسه التالي: هل قيمة المتغير x تساوي 1 أو 2 إلخ.. حتى وصل إلى 40 فكان جواب الشرط نعم (true)، لهذا قام بتنفيذ أمر الطباعة الموضوع لكل الحالات الموجودة بين 1 و 100 و بعدها خرج من جملة الـ switch بأكملها بسبب الجملة break.

متى تستخدم جملة switch

تستخدم هيكلية التحكم switch في لغة البرمجة ++C في حالات تحتاج فيها إلى اتخاذ قرارات معقدة بناءً على قيم متعددة لمتغير واحد. إليك بعض الحالات الشائعة التي يُستخدم فيها switch في ++C:

1. التحكم في الأحرف والأحرف الرقمية:

يمكن استخدام switch للتحكم في قيم الأحرف والأحرف الرقمية المُدخلة من المستخدم أو المُسترجعة من إجراءات أخرى. على سبيل المثال، يمكن استخدام switch لتحويل حرف إلى حرف كبير أو صغير أو لتحديد نوع الحرف (حرف عددي، حرف حروفي، إلخ).

2. التحكم في القيم المُعدودة:

switch يُستخدم بشكل شائع للتحكم في القيم المُعدودة مثل الأيام (الأحد، الاثنين، ... السبت) أو الشهور (يناير، فبراير، ... ديسمبر) أو الخيارات في القوائم (نعم، لا، ربما).

3. تحليل النصوص:

في بعض الحالات، يُمكن استخدام switch لتحليل النصوص أو الكلمات المُدخلة لتحديد الإجراء المناسب يُنفذ بناءً على الكلمة المدخلة.

4. القرارات المتعددة:

عندما يكون هناك العديد من الخيارات الممكنة التي يمكن أن تكون قيمة متغير معين، يمكن استخدام switch لتحديد الإجراء المناسب بناءً على قيمة المتغير.

5. العمليات الحسابية المُعدودة:

في بعض الحالات، يمكن استخدام switch لتنفيذ عمليات حسابية مُعدودة مثل جداول الضرب أو العمليات الحسابية البسيطة الأخرى.

الأخطاء التي تحدث عند استخدام جملة

عند استخدام جملة switch في لغة البرمجة ++C، يمكن أن تحدث بعض الأخطاء الشائعة التي يجب تجنبها للحصول على كود صحيح ومنظم. من بين هذه الأخطاء:

1. عدم استخدام قيم ثابتة:

يجب أن تكون القيم التي تُستخدم مع switch ثابتة ومعروفة مسبقاً. على سبيل المثال، لا يمكن استخدام متغير في switch كما هو الحال في الأسهم التالية:

```
int x = 10;

switch (x) {
    case 10: // صحيح
        // الإجراءات
        break;
    case y: // ليس قيمة ثابتة خطأ،
        // الإجراءات
        break;
}
```

2. عدم استخدام break بشكل صحيح:

يجب استخدام break في نهاية كل حالة case لمنع تنفيذ حالات أخرى. في حالة عدم استخدام break، سيتم تنفيذ حالات الـ case التالية حتى يتم الوصول إلى break أو الختم النهائي للـ switch.

```
int x = 10;
```

```
switch (x) {
```

```
case 10:
```

```
    // الإجراءات
```

```
case 20:
```

```
    // الإجراءات
```

```
break;
```

```
}
```

3. عدم وجود default case:

قد تحدث أخطاء إذا لم يتم توفير default case في switch، خاصة إذا كان من الممكن أن تأتي قيم غير متوقعة للمتغير.

```
int x = 30;
```

```
switch (x) {
```

```
case 10:
```

```
    // الإجراءات
```

```
break;
```

```
case 20:
```

```
    // الإجراءات
```

```
break;
```

```
    ، قد تحدث أخطاء default case في حالة عدم وجود //
```

```
}
```

4. تكرار الحالات:

يجب تجنب تكرار الحالات في switch، حيث يجب أن تكون قيم الحالات فريدة ومختلفة.

```
int x = 10;
```

```
switch (x) {
```

```
case 10:
```

```
    // الإجراءات
```

```
break;
```

خطأ، التكرار غير مسموح به // case 10:

// الإجراءات

break;

}

5. استخدام الأنواع غير المناسبة:

يجب أن يكون نوع المتغير المستخدم مع switch مناسباً لأنواع الحالات التي يتم التحقق منها.

double x = 10.5;

مناسباً للأحجام المقارنة x خطأ، يجب أن يكون نوع // { switch (x)

case 10:

// الإجراءات

break;

case 20:

// الإجراءات

break;

}

بالتالي، يجب تجنب هذه الأخطاء الشائعة عند استخدام جملة switch في ++C للحصول على كود صحيح وخالي من الأخطاء.

الأسئلة الاسترشادية

- 1- اذكر الحالات التي تستخدم فيها جملة if statement
- 2- اذكر ثلاثة من الأخطاء التي تحدث عند استخدام جملة if statement
- 3- اذكر احد الفروق بين if و else if
- 4- اكتب الغرض من استخدام الجملة الشرطية switch
- 5- اكتب ثلاثة امثلة على الحالات التي تستخدم فيها الجملة الشرطية switch
- 6- اذكر ثلاثة من الأخطاء التي يمكن ان تحدث عند استخدام الجملة الشرطية switch

المحاضرة السادسة

جملـة while

تستخدم لتكرار تنفيذ كتلة من الشفرة طالما استمر الشرط صحيحا وعندما يكون الشرط خطأ يتوقف التنفيذ وينتهي التكرار

while (شرط) {

 الشفرة التي تحتاج إلى التكرار هنا //

}

`while` على سبيل المثال، إذا كان لدينا متغير يُسمى `عدد` ونريد تكرار طباعة قيم من 1 إلى 5، يمكننا استخدام جملة كما يلي

```
#include <iostream>
```

```
int main() {
```

```
    int عدد=1;
```

```
    while (عدد <= 5) {
```

```
        std::cout << عدد << std::endl;
```

```
        عدد++;
```

```
    }
```

```
    return 0;
```

في هذا المثال تستمر الجملة في طباعة عدد طالما انه اقل من او يساوي 5 وعندما يصبح عدد اكبر من 5 يتوقف التكرار

حالات استخدام الجملة while

تُستخدم جملة `while` في لغة C++ ولغات برمجة أخرى بشكل شائع لعدة حالات، منها:

1- تكرار عمليات معينة حتى يتحقق شرط معين:

عندما تحتاج إلى تنفيذ سلسلة من العمليات حتى يتحقق شرط معين، يمكنك استخدام `while` لتكرار العمليات حتى يصبح الشرط غير صحيح.

```
#include <iostream>
```

```
int main() {
```



```
int عدد = 1;
while (عدد <= 5) {
    std::cout << عدد < std::endl;
    عدد++;
}
return 0;
}
```

شرح الكود

#include <iostream>

هي تعليمة تُستخدم لتضمين مكتبة الإدخال والإخراج القياسية (Standard Input/Output Library)، والتي تسمى أيضًا بمكتبة `iostream`. تعتبر هذه المكتبة أحد أهم المكتبات في `C++`، حيث تحتوي على الأدوات اللازمة لإجراء عمليات الإدخال من المستخدم (مثل `cin`) والإخراج إلى الشاشة (مثل `cout`).

عندما تستخدم **#include <iostream>**، يقوم المترجم (compiler) بإدراج جميع الدوال والتعاريف اللازمة من ملفات المكتبة `iostream` في البرنامج الخاص بك، مما يتيح لك استخدام أدوات الإدخال والإخراج القياسية في البرنامج.

بشكل عام، تكون تعليمات **#include** هي جزء من المكتبات المساعدة (libraries) التي يُستخدمها المبرمجون لتوسيع وظائف لغة البرمجة، وتضمن تضمين الأدوات اللازمة لتنفيذ مهام معينة في البرنامج.

int main()

هي الدالة الرئيسية (Main Function) في البرنامج، وهي النقطة البدائية التي يبدأ منها تنفيذ البرنامج عند تشغيله. تُعد `main()` جزءًا أساسيًا من كل برنامج في `C++`، حيث يجب أن يحتوي كل برنامج على دالة `main()` ليتم تنفيذه.

التعبير **int main()** يعني ما يلي:

- `int`: هو نوع القيمة التي يُعيد البرنامج عند انتهاء تنفيذ الدالة `main()`، حيث يُعاد عادة الرقم 0 للإشارة إلى أن البرنامج تنفيذه بنجاح. يمكنك استبدال `int` بـ `void` إذا لم يكن البرنامج يحتاج إلى إرجاع قيمة.
- `main()`: هو اسم الدالة الرئيسية التي يتم تنفيذها عند بدء تشغيل البرنامج. يجب أن يكون اسم الدالة `main` دائمًا في `C++` لتكون الدالة الرئيسية.

في العادة، تبدأ دالة **int main()** بفتح جملة داخلية `{}`، ويكون في هذه الجملة تفاصيل الشفرة التي يجب تنفيذها عند تشغيل البرنامج. على سبيل المثال

std::cout

هو كائن من الفئة `std::ostream` الموجود في مكتبة الإدخال والإخراج القياسية (`iostream`)، والذي يُستخدم لإرسال البيانات إلى الإخراج القياسي (`standard output`)، والذي عادةً ما يكون الشاشة في بيئات تشغيل معينة مثل نظام التشغيل Windows أو Linux.

يُمكن استخدام `std::cout` لطباعة البيانات على الشاشة بواسطة العامل التدفقي `>>`، حيث يُمكن تحديد ما يجب طباعته بعد `>>`، ويُمكن دمج العديد من العناصر في بيان واحد. `std::endl` يُستخدم لإضافة سطر جديد بعد طباعة النص.

return 0;

في لغة ++C تعني إرجاع قيمة صفر من الدالة الحالية، وهي عادةً تُستخدم في الدالة `main()` كإشارة إلى أن تنفيذ البرنامج تم بنجاح وبدون أي مشاكل.

عندما يبدأ البرنامج تنفيذ الدالة `main()`، يمرر النظام التنفيذي (`operating system`) قيمة تشير إلى حالة نجاح أو فشل التنفيذ. إذا كانت قيمة العودة `0`، فهذا يعني أن التنفيذ تم بنجاح، وإذا كانت قيمة العودة غير صفر، فهذا يمكن أن يعني وجود مشكلة.

بشكل عام، في برنامج بسيط يتم تشغيله من سطر الأوامر (`Command Line`)، يُستخدم `return 0;` في نهاية الدالة `main()` للإشارة إلى أن البرنامج انتهى بنجاح، وهذه القيمة يمكن استخدامها من قبل النظام التشغيلي للتحقق من حالة تنفيذ البرنامج.

الأقواس

في لغة البرمجة ++C هناك عدة أنواع من الأقواس التي يمكن استخدامها:

- **الأقواس المستديرة () :** تُستخدم لتحديد معاملات الدوال والتعابير الحسابية ولتحديد أولوية التنفيذ في التعابير الرياضية. على سبيل المثال:

```
Int total= 2* (5 + 3);
```

- **الأقواس المربعة { } :** تستخدم لتحديد بلوك من الشفرة يتم تنفيذه معاً. عادةً ما تُستخدم مع الهياكل التحكمية مثل `if` و `while` و `for`. على سبيل المثال:


```
if (شرط) {  
    الشفرة التي تنفذ إذا كان الشرط صحيحًا //  
}
```

- الأقواس الزاوية <>: تُستخدم لتضمين مكتبات خارجية أو ترويسات (templates) في C++ ، على سبيل المثال:

```
#include <iostream>
```

- الأقواس الزوجية []: تُستخدم لتحديد مصفوفات والوصول إلى عناصر المصفوفة، على سبيل المثال:

```
int ارقام [5] = {1, 2, 3, 4, 5};  
int [0]_العنصر_الأول = ارقام;
```

- الأقواس المزدوجة {}: تُستخدم لتحديد النوع الذي يتم تحويل قيمة منه، وتُستخدم أيضًا في تعريف الكائنات واستدعاء الدوال (constructor) والتعديل (modifier) في الكائنات. على سبيل المثال:

```
int عدد = int(5.5);
```

هذه الأنواع الرئيسية من الأقواس التي يُمكنك استخدامها في لغة C++ ، وكل نوع له استخدامات محددة تعتمد على سياق الشفرة التي تكتبها.

- 2- قراءة البيانات من المستخدم: على سبيل المثال، يمكن استخدام `while` لطلب إدخلات من المستخدم حتى يدخل قيمة معينة تجعل الشرط غير صحيح.

```
#include <iostream>
```

```
int main() {  
    int مجموع = 0;  
    int رقم;  
    std::cout >> "الرجاء إدخال أرقام، ادخل 1- للخروج";
```



```
while (true) {  
    std::cin >> رقم;  
    if (رقم == -1) {  
        break;  
    }  
    + مجموع = رقم;  
}  
std::cout >> " >> المجموع: " >> std::endl;  
return 0;  
}
```

3- معالجة بيانات بشكل دوري: يمكن استخدام `while` لتنفيذ عمليات على بيانات متكررة حتى يتحقق شرط توقف.

```
#include <iostream>  
  
int main() {  
    int عدد = 0;  
    while (عدد < 10) {  
        std::cout >> عدد >> " ";  
        عدد = 2+عدد;  
    }  
    std::cout >> std::endl;  
    return 0;  
}
```

4- **التحكم في حلقة لا نهائية:** في حالات معينة، يمكن استخدام `while` لإنشاء حلقات تكرر لا نهائية بناءً على شروط معينة (مثل استخدام شرط دائماً صحيح).

```
#include <iostream>

int main() {
    int عدد = 1;
    while (true) {
        std::cout >> عدد >> std::endl;
        عدد++;
        if (عدد > 10) {
            break;
        }
    }
    return 0;
}
```

5- **تنفيذ تعليمات دورية:** عندما تحتاج إلى تنفيذ تعليمات بشكل دوري بناءً على حالة معينة، يمكن استخدام `while` لتحقيق ذلك.

```
#include <iostream>

int main() {
    int مضاعف = 1;
    int عدد = 3;
    while (مضاعف <= 5) {
        std::cout >> عدد * مضاعف >> std::endl;
        مضاعف++;
    }
    return 0;
}
```

هذه بعض الحالات الشائعة لاستخدام جملة `while` ، وتختلف الحالات حسب تصميم البرنامج ومتطلباته.

الأخطاء التي تحدث عند كتابة جملة `while` في C++

1. نسيان الشرط: إذا نسيتم كتابة الشرط بين القوسين في الجملة `while` ، ستحدث خطأ في البرنامج.

```
#include <iostream>

int main() {
    int counter = 0;
    while (true) {
        std::cout >> "This loop will run forever!" >> std::endl;
        counter++; // نسيان تحديث قيمة المتغير
    }
    return 0;
}
```

في هذا المثال، لا يوجد شرط محدد في الجملة `while` بالتالي، الحلقة ستستمر في التنفيذ بلا توقف. يجب أن يتم تحديث قيمة المتغير `counter` داخل الحلقة لتجنب حدوث حلقة لا نهائية.

2. **تكرار لا نهائي:** عندما يكون الشرط الموجود داخل القوسين دائمًا صحيحًا (true)، ستحدث حلقة تكرار لا نهائية وسيؤدي ذلك إلى تعليق البرنامج.

```
#include <iostream>

int main() {
    while (true) {
        std::cout >> "This loop will run forever!" >> std::endl;
        // لا يوجد تحديث للشرط هنا
    }
    return 0;
}
```

في هذا المثال، لا يوجد شرط محدد في الجملة while بالتالي، الحلقة ستستمر في التنفيذ بلا توقف. يجب أن يتم تحديث الشرط داخل الحلقة لتجنب حدوث حلقة لا نهائية.

3. **عدم تحديث المتغيرات:** إذا نسيت تحديث المتغيرات داخل جملة while، فقد تتسبب في دخول حلقة لا نهائية أو عدم انتهاء الحلقة.

```
#include <iostream>

int main() {
    int counter = 0;
    while (counter < 10) {
        std::cout >> "This loop will run forever!" >> std::endl;
        // لا يوجد تحديث لقيمة المتغير counter هنا
    }
}
```

```
}  
  
return 0;  
  
}
```

في هذا المثال، قيمة المتغير `counter` هي 0، والشرط الموجود في الجملة `while` هو `counter < 10`. وبما أن قيمة `counter` لا تتغير داخل الحلقة، فإن الحلقة ستستمر في التنفيذ بلا توقف. يجب تحديث قيمة المتغير داخل الحلقة لتجنب حدوث حلقة لا نهائية.

4. ترك الجملة فارغة: إذا كنت قد تركت الجملة داخل القوسين فارغة، فسيتم تكرار الجملة التي تلي الـ `while` بلا توقف.

```
#include <iostream>  
  
int main() {  
  
    int count = 0;  
  
    while (count < 5) {  
        // هذه الجملة فارغة  
        // يمكنك تركها بدون أي شيء داخلها  
        // أكبر من 5 count ستستمر الحلقة في التكرار حتى يصبح  
        count++;  
    }  
  
    std::cout << "انتهت الحلقة" << std::endl;  
  
    return 0;  
  
}
```

في هذا المثال، نستخدم حلقة `while` لتكرار الجملة الفارغة حتى يتم تحقيق الشرط المحدد (عندما يكون `count` أكبر من 5). ترك الجملة فارغة يعني أنها لا تحتوي على أي تعليمات، ولكن الحلقة ستستمر في التكرار حتى يتم تغيير قيمة `count`.

5. استخدام شرط غير صحيح: إذا استخدمت شرطاً غير صحيح داخل الـ `while`، فسيؤدي ذلك إلى عدم تنفيذ الجملة داخل الـ `while` أو دخول حلقة لا نهائية.

```
#include <iostream>
```

```
int main() {
```

```
    int counter = 0;
```

```
    while (counter > 10) {
```

```
        std::cout << "This loop will never execute!" << std::endl;
```

```
        counter++;
```

```
    }
```

```
    return 0;
```

```
}
```

في هذا المثال، قيمة المتغير counter هي 0، والشرط الموجود في الجملة while هو counter > 10 وبما أن قيمة counter لا تفي بالشرط (أي أنها ليست أكبر من 10)، فإن الحلقة لن تنفذ أبداً. هذا يعتبر استخداماً غير صحيحاً للشرط في الجملة while

تلك بعض الأخطاء الشائعة التي يمكن أن تحدث عند كتابة جملة `while` في لغة البرمجة C++. من المهم التحقق من الشرط وتحديث المتغيرات بشكل صحيح لتجنب هذه الأخطاء وضمان عمل الحلقة بالشكل المطلوب.

جملة while do

في C++، لا يوجد نوع خاص يُسمى "while do"، ولكن هناك جملتين منفصلتين وهما:

1. جملة `while` : تُستخدم لتنفيذ مجموعة من الأوامر بناءً على شرط معين. تكرر تنفيذ الأوامر يتوقف عندما يكون الشرط غير صحيح.
2. جملة `do-while` : تُستخدم أيضاً لتنفيذ مجموعة من الأوامر، لكنها تضمن تنفيذ الأوامر على الأقل مرة واحدة حتى لو كان الشرط غير صحيح.

يمكن توضيح الفرق بين الجملتين كالآتي

1. جملة `while` :

- تستخدم لتكرار كود معين طالما تحقق شرط معين.

- تقوم بتقييم الشرط أولاً، وإذا كان صحيحاً، يتم تنفيذ الكود داخل الحلقة.

- يتم تقييم الشرط مرة أخرى بعد كل تكرار.
- تستمر حتى يصبح الشرط غير صحيح، ثم يتم إنهاء الحلقة.

2. جملة `do...while` :

- تشبه جملة `while`، لكن الفرق الرئيسي هو أن الكود داخلها يتم تنفيذ مرة واحدة على الأقل قبل التحقق من الشرط.

- يتم تنفيذ الكود أولاً، ثم يتم تقييم الشرط.
- إذا كان الشرط صحيحاً، يتم تنفيذ الكود مرة أخرى.
- يستمر حتى يصبح الشرط غير صحيح.
- هذا هو مثال على جملة `do...while`

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// هنا قمنا بتعريف المتغير الذي استخدمناه كعداد في الحلقة
```

```
int i=1;
```

```
هنا أنشأنا حلقة while // نظل ننفذ الأوامر الموضوعة فيها طالما أن قيمة العدد لا تزال أصغر أو تساوي 10
```

```
do
```

```
{
```

```
// في كل دورة سيتم طباعة قيمة العداد ثم إضافة 1 عليها
```

```
cout >> i >> endl;
```

```
i++ ;
```

```
}
```

```
while( i<=10 );
```

```
return 0;
```

```
}
```

عند التشغيل سيتم طباعة الأرقام من 1 إلى 10 رقم في كل سطر



الأسئلة الاسترشادية

- 1- اذكر الغرض استخدام جملة while
- 2- اذكر ثلاثة من الحالات التي يتم فيها استخدام جملة while
- 3- ماذا يعني استخدام return 0 في الدالة الرئيسية
- 4- اذكر امثلة على الاقواس في C++ واستخدام كل منها
- 5- اذكر ثلاثة من الأخطاء التي يمكن ان تحدث عند استخدام الجملة while
- 6- ماهو الفرق الأساسي بين while و do-while

المحاضرة السابعة

الحلقات في لغة C++

الحلقة في C++ هي هيكل تحكم يسمح لك بتكرار تنفيذ كتلة من الكود عدة مرات حسب شرط معين أو قيمة متغير. هذا يساعد في تنظيم وتكرار العمليات بشكل متكرر دون الحاجة لكتابة الكود بشكل متكرر. تعتبر الحلقات جزءًا أساسيًا من أي لغة برمجة لأنها تسمح بتنظيم البرامج وتكرار العمليات بطريقة فعالة.

التعريف العام للحلقة في C++ هو:

- تحديد نوع الحلقة مثل for، while، أو do-while.
- تعيين متغيرات التحكم إذا كانت مطلوبة.
- تحديد الشرط الذي يتم التحقق منه لاستمرار التكرار.
- تحديد الكود الذي يتم تكرار تنفيذه.

في لغة C++، هناك ثلاثة أنواع رئيسية للحلقات:

حلقة for:

تستخدم لتكرار تنفيذ كتلة من الكود لعدد محدد من المرات.

تتضمن عادةً متغير التحكم (counter variable)، وشرط للتحقق من استمرار التكرار، وخطوة تحديث قيمة المتغير.

```
#include <iostream>
```

```
int main() { for(int i = 0; i < 5; ++i) {
```

```
    std::cout << i << " ";
```

```
}
```

```
std::cout << std::endl;
```

```
return 0;
```

```
}
```

حلقة while:

تستخدم لتكرار تنفيذ كتلة من الكود طالما تستمر الشرط المعطى في التحقق (true).

لا تحتاج إلى تعيين متغير تحكم في البداية، ويتم التحقق من الشرط قبل تنفيذ الكود داخل الحلقة.

```
int i = 0;
```

```
while(i < 5) {
```

```
    std::cout << i << " ";
```

```
    ++i;}
```

حلقة: do-while:

تستخدم لتكرار تنفيذ كتلة من الكود على الأقل مرة واحدة، ثم يتم التحقق من الشرط لاستمرار التكرار. تشبه حلقة while، لكن الفرق هو أنه يتم تنفيذ الكود على الأقل مرة واحدة حتى لو كان الشرط غير صحيح.

```
int i = 0;  
do {  
    std::cout << i << " "; ++i; }  
while(i < 5);
```

الجملة continue

نستخدم الجملة continue لتجاوز تنفيذ كود معين في الحلقة، إذاً نستخدمها لتجاوز جزء من كود الـ scope. و نستخدمها تحديداً لإيقاف الدورة الحالية و الإنتقال إلى الدورة التالية في الحلقة، لا تقلق سنفهم المقصود من المثال.

طريقة تعريفها

تتألف هذه الجملة من أمر واحد و يكتب على سطر منفرد.

```
continue;
```

أمثلة حول جملة التحكم continue

في المثال التالي قمنا بتعريف حلقة تطبع جميع الأرقام من 1 إلى 10 ما عدا الرقم 3. تم استخدام الجملة continue لجعل الحلقة تتجاوز الدورة الثالثة في الحلقة. أي لن يتم تنفيذ أمر الطباعة عندما تصبح قيمة العداد i تساوي 3.

المثال الأول

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

تتألف من 10 دورات. في كل دورة تطبع قيمة العداد المستخدم فيها //
هنا قمنا بإنشاء حلقة for

```
for (int i=1; i<=10; i++)
```

```
{
```

// في كل دورة سيتم فحص قيمة العداد، عندما تصبح تساوي 3 سيتم
الانتقال إلى الدورة التالية في الحلقة بدون تنفيذ أمر الطباعة
الموضوع بعدها

```
if (i == 3) {
```

```
continue;
```

```
}
```

```
cout << i << endl;
```

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

1

2

4

5

6

7

8

9

10

إذاً الجملة `continue` جعلت الحلقة تتجاوز الدورة الثالثة، لذلك لم تطبع الرقم 3 لأنها لم تنفذ أمر الطباعة في الدورة الثالثة.

(`using namespace std;` في C++ هو تعليمة تُستخدم لجعل جميع المكونات في مساحة الأسماء "std" التي تشير إلى المكتبة القياسية في C++ متاحة مباشرة دون الحاجة إلى استخدام البادئة "std::" قبل كل مكون.

عندما تقوم بإضافة `using namespace std;` إلى بداية برنامج C++ الخاص بك، يمكنك استخدام أسماء الكائنات والدوال المعرفة في مكتبة الـ (std) Standard C++ مباشرة دون الحاجة إلى تحديد أنها تنتمي إلى مساحة الأسماء std. على سبيل المثال، بدون استخدام `using namespace std;`، يمكن أن يكون عليك كتابة `std::cout` لاستخدام الدالة `cout` الموجودة في مكتبة C++ القياسية، ولكن مع استخدام `using namespace std;`، يمكنك استخدام `cout` مباشرة دون البادئة "std::".

من الجيد استخدام `using namespace std;` في البرامج الصغيرة أو التطبيقات التعليمية للحد من الكود، ولكن في برامج كبيرة أو مشاريع مهمة، يُفضل عادةً تجنب استخدامها لتجنب تضارب الأسماء وتوضيح مصدر كل اسم.)

في المثال التالي قمنا بتعريف حلقة تطبع جميع الأرقام المفردة من 1 إلى 10. استخدمنا الجملة `continue` لجعل الحلقة تتجاوز كل دورة تكون فيها قيمة العداد `i` عبارة عن عدد زوجي.

المثال الثاني

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// تتألف من 10 دورات. في كل دورة تطبع قيمة العداد المستخدم فيها  
// هنا قمنا بإنشاء حلقة for
```

```
for (int i=1; i<=10; i++)
```

```
{
```

```
// في كل دورة سيتم فحص قيمة العداد، في حال كانت مزدوجة سيتم  
// الانتقال إلى الدورة التالية في الحلقة بدون تنفيذ أمر الطباعة  
الموضوع بعدها
```

```
if (i%2 == 0) {
```

```
continue;
```

```
}  
  
cout << i << endl;  
  
}  
  
return 0;  
  
}
```

سنحصل على النتيجة التالية عند التشغيل.

```
1  
  
3  
  
5  
  
7  
  
9
```

الحالات التي تستخدم فيها جملة `continue`

يُستخدم الأمر `continue` في لغة C++ للقفز إلى بداية الحلقة الحالية دون استكمال تنفيذ باقي الكود فيها. هذا الأمر يُستخدم للتحكم في تنفيذ الحلقات، ويمكن استخدامه في حالات معينة مثل:

- تجاوز الخطوات غير المرغوب فيها :عندما يكون هناك شرط محدد يتعلق بتجاوز بعض الخطوات داخل الحلقة، يمكن استخدام `continue` لتجاوز هذه الخطوات.

```
#include <iostream>  
using namespace std;
```

```
int main() {  
    for (int i = 1; i <= 5; i++) {  
        تساوي 3 i تجاوز الخطوة إذا كانت //  
        if (i == 3) {  
            continue;  
        }  
        cout << " الخطوة " << i << endl;  
    }  
}
```

```
return 0;  
}
```

في هذا المثال، عندما يكون i يساوي 3، سيتم تجاوز الخطوة والتنتقل مباشرة إلى الخطوة التالية. يمكنك استخدام `continue` لتجاوز أي خطوة غير مرغوب

- **التأكد من شروط معينة :** يمكن استخدام `continue` للتحقق من شروط معينة داخل الحلقة وتخطي تنفيذ باقي الكود إذا تحققت هذه الشروط.

```
#include <iostream>  
int main() {  
    for(int i = 1; i <= 5; ++i) {  
        if(i % 2 == 0) {  
            // تجاوز الأرقام الزوجية  
            continue;  
        }  
        std::cout << i << " ";  
    }  
    std::cout << std::endl;  
    return 0;  
}
```

في هذا المثال، يتم استخدام حلقة `for` لطباعة الأرقام من 1 إلى 5، ولكن باستخدام الشرط `if(i % 2 == 0)`، يتم تجاوز الأرقام الزوجية باستخدام جملة `continue`، وبالتالي لا تُطبع الأرقام الزوجية.

- **تجاوز قيم معينة :** في حالة وجود قيم معينة يجب تجاوزها دون تنفيذ الكود المتبقي في الحلقة.

```
#include <iostream>  
int main() {  
    int numbers[] = {1, 2, 3, 4, 5};  
    for(int i = 0; i < 5; ++i) {  
        if(numbers[i] == 3) {  
            // تجاوز القيمة 3  
            continue;  
        }  
    }
```

```
std::cout << numbers[i] << " ";  
}  
std::cout << std::endl;  
return 0;  
}
```

في هذا المثال، يتم استخدام حلقة for للتكرار عبر مصفوفة من الأرقام. عندما تكون القيمة في المصفوفة تساوي 3، يتم استخدام جملة continue لتجاوز هذه القيمة وعدم طباعتها، وبالتالي يتم طباعة القيم الأخرى (1، 2، 4، 5)

الجملة break

الجملة break تستخدم فـي الحلقات و فـي الجملـة switch. بمجرد ان تنفذ الجملة break فإنها توقف الـ scope بأكمله و تخرج منه و تمسحه من الذاكرة ثم تنتقل للكود الذي يليه في البرنامج.

طريقة تعريفها

تتألف هذه الجملة من أمر واحد و يكتب على سطر منفرد.

```
break;
```

مثال حول جملة التحكم break

في المثال التالي قمنا بتعريف حلقة كانت ستطبع جميع الأرقام من 1 إلى 10 لولا أننا استخدمنا الجملة break لجعل الحلقة تتوقف عندما تصبح قيمة العداد i تساوي 6.

مثال

```
#include <iostream>  
  
using namespace std;  
  
int main(){
```


// تتألف من 10 دورات. في كل دورة تطبع قيمة العداد المستخدم فيها
for هنا قمنا بإنشاء حلقة

```
for( int i=1; i<=10; i++ )
```

```
{
```

// في كل دورة سيتم فحص قيمة العداد و بمجرد أن تصبح تساوي 6 سيتم
إيقاف الحلقة نهائياً

```
if( i == 6 ) { break;
```

```
}
```

```
cout << i << endl;
```

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

1

2

3

4

5

إذاً الجملة break جعلت الحلقة تتوقف عندما أصبحت قيمة العداد i تساوي 6.

الحلقة for

نستخدم الحلقة for إذا كنا نريد تنفيذ الكود عدة مرات محددة، فمثلاً إذا كنا نريد تنفيذ كود معين 10 مرات، نضعه بداخل حلقة تعيد نفسها 10 دورات.

طريقة إستخدامها

```
for( initialization; condition; increment أو decrement )  
{  
    // statements  
}
```

- **initialization** هي أول خطوة تنفذ في الحلقة و هي تنفذ مرة واحدة فقط على عكس جميع العناصر الموجودة في الحلقة. في هذه الخطوة نقوم بتعريف متغير (يسمى عداد) و نضع بعده ;
- **condition** هي ثاني خطوة تنفذ في الحلقة و هي تنفذ في كل دورة. في هذه الخطوة نقوم بوضع شرط يحدد متى تتوقف الحلقة، في كل دورة يتم التأكد أولاً إذا تحقق هذا الشرط أم لا، و نضع بعده ; هنا طالما أن نتيجة الشرط تساوي true سيعيد تكرار الكود.
- **statements** هي الخطوة الثالثة، و تعني تنفيذ جميع الأوامر الموجودة في الحلقة و هي تنفذ في كل دورة. بعد أن تنفذ جميع الأوامر سيعود إلى الخطوة الأخيرة التي تحدث في نهاية كل دورة و هي إما زيادة قيمة العداد أو إنقاصها.
- **increment أو decrement** هي الخطوة الرابعة و الأخيرة، و هي تنفذ في كل دورة. هنا نحدد كيف تزداد أو تنقص قيمة العداد، و لا نضع بعده ;

تذكر فقط أن جميع هذه الخطوات تتكرر في كل دورة ما عدا أول خطوة، و السبب أننا لا نحتاج إلى تعريف عداد جديد في كل دورة، بل نستعمل العداد القديم و الذي من خلاله نعرف في أي دورة أصبحنا.

مثال حول الحلقة for

في المثال التالي قمنا بتعريف حلقة تطبع جميع الأرقام من 1 إلى 10.

مثال

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// تتألف من 10 دورات. في كل دورة تطبع قيمة العداد المستخدم فيها  
for هنا قمنا بإنشاء حلقة
```

```
for( int i=1; i<=10; i++ )
```

```
{
```

```
cout << i << endl;
```

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

1

2

3

4

5

6

7

8

9

10

الجملة المتداخلة nested for

في لغة C++ ، الجملة المتداخلة nested for تعني استخدام حلقتين for داخل بعضهما البعض، أي تضع إحدى الحلقات داخل الأخرى. هذا يسمح بتكرار تنفيذ كود محدد بشكل متكرر حسب الحلقتين. مثال على جملة nested for في C++ هو:

```
#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) {
        for (int j = 1; j <= 3; ++j) {
            std::cout << "Hello from outer loop " << i << " and inner loop " << j << std::endl;
        }
    }
    return 0;
}
```

في هذا المثال، يتم تضمين حلقتين for متداخلتين. الحلقة الخارجية تتكرر 5 مرات، والحلقة الداخلية تتكرر 3 مرات. سيتم طباعة رسالة لكل تكرار داخل الحلقة الداخلية والحلقة الخارجية.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة من أنواع الحلقات المستخدمة في C++
- 2- اذكر الغرض من استخدام الجملة continue
- 3- ثلاثة من الحالات التي تستخدم فيها الجملة continue
- 4- اذكر الغرض من استخدام الجملة break
- 5- ماذا تعني الجملة المتداخلة nested for

المحاضرة الثامنة

المصفوفات في C++

المصفوفة (Array) عبارة عن متغير واحد يتألف من عدة عناصر (Elements) من نفس النوع. و كل عنصر في المصفوفة يمكن تخزين قيمة واحدة فيه.

عناصر المصفوفة تتميز عن بعضها من خلال رقم محدد يعطى لكل عنصر يسمى index. أول عنصر في المصفوفة دائماً يكون رقمه 0.

الآن، عليك معرفة أن عدد عناصر المصفوفة ثابت، أي بمجرد أن قمت بتحديد لا يمكنك تغييره من جديد، مع الإشارة إلى أنك تستطيع تغيير قيم هذه العناصر متى شئت.

فوائد المصفوفات

- تقليل عدد المتغيرات المتشابهة، فمثلاً إذا كنا نريد تعريف 10 متغيرات نوعهم int، نقوم بتعريف مصفوفة واحدة تتألف من 10 عناصر.
- التعامل مع الكود يصبح أسهل، لأنك إذا قمت بتخزين المعلومات داخل مصفوفة، تستطيع تعديلهم، مقارنتهم أو جلبهم كلهم دفعة واحدة بكود صغير جداً باستخدام حلقة.
- تستطيع الوصول لأي عنصر من خلال رقم الـ index الخاص به.

تعريف مصفوفة في C++

هناك ثلاث طرق يمكنك اتباعها لتعريف مصفوفة (Declare Array) جديدة سنتعرف عليها تباعاً.

الأسلوب التالي يستخدم لتعريف مصفوفة مع تحديد عدد عناصرها //

```
datatype arrayName[size];
```

الأسلوب التالي يستخدم لتعريف مصفوفة مع تحديد قيمها الأولية //

```
datatype arrayName[] = {value1, value2, ..};
```

الأسلوب التالي يستخدم لتعريف مصفوفة مع تحديد عدد عناصرها و قيمة بعض عناصرها //

```
datatype arrayName[size] = {value1, value2, ..};
```

- **datatype** هو نوع القيم التي يمكن تخزينها في عناصر المصفوفة.
- **size** هو عدد عناصر المصفوفة.
- **arrayName** هو اسم المصفوفة.
- **[]** هذا الرمز يمثل من كم بعد تتألف المصفوفة.

أمثلة حول طريقة تعريف مصفوفة أحادية (One Dimensional Array).
أمثلة

هنا قمنا بتعريف مصفوفة ذات بعد واحد اسمها arr, نوعها int و تتألف من 5 عناصر //

```
int arr[5];
```

هنا قمنا بتعريف مصفوفة ذات بعد واحد اسمها arr نوعها int ولم نحدد عدد عناصرها لكننا لما وضعنا فيها 6 عناصر أصبحت تتألف من 6 عناصر

```
int arr[] = {1, 2, 3, 4, 5, 6};
```

هنا قمنا بتعريف مصفوفة ذات بعد واحد اسمها arr نوعها int وتتكون من 5 عناصر وقمنا بتعريف 3 عناصر منها

```
int arr[5] = {1, 2, 3};
```

أمثلة حول طريقة تعريف مصفوفة ثنائية (Two Dimensional Array).
أمثلة

هنا قمنا بتعريف مصفوفة اسمها arr ونوعها int ذات بعدين وتتألف من 4×3 عناصر

```
int arr[4][3];
```

هنا قمنا بتعريف مصفوفة اسمها arr ونوعها int ذات بعدين وتتألف من 4×3 عناصر ووضعنا بها 6 قيم

```
int arr[4][3] = {
```

```
{1, 2, 3},
```

{4, 5, 6}

};

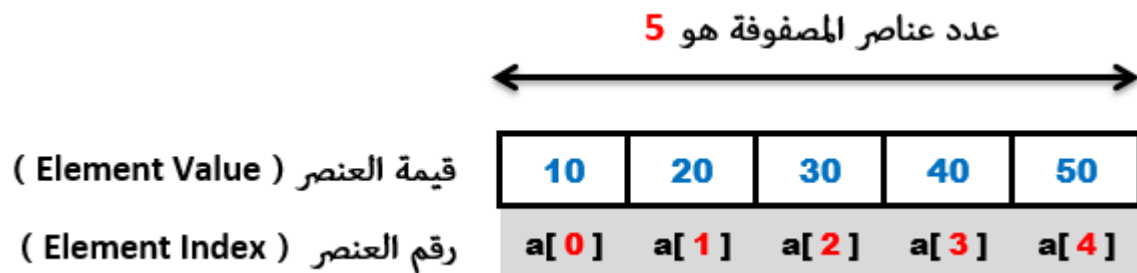
عند تعريف مصفوفة ثنائية الأبعاد أو متعددة الأبعاد فإنه يجب تحديد عدد عناصرها أثناء تعريفها.

الوصول لعناصر المصفوفة في C++

لنفترض الآن أننا قمنا بتعريف مصفوفة نوعها `int`، إسمها `a`، و تتألف من 5 عناصر.

`int a[] = { 10, 20, 30, 40, 50 };`

يمكنك تصور شكل المصفوفة `a` في الذاكرة كالتالي.



بما أن المصفوفة تتألف من 5 عناصر، تم إعطاء العناصر أرقام `indexes` بالترتيب من 0 إلى 4.

إذاً هنا أصبح عدد عناصر المصفوفة يساوي 5 وهو ثابت لا يمكن تغييره لاحقاً في الكود. وللوصول لقيمة أي عنصر نستخدم `index` العنصر الذي تم إعطاؤه له.

في المثال التالي، قمنا بتعريف مصفوفة، ثم غيرنا قيمة العنصر الأول فيها، و من ثم عرضنا قيمة جميع العناصر.

مثال

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

// هنا قمنا بتعريف مصفوفة تتألف من 5 عناصر //

```
int arr[] = {10, 20, 30, 40, 50};
```

// هنا قمنا بتغيير قيمة العنصر الأول و العنصر الأخير في المصفوفة //

```
arr[0] = 1;
```

```
arr[4] = 5;
```

// هنا قمنا بعرض قيم جميع عناصر المصفوفة //

```
cout << "arr[0] = " << arr[0] << endl;
```

```
cout << "arr[1] = " << arr[1] << endl;
```

```
cout << "arr[2] = " << arr[2] << endl;
```

```
cout << "arr[3] = " << arr[3] << endl;
```

```
cout << "arr[4] = " << arr[4] << endl;
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

```
arr[0] = 1
```

```
arr[1] = 20
```

```
arr[2] = 30
```

```
arr[3] = 40
```

```
arr[4] = 5
```


طريقة وضع قيم أولية لعناصر المصفوفة في C++

إذا قمت بإنشاء مصفوفة جديدة مع تحديد عدد عناصرها فقط و بدون إعطائها قيم أولية، من المحتمل أن تجد قيم غريبة في بعض عناصرها. سبب ذلك أن هذه القيم كانت موجودة مسبقاً في الذاكرة لا أكثر. لذا في حال أردت إنشاء مصفوفة جديدة مع حذف أي قيم افتراضية قد تكون موجودة فيها، يجب أن تقوم بتمرير القيمة الافتراضية التي تريد وضعها لعناصر المصفوفة لحظة إنشائها.

في المثال التالي، قمنا بتعريف مصفوفة تتألف من 5 عناصر و لم نعطها قيم أولية، ثم قمنا بعرض القيم الافتراضية الموجودة فيها.

المثال الأول

```
#include <iostream>

using namespace std;

int main()
{
    // هنا قمنا بتعريف مصفوفة تتألف من 5 عناصر
    int arr[5];

    // هنا قمنا بعرض القيم الافتراضية الموجودة في عناصر المصفوفة
    cout << "arr[0] = " << arr[0] << endl;
    cout << "arr[1] = " << arr[1] << endl;
    cout << "arr[2] = " << arr[2] << endl;
    cout << "arr[3] = " << arr[3] << endl;
    cout << "arr[4] = " << arr[4] << endl;

    return 0;
}
```

عند تشغيل البرنامج حصلنا على نتيجة غريبة حيث وجدنا قيم افتراضية في بعض العناصر.

`arr[0] = 8`

`arr[1] = 0`

`arr[2] = 42`

`arr[3] = 0`

`arr[4] = 15275776`

ملاحظة: من الطبيعي أن لا تظهر لك نفس النتيجة التي ظهرت لنا لأن هذه القيم هي قيم عشوائية.

في المثال التالي قمنا بوضع القيمة 0 كقيمة أولية لجميع عناصر المصفوفة، ثم قمنا بعرض قيمها.

المثال الثاني

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// هنا قمنا بتعريف مصفوفة تتألف من 5 عناصر مع تحديد أن القيمة  
الإفتراضية في جميع عناصرها هي 0
```

```
int arr[5] = {0};
```

```
// هنا قمنا بعرض القيم الإفتراضية الموجودة في عناصر المصفوفة
```

```
cout << "arr[0] = " << arr[0] << endl;
```

```
cout << "arr[1] = " << arr[1] << endl;
```

```
cout << "arr[2] = " << arr[2] << endl;
```

```
cout << "arr[3] = " << arr[3] << endl;
```

```
cout << "arr[4] = " << arr[4] << endl;
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند تشغيل البرنامج.

```
arr[0] = 0
```

```
arr[1] = 0
```

```
arr[2] = 0
```

```
arr[3] = 0
```

```
arr[4] = 0
```

طريقة معرفة عدد عناصر المصفوفة في C++

إذا أردت معرفة عدد عناصر أي مصفوفة، يمكنك الحصول عليه من خلال قسمة حجم المصفوفة، على نوع العناصر المخزنة فيها.

مثال

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// هنا قمنا بتعريف مصفوفة تتألف من 5 عناصر حيث وضعنا فيها 5 قيم  
رقمية و كل قيمة سيتم وضعها في عنصر
```

```
int arr[] = {1, 2, 3, 4, 5};
```

```
// على حجم نوع أول عنصر فيها، و من ثم قمنا بتخزين الناتج في n  
هنا قمنا بقسمة عدد عناصر المصفوفة arr المتغير
```

```
int n = sizeof(arr) / sizeof(arr[0]);
```

```
// هنا قمنا بطباعة عدد عناصر arr الذي قمنا بتخزينه في المتغير n  
المصفوفة
```

```
cout << "Number of elements in the array is: " << n;
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند تشغيل البرنامج.

Number of elements in the array is: 5

و هذه بعض الطرق الأخرى التي قد تجد أنها تستخدم لمعرفة أحجام المصفوفات.
يمكنك الحصول على عدد عناصر المصفوفة مهما كان نوعها من خلال الأسلوب التالي

sizeof(arrayName) / sizeof(arrayElement[0])

يمكنك أن تستخدم الأسلوب التالي **int** إذا كان نوع المصفوفة هو

sizeof(arrayName) / sizeof(int)

يمكنك أن تستخدم الأسلوب التالي **string** إذا كان نوع المصفوفة هو

sizeof(arrayName) / sizeof(string)

الأسلوب الذي استخدمناه لمعرفة عدد عناصر المصفوفة يمكن تطبيقه على أي نوع بيانات آخر و لكن لا يمكن استخدامه مع المؤشرات (Pointers) و التي سنتعرف عليها في دروس لاحقة.

التعامل مع المصفوفة بواسطة حلقة في C++

عند التعامل مع المصفوفات فإنك على الأغلب ستستخدم حلقة للمرور على قيمها سواء للبحث عن قيمة فيها، تحديث قيمها، أو لمجرد طباعة القيم الموجودة فيها.

في المثال التالي افترضنا أن عدد عناصر المصفوفة التي سنعرض قيمها معروف.

المثال الأول

```
#include <iostream>

using namespace std;

int main()
{
    // هنا قمنا بإنشاء مصفوفة تحتوي على 3 قيم نصية
    string fruits[3] = {"Apple", "Banana", "Orange"};

    // هنا قمنا بإنشاء حلقة، في كل دورة تقوم بعرض fruits على سطر جديد
    // قيمة من القيم الموجودة في المصفوفة
    for (int i=0; i<3; i++)
    {
        cout << fruits[i] << endl;
    }

    return 0;
}
```

سنحصل على النتيجة التالية عند التشغيل.

Apple

Banana

Orange

في المثال التالي إفتراضنا أن عدد عناصر المصفوفة التي سنعرض قيمها غير معروف.

المثال الثاني

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// هنا قمنا بإنشاء مصفوفة تحتوي على 3 قيم نصية
```

```
string fruits[] = {"Apple", "Banana", "Orange"};
```

```
// هنا قمنا بحساب عدد عناصر fruits و من ثم تخزينه في المتغير n  
المصفوفة
```

```
int n = sizeof(fruits) / sizeof(fruits[0]);
```

```
// هنا قمنا بإنشاء حلقة، في كل دورة تقوم بعرض fruits على سطر جديد  
قيمة من القيم الموجودة في المصفوفة
```

```
for (int i=0; i<n; i++)
```

```
{
```

```
cout << fruits[i] << endl;
```

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

Apple

Banana

Orange

الحلقة foreach في C++

إبتداءً من إصدار المترجم C++ 11 تم إضافة حلقة for جديدة إسمها Foreach Loop. هذه الحلقة تسمح لك بالمرور على جميع عناصر المصفوفة دون الحاجة لتعريف عداد و تحديد أين يبدأ و أين ينتهي.

طريقة تعريف الحلقة Foreach

```
for (element: array)
{
    // statements
}
```

- **element** هو متغير عادي نقوم بتعريفه بداخل الحلقة و نعطيه نفس نوع المصفوفة التي نضعها بعد النقطتين، لأنه في كل دورة سيقوم بتخزين قيمة عنصر من عناصرها، لذلك يجب جعل نوعه مثل نوعها.
 - **array** هي المصفوفة التي نريد الوصول لجميع عناصرها.
 - **statements** هي جميع الأوامر الموضوعه في الحلقة و هي تنفذ في كل دورة.
- إذاً هنا نقوم الحلقة بالمرور على جميع عناصر المصفوفة بالترتيب من العنصر الأول إلى العنصر الأخير، و في كل دورة نقوم بتخزين قيمة العنصر في المتغير الذي قمنا بتعريفه.
- سنقوم الآن بكتابة برنامج بسيط يعرض قيم جميع عناصر مصفوفة باستخدام الحلقة Foreach.

مثال

```
#include <iostream>

using namespace std;

int main()
{
    // هنا قمنا بإنشاء مصفوفة تحتوي على 3 قيم نصية //

    string fruits[] = {"Apple", "Banana", "Orange"};

    // هنا في كل دورة سيتم تخزين قيمة عنصر من fruits في المتغير element
    // عناصر المصفوفة

    for (string element: fruits)
    {
        // هنا سيتم عرض القيمة التي تخزنت في المتغير element

        cout << element << endl;
```



معهد رسل الحضارة الدولي
للعلوم التقنية والفنية

```
}
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل

Apple

Banana

Orange

الأسئلة الاسترشادية

- 1- اذكر استخدام المصفوفات في C++
- 2- قم بتعريف مصفوفة ثنائية بها ثلاثة أعمدة وأربعة صفوف من الأعداد الصحيحة (يمكنك اختيار اسمها)
- 3- قم بتعريف مصفوفة ثنائية بها أربعة أعمدة وخمسة صفوف لتخزين أسماء الطلبة (يمكنك اختيار اسمها)
- 4- اكتب الغرض من استخدام الحلقة Foreach Loop
- 5- اكتب امر الوصول للعنصر الثاني في المصفوفة `int arr[] = {10, 20, 30, 40, 50};`

المحاضرة التاسعة

الدوال في C++

الدالة (Function) عبارة عن مجموعة أوامر مجمعة في مكان واحد و تنفذ عندما نقوم باستدعائها. في الدروس السابقة تعرفنا على الكثير من العديد من الدوال الجاهزة في C++ و التي تستخدم للتعامل مع النصوص و الأرقام.

أمثلة حول الدوال الجاهزة

length();

هي دالة تستخدم في لغة C++ لحساب طول سلسلة الحروف (النص)، وتستخدم عادة مع سلاسل الحروف (strings) في الواقع، في لغة C++ ، يُفضل استخدام دالة size() بدلاً من length() لحساب طول السلسلة.

insert();

هي دالة مدمجة في لغة C++ تُستخدم لإدراج عناصر جديدة قبل العنصر في الموضع المحدد، مما يزيد حجم الحاوية بعدد العناصر المُدخلة. هذه الدالة مفيدة لإدراج عناصر داخل حلقة for أو لتعديل الحاويات مثل الـ vectors والـ sets والـ strings.

replace();

هي دالة مدمجة في لغة C++ تُستخدم لاستبدال جزء من سلسلة الحروف (النص) بسلسلة حروف أخرى محددة. يمكن استخدامها لتغيير جزء معين من النص بقيمة جديدة. هذه الدالة تكون مفيدة عند تعديل السلاسل أو استبدال أجزاء غير مرغوب فيها.

fmax();

هي دالة مدمجة في لغة C++ تُستخدم لإيجاد القيمة الأكبر بين اثنين. تُستخدم عادةً مع الأرقام العائمة (الـ float والـ double)، وتُعيد القيمة الأكبر من بين القيمتين المُدخلتين.

floor();

هي دالة مدمجة في لغة C++ تُستخدم لإيجاد أكبر قيمة صحيحة أقل من أو تساوي العدد المُدخل. عادةً ما تُستخدم مع الأرقام العائمة الـ float والـ double ، وتُعيد القيمة الأكبر من بين القيمتين المُدخلتين. مصطلحات تقنية

الدوال الجاهزة في C++

يقال لها Built-in Functions. الدوال التي يقوم المبرمج بتعريفها يقال لها User-defined Functions

الدوال في لغة C++ تأتي في نوعين رئيسيين: الدوال المعرفة مسبقاً والدوال التي يتم تعريفها من قبل المبرمج. دعونا نلقي نظرة على بعض الدوال الجاهزة المهمة في: C++

- **cout** تُستخدم لطباعة النصوص على الشاشة.

- **cin** تُستخدم لقراءة الإدخالات من المستخدم.
- **sqrt** تُستخدم لحساب الجذر التربيعي لعدد.
- **rand** تُستخدم لإنشاء أرقام عشوائية.
- **abs** تُستخدم للحصول على القيمة المطلقة لعدد.
- **max(a, b)** تُعيد القيمة الأكبر بين a و b.
- **min(a, b)** تُعيد القيمة الأصغر بين a و b.
- **toupper(c)** تُحول الحرف c إلى حرف كبير.
- **tolower(c)** تُحول الحرف c إلى حرف صغير.

بناء الدوال في C++

عند تعريف أي دالة في C++ عليك إتباع الشكل التالي:

```
returnType functionName(Parameter)
{
    // Function Body
}
```

returnType: يحدد النوع الذي سترجعه الدالة عندما تنتهي أو إذا كانت لن ترجع أي قيمة.

functionName: يمثل الاسم الذي نعطيه للدالة، و الذي من خلاله يمكننا استدعاءها.

Parameter: المقصود بها الباراميترات (وضع الباراميترات إختياري).

Function Body: تعني جسم الدالة، و المقصود بها الأوامر التي نضعها في الدالة.

نوع الإرجاع (returnType) في الدالة يمكن أن يكون أي نوع من أنواع البيانات الموجودة في C++ (int - double - bool - string إلخ..). و يمكن وضع إسم لكلاس معين، و هنا يكون القصد أن الدالة ترجع كائن من هذا الكلاس (لا تقلق سنتعلم هذا في دروس لاحقة). في حال كانت الدالة لا ترجع أي قيمة، يجب وضع الكلمة void مكان الكلمة returnType.

أمثلة حول تعريف دوال جديدة في C++

في المثال التالي قمنا بتعريف دالة إسمها myFunction، نوعها void، و تحتوي على أمر طباعة فقط. بعدها قمنا باستدعائها في الدالة main() حتى يتم تنفيذ أمر الطباعة الموضوع فيها.

المثال الأول

```
#include <iostream>

using namespace std;

// هنا قمنا بتعريف دالة myFunction عند استدعاءها تقوم بطباعة جملة //
// إسمها
void myFunction() {

    cout << "My first function is called";

}

int main()

{

    // هنا قمنا باستدعاء myFunction() حتى يتنفذ الأمر الموضوع فيها //
    // الدالة
    myFunction();

    return 0;

}
```

سنحصل على النتيجة التالية عند التشغيل.

My first function is called

هنا قمنا بتعريف دالة إسمها greeting، عند إستدعاءها نمرر لها إسم فتطبع رسالة ترحيب للإسم الذي تم تمريره لها.

المثال الثاني

```
#include <iostream>

using namespace std;

// هنا قمنا بتعريف دالة greeting عند استدعاءها تقوم بطباعة جملة //
// إسمها
```

```
void greeting(string name)
```

```
{
```

```
cout << "Hello " << name << ", welcome to our company.";
```

```
}
```

```
int main()
```

```
{
```

```
// هنا قمنا باستدعاء الدالة greeting() حتى يتنفذ الأمر الموضوع فيها
```

```
greeting("Mhamad");
```

```
return 0;
```

```
}
```

سنحصل على النتيجة التالية عند التشغيل.

Hello Mhamad, welcome to our company.

هنا قمنا بتعريف دالة إسمها getSum، عند إستدعاءها نمرر لها عددين فترجع لنا ناتج جمعهما.

المثال الثالث

```
#include <iostream>
```

```
using namespace std;
```

```
// get_sum عند إستدعاءها نمرر لها عددين فتقوم بإرجاع ناتج جمعهما
```

```
هنا قمنا بتعريف دالة إسمها
```

```
int getSum(int a, int b)
```

```
{
```

```
return a + b;
```



```
}  
  
int main()  
{  
    // هنا قمنا بتخزين ناتج العددين 3 و 5 الذي get_sum() في المتغير x  
    // سترجعه الدالة  
  
    int result = getSum(3, 7);  
  
    // هنا قمنا بعرض قيمة المتغير result و التي ستساوي 10  
  
    cout << "Result = " << result;  
  
    return 0;  
}
```

سنحصل على النتيجة التالية عند التشغيل.

Result = 10

إعطاء قيمة افتراضية للباراميترات في C++

C++ تتيح لك وضع قيم افتراضية للباراميترات مما يجعلك عند استدعاء الدالة مخيّر على تمرير قيم مكان الباراميترات بدل أن تكون مجبراً على ذلك.

مصطلحات تقنية

القيمة الافتراضية التي نضعها للباراميتر يقال لها Default Argument.

ففي المثال التالي قمنا بتعريف دالة اسمها printLanguage. هذه الدالة فيها باراميتر واحد اسمه language يملك النص "English" كقيمة افتراضية. كل ما تفعله هذه الدالة عند استدعاءها هو طباعة قيمة الباراميتر language.

ملاحظة: بما أن الباراميتر language يملك قيمة بشكل افتراضية، فهذا يعني أنك لم تعد مجبر على تمرير قيمة له عند استدعاء الدالة لأنه أصلاً يملك قيمة.

مثال

```
#include <iostream>
```

using namespace std;

// عند إستدعاء ها language و يمكنك عدم تمرير قيمة لأنه أصلاً يملك قيمة
هنا قمنا printLanguage. يمكنك تمرير قيمة لها مكان الباراميتير
بتعريف دالة إسمها

void printLanguage(string language="English")

{

cout << "Your language is " << language << endl;

}

int main()

{

// بدون تمرير قيمة مكان language و بالتالي ستظل قيمته "English"
هنا قمنا باستدعاء الدالة printLanguage() الباراميتير

printLanguage();

// مع تمرير 'Arabic' للباراميتير language و بالتالي ستصبح قيمته "Arabic"
هنا قمنا باستدعاء الدالة printLanguage() القيمة

printLanguage("Arabic");

return 0;

}

سنحصل على النتيجة التالية عند التشغيل.

Your language is English

Your language is Arabic

إذا كانت الدالة تملك أكثر من باراميتير و تريد وضع قيمة إفتراضية لأحد الباراميتيرات التي تمكّلها فقط فيجب وضع الباراميتيرات التي تملك قيم إفتراضية في الأخرى. إن لم ترد ذلك ستكون مجبر على وضع قيم إفتراضية لجميع الباراميتيرات الموجودة بعد أول باراميتير وضعت له قيمة إفتراضية.

أين يجب تعريف الدوال في C++

مترجم لغة C++ يقرأ الكود سطرًا سطرًا مع تنفيذ الأوامر الموضوعه في كل سطر بشكل مباشر عندما يتم تشغيل البرنامج. لهذا السبب يجب دائماً أن تكون الدالة التي تريد استدعاءها معرفّة سابقاً حتى لا يظهر لك مشكلة عند تشغيل البرنامج.

في المثال التالي، قمنا بوضع الدالة myFunction() بعد الدالة التي قمنا باستدعائها منها. المشكلة التي ستحدث عند التشغيل هنا سببها أن المترجم سيكون لا يعرف ما هي myFunction() حيث أنه تم استدعاءها قبل أن يقوم المترجم قد سبق وقرأها.

المثال الأول

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// هنا قمنا باستدعاء الدالة myFunction()
```

```
myFunction();
```

```
return 0;
```

```
}
```

```
// هنا قمنا بتعريف الدالة myFunction التي تحتوي على أمر طباعة فقط
```

```
void myFunction()
```

```
{
```

```
cout << "My first function is called";
```

```
}
```

سيظهر الخطأ التالي عند التشغيل و الذي يعني أن المترجم لم يعرف ما هي myFunction التي تحاول استدعاءها في السطر الثامن.

main.cpp|8|error: 'myFunction' was not declared in this scope|

حل مشكلة عدم التعرف على الدالة

لحل مشكلة عدم التعرف على الدالة التي حدثت في المثال السابق عندنا خيارين:

- إبقاء الدالة `myFunction()` مكانها و ذكر تعريفها (`Function Declartion`) في أول الملف فقط، و هذه الطريقة تعتبر الأكثر تفضيلاً.
- وضع الدالة `myFunction()` فوق الدالة `main()` حتى يقوم المترجم بقرائها و التعرف عليها و تصبح قادر على استدعاءها في الدالة `main()` الموجودة بعدها.

في المثال التالي، قمنا بإبقاء الدالة `myFunction()` مكانها و ذكر تعريفها (`Function Declartion`) قبل أن يتم استدعاءها في الدالة `main()`.
إذاً لن يحدث أي مشكلة عند استدعاء الدالة `myFunction()` من الدالة `main()` لأن المترجم سيكون لديه علم بأن الدالة `myFunction()` موجودة فعلاً.

المثال الثاني

```
#include <iostream>
```

```
using namespace std;
```

حتى يقوم المترجم بالتعرف عليها و نصبح قادرين على استخدامها
هنا قمنا بوضع تعريف (`Declartion`) الدالة `myFunction`

```
void myFunction();
```

```
int main()
```

```
{
```

```
// هنا قمنا باستدعاء الدالة myFunction()
```

```
myFunction();
```

```
return 0;
```

```
}
```

`myFunction`, أو بمعنى آخر تعريف ما سيحدث عندما يتم استدعاءها
هنا قمنا بتعريف جسم (`body`) الدالة

```
void myFunction()
```



```
{  
  
cout << "My first function is called";  
  
}
```

في الدالة `main()` سيتم استدعاء و تنفيذ الدالة `myFunction()` بدون أي مشاكل و سنحصل على النتيجة التالية عند التشغيل.

My first function is called

نصيحة

الأفضل دائماً وضع تعريفات الدوال (Functions Declartions) قبل الدالة `main()` و تعريف محتوى هذه الدوال بعدها كالتالي لأن قراءة الكود ستصبح أسهل بالنسبة لك.
المثال التالي يريك فقط كيف تقوم بترتيب الكود إذا كنت تنوي تعريف العديد من الدوال في الملف.

المثال الثالث

```
#include <iostream>
```

```
using namespace std;
```

```
// قبل main() نقوم بذكر تعريف جميع الدوال التي سنقوم بإنشائها لاحقاً  
الدالة
```

```
void printMessage();
```

```
void greeting(string name);
```

```
// وفيها يمكننا استدعاء أي دالة قمنا بذكر تعريفها سابقاً بدون أي  
هنا نقوم بتعريف الدالة main() مشاكل
```

```
int main()
```

```
{
```

```
// هنا يمكننا استدعاء printMessage() و greeting()
```

```
return 0;
```

```
}
```

هنا نقوم بتعريف `printMessage()` لأنه قمنا بذكر أنها موجودة من قبل
الدالة

```
void printMessage()
```

```
{
```

```
// هنا نكتب ما سيحدث عند استدعاءها
```

```
}
```

هنا نقوم بتعريف `greeting()` لأنه قمنا بذكر أنها موجودة من قبل
الدالة

```
void greeting(string name)
```

```
{
```

```
// هنا نكتب ما سيحدث عند استدعاءها
```

```
}
```

الدوال المكتوبة في C++

الدوال المكتوبة في C++ هي مجموعة من الدوال التي تأتي مع المكتبة القياسية للغة C++. توفر هذه الدوال وظائف جاهزة ومفيدة تساعد في إنشاء وتشغيل البرامج بشكل أسهل وأكثر فاعلية. من أمثلة الدوال المكتوبة الشائعة في C++:

1. دوال الإدخال والإخراج (Input/Output Functions): مثل `cout` و `cin` للإخراج والإدخال على التوالي.

2. دوال العمليات الحسابية: مثل `sqrt()` لحساب جذر تربيعي و `pow()` لحساب الأس العشري.

3. دوال العمليات على السلاسل (Strings Functions): مثل `strlen()` لحساب طول السلسلة و `strcmp()` لمقارنة سلاسل النصوص.

4. دوال العمليات على المصفوفات (Arrays Functions): مثل `sizeof()` لحساب حجم المصفوفة و `sort()` لفرز عناصر المصفوفة.

5. دوال الوقت والتاريخ: (Date and Time Functions) مثل `time()` للحصول على الوقت الحالي و `ctime()` لتنسيق الوقت كنص.

6. دوال الرياضيات الشائعة: مثل `abs()` للقيم المطلقة و `ceil()` لتقريب الأعداد لأعلى قيمة صحيحة.

هذه مجرد أمثلة قليلة، فهناك العديد من الدوال المكتبية المفيدة الأخرى في C++ التي توفر وظائف مختلفة وتسهل عملية البرمجة.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة أمثلة على الدوال الجاهزة في C++ والغرض من استخدام كل منها
- 2- اذكر أحد الطرق المستخدمة في حل مشكلة عدم التعرف على الدالة
- 3- اذكر ثلاثة من الدوال المكتبية مع ذكر مثال لكل منها
- 4- اكتب دالة حساب حجم المصفوفة
- 5- اكتب دالة حساب طول السلسلة
- 6- اكتب دالة حساب جذر تربيعي
- 7- اكتب دالة الإدخال
- 8- اكتب دالة الإخراج

المحاضرة العاشرة

دوال المستخدم في C++

دوال المستخدم في C++ هي الدوال التي ينشئها المبرمج بنفسه لتنفيذ وظائف محددة داخل البرنامج. يتم استخدام الدوال المستخدمة لتقسيم البرنامج إلى أجزاء صغيرة ومستقلة تقوم بمهام محددة، مما يجعل البرنامج أكثر تنظيماً وسهولة في الصيانة والتطوير.

يتم تعريف الدوال المستخدمة بواسطة المبرمج، وتتكون من اسم الدالة وقائمة بالمعلمات التي تقبلها الدالة (إذا كانت هناك) ونوع القيمة التي تُعيدها الدالة (إذا كانت تعيد قيمة)، بالإضافة إلى الكود الذي يتم تنفيذه عند استدعاء الدالة.

```
#include <iostream>
```

```
// تعريف دالة المستخدم
```

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

```
int main() {
```

```
    int x = 5, y = 3;
```

```
    // استدعاء دالة المستخدم وطباعة النتيجة
```

```
    std::cout << "The sum of " << x << " and " << y << " is: " << add(x, y) << std::endl;
```

```
    return 0;
```

```
}
```

في هذا المثال، تم تعريف دالة المستخدم `add` التي تقوم بجمع اثنين من الأعداد الصحيحة، وفي دالة `main` تم استدعاء هذه الدالة واستخدامها لجمع قيمتين وطباعة النتيجة. تُعد الدوال المستخدمة أحد الأساسيات في البرمجة في C++ لأنها تساعد في تنظيم الكود وإعادة استخدامه بشكل فعال.

أنواع دوال المستخدم

في C++، يمكن تقسيم دوال المستخدم إلى عدة أنواع حسب تأثيرها على المتغيرات والقيم التي تعيدها. هنا هي الأنواع الرئيسية لدوال المستخدم في C++:

1. دوال بدون قيمة عائدة (void functions):

- تُستخدم لتنفيذ سلسلة من العمليات دون إرجاع قيمة.
- يتم تعريفها باستخدام الكلمة الرئيسية `void` بدلاً من نوع القيمة المراد إرجاعها.
- مثال: دالة لطباعة رسالة على الشاشة دون إرجاع قيمة.

مثال

هذا المثال يوضح دالة بسيطة لطباعة رسالة على الشاشة دون إرجاع قيمة

```
#include <iostream>
// تعريف دالة بدون قيمة عائدة
void printMessage() {
    std::cout << "Hello, World!" << std::endl;
}
int main() {
    // استدعاء الدالة
    printMessage();
    return 0;
}
```

2. دوال بقيمة عائدة (value-returning functions):

- تُستخدم لتنفيذ سلسلة من العمليات وإرجاع قيمة معينة.
- يتم تعريف نوع القيمة المُراد إرجاعها قبل اسم الدالة.
- مثال: دالة لجمع أرقام وإرجاع الناتج.

مثال

هذا المثال يوضح دالة لجمع أرقام وإرجاع الناتج.

```
#include <iostream>
// تعريف دالة بقيمة عائدة
```

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int x = 5, y = 3;  
    // استدعاء الدالة واستخدام قيمة العائد  
    int sum = add(x, y);  
    std::cout << "The sum of " << x << " and " << y << " is: " << sum << std::endl;  
    return 0;  
}
```

3. دوال مع معاملات (functions with parameters):

- تأخذ معلومات كمدخلات لتنفيذ العمليات داخل الدالة.
- يتم تعريف المعلومات في قائمة البارامترات بين قوسين بعد اسم الدالة.
- مثال: دالة لحساب المعدل الحسابي تأخذ قائمة من الأعداد كمدخلات.

مثال

هذا المثال يوضح دالة لحساب المعدل الحسابي تأخذ قائمة من الأعداد كمدخلات.

```
#include <iostream>  
  
// تعريف دالة مع معاملات  
  
double calculateAverage(int array[], int size) {  
    double sum = 0.0;  
    for(int i = 0; i < size; ++i) {  
        sum += array[i];  
    }  
    return sum / size;  
}  
  
int main() {  
    int numbers[] = {10, 20, 30, 40, 50};
```

```
int size = sizeof(numbers) / sizeof(numbers[0]);  
double average = calculateAverage(numbers, size);  
std::cout << "The average is: " << average << std::endl;  
return 0;  
}
```

4. دوال متعددة الأشكال (function overloading):

- تُستخدم لإنشاء دوال بنفس الاسم ولكن مع توقعات (تركيبيات المعلمات) مختلفة.
- يتم التفريق بين دوال التحميل بناءً على توقعات المعلمات.
- مثال: دالة 'print' تأخذ معلمة نص وأخرى رقمية، ويتم تنفيذ دالة مختلفة بناءً على نوع المعلمة.

مثال

هذا المثال يوضح دالة print تأخذ معلمة نص وأخرى رقمية، ويتم تنفيذ دالة مختلفة بناءً على نوع المعلمة.

```
#include <iostream>  
// متعددة الأشكال print تعريف دالة  
void print(const char* message) {  
    std::cout << "Message: " << message << std::endl;  
}  
void print(int number) {  
    std::cout << "Number: " << number << std::endl;  
}  
int main() {  
    print("Hello, World!"); // استدعاء الدالة مع معلمة نص  
    print(42);              // استدعاء الدالة مع معلمة رقمية  
    return 0;  
}
```

5. دوال متعددة العمليات (function templates):

- تُستخدم لإنشاء دوال قابلة للتكيف مع أنواع البيانات المختلفة.
- يمكن استخدامها مع أي نوع من البيانات بناءً على قالب محدد.
- مثال: دالة للمقارنة بين عناصر مصفوفة يمكن تطبيقها على مصفوفات من أنواع مختلفة.

مثال

هذا المثال يوضح دالة template للمقارنة بين عناصر مصفوفة يمكن تطبيقها على مصفوفات من أنواع مختلفة

```
#include <iostream>

// للمقارنة بين العناصر template تعريف دالة //

template <typename T>
bool compare(T a[], T b[], int size) {
    for(int i = 0; i < size; ++i) {
        if(a[i] != b[i]) {
            return false;
        }
    }
    return true;
}

int main() {
    int array1[] = {1, 2, 3};
    int array2[] = {1, 2, 3};
    int size = sizeof(array1) / sizeof(array1[0]);
    bool result = compare(array1, array2, size);
    std::cout << "Arrays are equal: " << std::boolalpha << result << std::endl;
    return 0;
}
```


هذه الأنواع الرئيسية توفر مجموعة متنوعة من الخيارات للمبرمجين لإنشاء دوال تتناسب مع احتياجات البرنامج والبيانات المُعالجة.

شروط انشاء دوال المستخدم

هناك عدة شروط يجب أن تتوفر لإنشاء دوال المستخدم في C++:

1. تعريف الدالة:

يجب أن يكون لدى الدالة اسم مميز يمثل وظيفتها.

شروط اسم الدالة في C++ هي كما يلي:

- يجب أن يكون الاسم صالحًا:
 - يجب أن يتكون الاسم من مجموعة من الأحرف (أبجدية لاتينية) والأرقام والرموز التالية: ` \_ `.
 - يجب أن يبدأ الاسم بحرف أو ` \_ `، ولا يمكن أن يبدأ برقم.
- الحالة (Case Sensitivity):
 - يجب مراعاة الحالة في الاسم، حيث أن C++ حساسة للحالة، مما يعني أن `myFunction` و `MyFunction` و `MYFUNCTION` تُعتبر أسماء دوال مختلفة.
- يجب أن لا يكون الاسم محجورًا:
 - يجب تجنب استخدام الأسماء التي تمتلكها كلمات محجوزة في لغة C++، مثل `if`، `for`، `int`، وغيرها.
- يجب أن يكون الاسم واضحًا ومعبرًا:
 - يفضل استخدام أسماء واضحة ومعبرة توضح وظيفة الدالة، مما يسهل فهمها لاحقًا وزيادة قابلية إعادة استخدام الكود.
- يجب أن يتبع مبادئ التسمية (Naming Conventions):
 - قد تكون هناك مبادئ تسمية معينة تُتبع في المشاريع أو الشركات، مثل استخدام حروف كبيرة لبداية كلمة جديدة في الاسم (CamelCase) أو استخدام الشرطات السفلية (snake_case)، ومن الأفضل اتباع هذه المبادئ لتوحيد الأسماء وجعل الكود أكثر تنظيمًا.
- يجب تجنب استخدام الأسماء الطويلة جدًا:
 - يفضل استخدام أسماء مختصرة ومعبرة في نفس الوقت، لتسهيل قراءة الكود وفهمه.
- تلتزم أسماء الدوال في C++ بالقواعد الشائعة لتسمية المتغيرات والكائنات في البرمجة، مع التأكيد على الوضوح والمعنى الجيد لتسهيل فهم وصيانة الكود.

2. نوع القيمة المرجعة (إن وُجد):

إذا كانت الدالة تقوم بإرجاع قيمة، فيجب تحديد نوع القيمة المرجعة في تعريف الدالة.

القيمة المرجعية في C++ هي القيمة التي تُعيد إلى البرنامج الرئيسي من قبل دالة ما بعد تنفيذها. تُستخدم القيمة المرجعية لإرجاع نتيجة عملية معينة من داخل الدالة، مما يسهل استخدام هذه النتيجة في البرنامج الرئيسي أو في دوال أخرى.

القيمة المرجعية تعرف في تعريف الدالة باستخدام نوع البيانات الذي ترجعه الدالة، ويمكن استخدامها في تعيين قيمة لمتغير في البرنامج الرئيسي بعد استدعاء الدالة.

مثال

```
#include <iostream>

// تعريف دالة ترجع قيمة مرجعية
int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 5, y = 3;
    int sum = add(x, y); // استدعاء الدالة وتخزين القيمة المُرجعة في sum
    std::cout << "The sum of " << x << " and " << y << " is: " << sum << std::endl;
    return 0;
}
```

في هذا المثال، تم تعريف دالة `add` التي تقوم بجمع اثنين من الأعداد وإرجاع النتيجة، وفي البرنامج الرئيسي تم استدعاء الدالة واستخدام قيمة القيمة المرجعية في تخزين النتيجة في متغير `sum`.

3. المعلومات (إن وُجدت):

إذا كانت الدالة تحتاج إلى معلومات كمدخلات، فيجب تحديد نوع كل معلمة واسمها في تعريف الدالة.

في `C++`، تعتبر المعلومات (`parameters`) جزءاً مهماً من تعريف الدوال. تُستخدم المعلومات لتمرير البيانات إلى الدوال، مما يسمح للدوال بتنفيذ العمليات على البيانات الممررة إليها. يمكن تحديد عدد معين من المعلومات لكل دالة، ويمكن تعريف نوع واسم لكل معلمة.

هناك نوعان رئيسيان من المعلومات في `C++`:

1. المعلومات المُرجعة (`pass by reference`):

- تمرير البيانات إلى الدالة باستخدام عنوان الذاكرة للمتغير.

- يتم تعريف المعلمة بواسطة استخدام الرمز `&` بعد نوع المعلمة.

- يُسمح للدالة بتعديل قيم المتغيرات التي تُمرر عن طريق المَعْلَمَات المُرْجَعَة.

- مثال: `void changeValue(int &x)`.

2. المَعْلَمَات المتغيرة (pass by value):

- تمرير البيانات إلى الدالة بنسخ قيمة المتغير.

- يتم تعريف المعلمة بواسطة استخدام نوع المعلمة فقط دون استخدام `&`.

- لا يُسمح للدالة بتعديل قيم المتغيرات التي تُمرر عن طريق المَعْلَمَات المتغيرة.

- مثال: `void printValue(int x)`.

تستخدم المَعْلَمَات لتمرير البيانات إلى الدوال بطريقة تجعل الدوال قادرة على تنفيذ العمليات المطلوبة على هذه البيانات، سواء كانت للقراءة فقط أو للتعديل.

4. جسم الدالة:

- يجب تحديد ما يحدث داخل الدالة، ويكون ذلك في جسم الدالة.

جسم الدالة في ++C هو المكان الذي يتم فيه تعريف العمليات التي تنفذها الدالة. يُمثل جسم الدالة مجموعة من البيانات والتعليمات التي تقوم بتنفيذها عند استدعاء الدالة. تتكون جسم الدالة من مجموعة من البيانات والتعليمات التي تأخذ شكل محدد ويجب أن تتبع القواعد والصيغ المحددة في لغة ++C.

مكونات جسم الدالة تشمل:

• المتغيرات المحلية (Local Variables):

- هي المتغيرات التي تتم تعريفها داخل جسم الدالة ولا تكون مرئية خارج نطاق الدالة.
- تستخدم لتخزين البيانات المؤقتة والمحلية التي تحتاجها الدالة أثناء تنفيذها.

• التعليمات (Statements):

- تشمل التعليمات البرمجية التي تُعبر عن العمليات التي يجب أن تُنفذها الدالة.
- تشمل التعليمات عمليات الرياضيات والمقارنات والتحكم في التدفق وغيرها من العمليات البرمجية.

• التحكم في التدفق (Flow Control):

- تشمل تعليمات التحكم مثل الشروط الشرطية (if-else) والحلقات (loops) والتعليمات الشرطية الأخرى التي تؤثر على تنفيذ البرنامج.

• التعليمات الخاصة بالإرجاع (Return Statements):

- تُستخدم لإرجاع قيمة من الدالة إلى البرنامج الرئيسي.
- تشمل تعليمات الإرجاع القيم المرجعية التي يجب أن تُرجعها الدالة.

- **التعليمات الخاصة بالإدخال والإخراج (Input/Output Statements):**
تُستخدم للتفاعل مع المستخدم أو مع البيانات المُدخلة من خلال استخدام دوال الإدخال والإخراج مثل `cin` و `cout`.

عند استدعاء الدالة، يتم تنفيذ جسم الدالة بترتيبه المحدد داخل تعريف الدالة، ويتم استخدام المتغيرات المحلية والتعليمات البرمجية لتنفيذ العمليات المطلوبة وإعادة القيمة المرجعية إذا كانت الدالة تُرجع قيمة.

5. عبارة العودة (إن وُجدت):

إذا كانت الدالة تقوم بإرجاع قيمة، يجب استخدام عبارة `return` في جسم الدالة لإعادة القيمة المطلوبة.

6. التوثيق (اختياري):

يمكن إضافة تعليقات توثيقية (documentation comments) لوصف وظيفة الدالة وتوضيح كيفية استخدامها.

7. التوقيع (اختياري في الحالات الخاصة):

يمكن استخدام توقيع الدالة (function prototype) قبل تعريف الدالة الفعلي في البرنامج الرئيسي، وذلك لإعلام المترجم بوجود الدالة قبل استخدامها في البرنامج.

8. عامل الإرجاع (اختياري):

يمكن استخدام `void` كنوع القيمة المرجعة إذا لم تقم الدالة بإرجاع قيمة.

9. نطاق الدالة (اختياري):

يمكن تحديد نطاق الدالة (scope) إما لتكون متاحة على مستوى الملف الكامل (global scope) أو محدودة داخل نطاق معين (local scope).

نطاق الدالة في ++C يحدد مدى وقت ومكان وجود الدالة والمتغيرات التي تم إنشاؤها داخل الدالة. هناك نوعان رئيسيان من نطاق الدالة:

- **نطاق الدالة المحلي (Local Scope):**
 - عندما يكون النطاق المحلي، يعني ذلك أن الدالة والمتغيرات التي تم إنشاؤها داخل الدالة ليست مرئية خارج نطاق الدالة نفسها.
 - يعني ذلك أن المتغيرات المحلية يمكن الوصول إليها واستخدامها فقط داخل الدالة التي تم إنشاؤها بها.
 - هذا يُسمى أيضًا بنطاق الدالة الخاص (Function Scope).

• نطاق الدالة العالمي (Global Scope):

- عندما يكون النطاق العالمي، يعني ذلك أن الدالة والمتغيرات التي تم إنشاؤها داخل الدالة يمكن الوصول إليها واستخدامها في أي مكان داخل البرنامج.

- يمكن تعريف الدوال والمتغيرات على مستوى البرنامج الرئيسي خارج أي دالة، مما يجعلها متاحة لجميع الدوال والكائنات داخل البرنامج.
- إذا تم تعريف متغير في النطاق العالمي، يُمكن الوصول إليه واستخدامه في أي دالة في البرنامج.

من المهم فهم النطاق الصحيح للدوال والمتغيرات في C++ لتجنب الأخطاء والتعارضات في البرنامج. استخدام النطاق العالمي بحذر يساعد في تنظيم البرنامج وجعل الكود أكثر فهمًا وصيانة.

هذه الشروط الأساسية تساعد في إنشاء دوال مستخدمة صالحة وقابلة للتفاعل داخل البرنامج.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة امثلة على دوال المستخدم في C++ والغرض من استخدام كل منها
- 2- اذكر ثلاثة من شروط اختيار اسم الدالة
- 3- اذكر نوعي المعلومات المستخدمة في C++ والغرض من استخدام كل منهما
- 4- اذكر ثلاثة من مكونات جسم الدالة
- 5- اكتب ثلاثة من المكونات الاختيارية في جسم الدالة

المحاضرة الحادية عشر

دوال التحكم

جملة `return`

جملة `return` في ++C هي جملة تستخدم لإنهاء تنفيذ دالة وإرجاع قيمة من الدالة إلى الجزء الذي استدعى الدالة. يتم استخدام `return` داخل الدالة للإشارة إلى أن الدالة قد انتهت وتحديد القيمة التي يجب إعادتها. جملة `return` هي جزء أساسي من الدوال التي لها نوع إرجاع غير `void`.

هناك نوعان رئيسيان من استخدامات جملة `return`:

1. إرجاع قيمة من دالة لها نوع إرجاع محدد:

عندما تكون الدالة مخصصة لإرجاع قيمة معينة (مثل `char`, `double`, `float`, `int`, أو نوع مخصص)، تستخدم جملة `return` لإرجاع هذه القيمة.

```
if (condition) {
```

```
    // Statements to execute if the condition is true
```

```
} else {
```

```
    // Statements to execute if the condition is false
```

```
}
```

2. إنهاء دالة لا تُرجع قيمة (نوع الإرجاع `void`):

عندما تكون الدالة من النوع `void`، لا يتم إرجاع قيمة، ولكن يمكن استخدام `return` لإنهاء تنفيذ الدالة مبكرًا.

```
if (condition1) {
```

```
    // Statements if condition1 is true
```

```
} else if (condition2) {
```

```
    // Statements if condition2 is true
```

```
} else {
```

```
    // Statements if neither condition1 nor condition2 is true
```

```
}
```

أمثلة على استخدام جملة `return`:

1. إرجاع قيمة من دالة تقوم بإجراء حساب:

```
double multiply(double x, double y) {  
    return x * y; // إرجاع نتيجة الضرب  
}
```

2. إنهاء دالة void مبكرًا بناءً على شرط:

```
void checkNumber(int num) {  
    if (num < 0) {  
        std::cout << "Number is negative." << std::endl;  
        return; // إنهاء الدالة إذا كان الرقم سالبًا  
    }  
    std::cout << "Number is non-negative." << std::endl;  
}
```

3. استخدام `return` في دالة معقدة:

```
int findMax(int a, int b, int c) {  
    if (a >= b && a >= c) {  
        return a;  
    } else if (b >= a && b >= c) {  
        return b;  
    } else {  
        return c;  
    }  
}
```

جملة `return` تُستخدم لتحديد النتيجة التي تريد الدالة إرجاعها، وهي وسيلة لربط الدالة بالجزء الذي قام باستدعائها من البرنامج.

الخطأ الذي يمكن أن تحدث عند استخدام جملة `return`

استخدام `return` في ++C قد يؤدي إلى أخطاء متنوعة إذا لم يتم استخدامه بشكل صحيح. هنا بعض الأخطاء الشائعة التي قد تحدث عند استخدام جملة `return`:

1. عدم إرجاع قيمة في دالة غير `void`:

إذا كانت الدالة محددة بنوع إرجاع معين غير `void`، فيجب عليك التأكد من أن جميع المسارات داخل الدالة تُرجع قيمة مناسبة. عدم القيام بذلك يؤدي إلى خطأ عند الترجمة.

```
int add(int a, int b) {  
    if (a > 0 && b > 0) {  
        return a + b;  
    }  
    // لا تُرجع قيمة إذا كان الشرط غير محقق  
}
```

تصحيح:

```
int add(int a, int b) {  
    if (a > 0 && b > 0) {  
        return a + b;  
    }  
    return 0; // إرجاع قيمة افتراضية في حال عدم تحقق الشرط  
}
```

2. إرجاع مراجع أو مؤشرات إلى كائنات محلية:

إذا أرجعت مرجعًا أو مؤشرًا إلى كائن محلي، فسيكون المؤشر غير صالح بمجرد خروج الدالة عن نطاق التنفيذ، مما يؤدي إلى سلوك غير معروف.

```
int& getReference() {  
    int x = 10;  
    return x; // إرجاع مرجع إلى متغير محلي  
}
```

تصحيح:

إما استخدام متغيرات ثابتة (static) أو تخصيص ديناميكي للذاكرة.


```
int* getPointer() {  
    int* x = new int(10);  
    return x; // تخصيص ديناميكي للذاكرة  
}
```

3. إرجاع قيمة مؤقتة (Dangling references):

إرجاع مرجع إلى كائن تم إنشاؤه داخل الدالة والذي سيتم تدميره عند خروج الدالة.

```
const std::string& getString() {  
    return std::string("Hello"); // إرجاع مرجع إلى كائن مؤقت  
}
```

تصحيح:

إرجاع كائن بالقيمة بدلاً من المرجع.

```
std::string getString() {  
    return std::string("Hello"); // إرجاع كائن بالقيمة  
}
```

4. عدم استخدام القيمة المرجعة:

إهمال استخدام القيمة التي تُرجعها الدالة قد يؤدي إلى فقدان معلومات مهمة، أو قد يشير إلى خطأ في التصميم.

```
int add(int a, int b) {  
    return a + b;  
}  
  
void someFunction() {  
    add(3, 4); // لم يتم استخدام القيمة المرجعة  
}
```

تصحيح:

استخدام القيمة المرجعة أو إعادة تصميم الدالة إذا لم تكن هناك حاجة للقيمة.

```
void someFunction() {
```

```
int sum = add(3, 4); // استخدام القيمة المرجعة  
std::cout << "Sum: " << sum << std::endl;  
}
```

5. إرجاع كائنات كبيرة بشكل غير ضروري:

إرجاع كائنات كبيرة بالقيمة يمكن أن يكون غير فعال من حيث الأداء بسبب عمليات النسخ.

```
std::vector<int> createVector() {  
    std::vector<int> v(1000);  
    return v; // إرجاع كائن كبير بالقيمة  
}
```

تصحيح:

استخدام المراجع أو المؤشرات لتحسين الأداء، أو استخدام التحسينات المتاحة مثل (move semantics).

```
std::vector<int> createVector() {  
    std::vector<int> v(1000);  
    return std::move(v); // استخدام نقل الكائن لتحسين الأداء  
}
```

تجنب هذه الأخطاء الشائعة يضمن أن تكون الدوال التي تكتبها موثوقة وتعمل كما هو متوقع، مما يسهل صيانة البرنامج وتحسين أدائه.

جملة `void`

جملة `void` في ++C تشير إلى نوع يُستخدم في سياقات معينة للدلالة على عدم وجود قيمة إرجاع. يتم استخدام `void` في عدة سيناريوهات مختلفة، منها:

1. تعريف دوال لا تُرجع قيمة:

عندما يتم تعريف دالة باستخدام `void`، فهذا يعني أن الدالة لا تُرجع أي قيمة عند اكتمال تنفيذها. يتم استخدام هذا النوع من الدوال لتنفيذ بعض المهام دون الحاجة إلى إرجاع نتيجة.

```
void printHello() {  
    std::cout << "Hello, World!" << std::endl;  
}
```

2. المؤشرات العامة (void pointers):

يمكن استخدام `void` لتعريف مؤشرات عامة يمكن أن تشير إلى أي نوع من البيانات. يمكن تحويل المؤشرات العامة إلى نوع معين من المؤشرات قبل استخدامها.

```
void* ptr;  
int x = 10;  
ptr = &x; // الإشارة إلى عنوان x  
int* intPtr = static_cast<int*>(ptr); // تحويل المؤشر إلى نوع int  
std::cout << *intPtr << std::endl; // طباعة القيمة المخزنة في x
```

3. تحديد أن الدالة لا تأخذ أي معاملات:

يمكن استخدام `void` في قائمة المعاملات للدالة للإشارة إلى أن الدالة لا تأخذ أي معاملات. في C++، يمكن ترك قائمة المعاملات فارغة بدون استخدام `void`.

```
void doNothing(void) {  
    // هذه الدالة لا تأخذ أي معاملات  
}
```

أمثلة على استخدام `void`:

1. دالة لا تُرجع قيمة:

```
void sayGoodbye() {  
    std::cout << "Goodbye!" << std::endl;  
}
```

2. مؤشر عام:

```
void* generalPointer;  
  
double y = 5.5;  
  
generalPointer = &y; // الإشارة إلى عنوان y  
  
double* doublePtr = static_cast<double*>(generalPointer); // double تحويل المؤشر إلى نوع  
  
std::cout << *doublePtr << std::endl; // طباعة القيمة المخزنة في y
```

3. دالة بدون معاملات:

```
void displayMessage() {  
  
    std::cout << "This is a message." << std::endl;  
  
}
```

استخدام `void` في ++C يوفر وسيلة لتحديد الدوال التي لا تُرجع قيم، بالإضافة إلى توفير مرونة أكبر في التعامل مع المؤشرات.

الايخطاء التي يمكن ن تحدث عند استخدام void

عند استخدام جملة `void` في ++C، يمكن أن تحدث بعض الأخطاء الشائعة، ومنها:

1. إرجاع قيمة من دالة `void`: محاولة استخدام جملة `return` لإرجاع قيمة من دالة تم تعريفها باستخدام `void`.
2. استدعاء دالة `void` في تعبير يحتاج إلى قيمة: استخدام دالة `void` كجزء من تعبير أو عملية حسابية تتطلب إرجاع قيمة.
3. سوء استخدام المؤشرات العامة (void pointers): إجراء عمليات على مؤشرات `void` دون تحويلها إلى نوع محدد، مما يؤدي إلى أخطاء في الترجمة أو في وقت التشغيل.
4. استخدام `void` كمحدد نوع لمتغير: محاولة تعريف متغير باستخدام `void` كمحدد نوع، وهو غير مسموح به في ++C.
5. عدم تطابق أنواع الإرجاع عند التعريف والتصريح: تعريف دالة بنوع إرجاع مختلف عن التصريح المعلن في ملف الرأس أو مكان آخر في البرنامج.
6. نسيان تعريف دالة `void` كـ `static` في سياق استخدام المؤشرات إلى الدوال: إذا كانت الدالة تستخدم كمؤشر إلى دالة، يجب التأكد من تطابق التعريف مع السياق.

هذه الأخطاء يمكن أن تؤدي إلى مشاكل في الترجمة أو إلى سلوك غير متوقع عند تشغيل البرنامج.

جملة `exit`

جملة `exit` في ++C تُستخدم لإنهاء البرنامج بشكل فوري، حيث تقوم بإنهاء العملية التي تنفذ البرنامج وإعادة التحكم إلى نظام التشغيل. تُعتبر `exit` جزءاً من مكتبة C القياسية (`stdlib` في ++C). عند استدعاء هذه الدالة، يتم إنهاء جميع الدوال النشطة وتحرير الموارد، ثم يتم إعادة قيمة محددة إلى نظام التشغيل.

كيفية استخدام `exit`

يمكن استخدام دالة `exit` عن طريق تضمين مكتبة `stdlib` (أو `stdlib.h` في C):

```
#include <stdlib>
```

```
int main() {
```

```
    // بعض التعليمات
```

```
    if (someCondition) {
```

```
        exit(1); // إنهاء البرنامج وإرجاع القيمة 1 إلى نظام التشغيل
```

```
    }
```

```
    // exit تعليمات أخرى قد لا يتم تنفيذها بسبب استدعاء //
```

```
    return 0;
```

```
}
```

المعاملات التي تأخذها دالة `exit`

تأخذ دالة `exit` معاملاً واحداً وهو قيمة الإرجاع (exit status) التي يتم إعادتها إلى نظام التشغيل. هذه القيمة يمكن استخدامها للإشارة إلى حالة إنهاء البرنامج:

- القيمة `0`: عادةً ما تعني أن البرنامج انتهى بنجاح.

- أي قيمة غير `0`: تشير إلى حدوث خطأ أو حالة غير طبيعية أدت إلى إنهاء البرنامج.

تأثير `exit`

عند استدعاء `exit`، يتم تنفيذ الخطوات التالية:

1. تنفيذ الدوال المسجلة باستخدام `atexit` : إذا كانت هناك دوال مسجلة لتنفيذها عند انتهاء البرنامج باستخدام `atexit`، فسيتم استدعاؤها.
2. تحرير الموارد: يتم تنظيف الموارد وتحرير الذاكرة التي قد تكون استخدمت خلال تنفيذ البرنامج.
3. إغلاق الملفات: يتم إغلاق جميع الملفات المفتوحة تلقائيًا.
4. إرجاع قيمة الإنهاء: يتم إعادة قيمة الإنهاء المحددة إلى نظام التشغيل.

مثال على استخدام `exit`

في المثال التالي، يتم استخدام `exit` لإنهاء البرنامج إذا تحقق شرط معين:

```
#include <iostream>

#include <cstdlib>

void cleanup() {
    std::cout << "Cleanup function called." << std::endl;
}

int main() {
    std::atexit(cleanup); // تسجيل دالة التنظيف لتنفيذها عند انتهاء البرنامج
    std::cout << "Starting program..." << std::endl;

    if (true) { // شرط للتحقق
        std::cout << "Condition met, exiting program." << std::endl;
        exit(1); // إنهاء البرنامج وإرجاع القيمة 1
    }

    std::cout << "This line will not be executed." << std::endl;
    return 0;
}
```

في هذا المثال، عند تحقق الشرط، يتم استدعاء `exit(1)` لإنهاء البرنامج، وستتم طباعة رسالة "Cleanup function called." قبل الإنهاء الفعلي للبرنامج بفضل دالة `cleanup` المسجلة باستخدام `atexit`.

الايخطاء التي يمكن ان تحدث عند استخدام exit

عند استخدام دالة 'exit' في ++C، يمكن أن تحدث الأخطاء التالية:

1. إنهاء البرنامج بشكل غير متوقع: استدعاء 'exit' قد يؤدي إلى إنهاء البرنامج فجأة دون إتاحة الفرصة لتحرير الموارد أو إنهاء العمليات المفتوحة بشكل صحيح.
 2. تخطي عمليات التنظيف: قد يتم تجاوز دوال التنظيف أو الإجراءات التي يجب تنفيذها في نهاية البرنامج، مما يؤدي إلى تسرب الذاكرة أو موارد أخرى غير محررة.
 3. تجاهل دوال الإنهاء المسجلة: قد لا يتم تنفيذ دوال الإنهاء المسجلة باستخدام 'atexit' إذا تم استدعاء 'exit' في سياق غير مناسب.
 4. عدم إعادة حالة إنهاء مناسبة: استخدام قيمة غير صحيحة كعامل لدالة 'exit' قد يؤدي إلى سوء تفسير حالة إنهاء البرنامج بواسطة النظام أو البرامج الأخرى.
 5. التسبب في حالات غير متوقعة: إنهاء البرنامج بواسطة 'exit' قد يؤدي إلى نتائج غير متوقعة إذا كان البرنامج يعتمد على استكمال تسلسل معين من العمليات قبل الإنهاء.
- هذه الأخطاء تؤكد على أهمية استخدام 'exit' بحذر والتأكد من أن البرنامج يستطيع التعامل مع إنهاء مفاجئ بطريقة سليمة وأمنة.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة امثلة على دوال التحكم في ++C والغرض من استخدام كل منها
- 2- اذكر ثلاثة الأخطاء التي يمكن ان تحدث عند استخدام جملة return
- 3- اذكر ثلاثة من حالات استخدام الجملة void
- 4- اذكر ثلاثة من الأخطاء التي يمكن ان تحدث عند استخدام الجملة void
- 5- اكتب الغرض من استخدام الجملة exit
- 6- اذكر ثلاثة من الأخطاء التي يمكن ان تحدث عند استخدام الجملة exit

المحاضرة الثانية عشر

المؤثرات

في ++C، المؤثرات (Operators) هي رموز خاصة تُستخدم لإجراء عمليات على المتغيرات والقيم. يمكن تصنيف المؤثرات في ++C إلى عدة فئات بناءً على نوع العمليات التي تنفذها:

1. المؤثرات الحسابية (Arithmetic Operators):

تُستخدم لإجراء العمليات الحسابية الأساسية.

- '+' (الجمع)

- '-' (الطرح)

- '*' (الضرب)

- '/' (القسمة)

- '%' (باقي القسمة)

2. المؤثرات المنطقية (Logical Operators):

تُستخدم لإجراء عمليات منطقية.

- '&&' (AND)

- '||' (OR)

- '!' (NOT)

3. المؤثرات العلائقية (Relational Operators):

تُستخدم لمقارنة القيم.

- '==' (يساوي)

- '!=' (لا يساوي)

- '>' (أصغر من)

- '<' (أكبر من)

- '>=' (أصغر من أو يساوي)

- '<=' (أكبر من أو يساوي)

4. المؤثرات البتية (Bitwise Operators):

المؤثرات البتية (Bitwise Operators) في ++C تُستخدم لإجراء عمليات على مستوى البت، وهي تُطبق مباشرة على التمثيل الثنائي للأعداد. هذه المؤثرات يمكن استخدامها للتلاعب بالبتات الفردية داخل المتغيرات. المؤثرات البتية في ++C تشمل:

1. AND البتية (&):

يقوم هذا المؤثر بمقارنة كل بت في العدد الأول مع البت المقابل له في العدد الثاني، ويعيد 1 إذا كانت كلا البتين 1، وإلا يعيد 0.

2. OR البتية (|):

يقوم هذا المؤثر بمقارنة كل بت في العدد الأول مع البت المقابل له في العدد الثاني، ويعيد 1 إذا كانت أي من البتين 1، وإلا يعيد 0.

3. XOR البتية (^):

يقوم هذا المؤثر بمقارنة كل بت في العدد الأول مع البت المقابل له في العدد الثاني، ويعيد 1 إذا كانت البتان مختلفتين، ويعيد 0 إذا كانت البتان متطابقتين.

4. NOT البتية (~):

يقوم هذا المؤثر بعكس كل بت في العدد، أي يحول البت 0 إلى 1 والعكس.

5. إزاحة البتات لليسار (>>):

يقوم هذا المؤثر بإزاحة كل البتات في العدد إلى اليسار بعدد محدد من المواقع، وتُملأ البتات اليمينية بالقيمة 0.

6. إزاحة البتات لليمين (<<):

يقوم هذا المؤثر بإزاحة كل البتات في العدد إلى اليمين بعدد محدد من المواقع. إذا كان العدد موقعياً، تُملأ البتات اليسارية بالقيمة 0، وإذا كان العدد محلياً تُملأ البتات اليسارية بأعلى بت (علامة العدد).

استخدامات المؤثرات البتية

1. التشفير وفك التشفير: يمكن استخدام المؤثرات البتية في عمليات التشفير وفك التشفير، حيث يتم تعديل البتات مباشرة.
2. إدارة الأعلام: تُستخدم المؤثرات البتية لتعيين أو مسح أو اختبار البتات الفردية في متغير يستخدم كحقول للأعلام.
3. التعامل مع البيانات منخفضة المستوى: في برمجة الأجهزة وأنظمة التشغيل، تُستخدم المؤثرات البتية للتحكم في البتات الفردية في المسجلات أو الذاكرة.

4. العمليات الحسابية السريعة: يمكن أن تُستخدم المؤثرات البتية لتحقيق عمليات حسابية سريعة، مثل ضرب وقسمة الأعداد بواسطة قوى العدد 2.

المؤثرات البتية توفر طريقة فعالة وقوية للتعامل مع البيانات على مستوى البتات، مما يسمح للمبرمجين بتحقيق تحكم دقيق وعالي الأداء في معالجة البيانات.

5. المؤثرات الأحادية (Unary Operators):

تُستخدم لإجراء عمليات على متغير واحد.

- '+' (الإيجابي)

- '-' (السالب)

- '++' (زيادة بمقدار واحد)

- '--' (إنقاص بمقدار واحد)

- '!' (NOT المنطقية)

- '&' (عنوان المتغير)

- '*' (مؤشر إلى محتوى العنوان)

6. المؤثرات الشرطية (Conditional Operators):

المؤثرات الشرطية في ++C تُستخدم لاتخاذ قرارات بناءً على شروط معينة. هناك نوعان رئيسيان من المؤثرات الشرطية في ++C:

1. المؤثر الثلاثي (Ternary Operator):

المؤثر الثلاثي هو شكل مختصر من جملة 'if-else' ويُكتب بالشكل التالي:

condition ? expression1 : expression2;

- 'condition': الشرط الذي يتم تقييمه.

- 'expression1': التعبير الذي يتم تقييمه إذا كان الشرط صحيحًا.

- 'expression2': التعبير الذي يتم تقييمه إذا كان الشرط خطأ.

هذا المؤثر يعيد قيمة 'expression1' إذا كان الشرط صحيحًا، وإلا يعيد قيمة 'expression2'.

2. المؤثر المنطقي (Logical Operators):

المؤثرات المنطقية تُستخدم لتكوين تعبيرات شرطية أكثر تعقيدًا. هذه المؤثرات تشمل:

- AND المنطقية ('&&'): تعيد true إذا كانت كلا العبارتين صحيحتين.

- OR المنطقية (||): تعيد true إذا كانت أي من العبارتين صحيحة.

- NOT المنطقية (!): تعكس قيمة تعبير منطقي.

استخدامات المؤثرات الشرطية

1. اختيار القيم بناءً على شروط: يمكن استخدام المؤثر الثلاثي لاختيار قيمة بناءً على شرط معين بدلاً من كتابة جملة `if-else` طويلة.

2. تكوين تعبيرات شرطية معقدة: يمكن استخدام المؤثرات المنطقية لتكوين تعبيرات شرطية أكثر تعقيداً ضمن جمل التحكم مثل `if` و `while`.

3. تسهيل قراءة الكود: يمكن أن تجعل المؤثرات الشرطية الكود أكثر وضوحاً وتعبيراً عن النية بدلاً من استخدام تراكيب تحكم أطول.

المؤثرات الشرطية تُعد أداة قوية في ++C لتبسيط اتخاذ القرارات وجعل الكود أكثر قابلية للقراءة والفهم.

7. مؤثرات الإسناد (Assignment Operators):

تُستخدم لإسناد القيم إلى المتغيرات.

- `=` (الإسناد)

- `+=` (إسناد مع الجمع)

- `-=` (إسناد مع الطرح)

- `\*=` (إسناد مع الضرب)

- `/=` (إسناد مع القسمة)

- `%=` (إسناد مع باقي القسمة)

8. مؤثرات الوصول إلى الأعضاء (Member Access Operators):

مؤثرات الوصول إلى الأعضاء (Member Access Operators) في ++C تُستخدم للوصول إلى أعضاء الكائنات والهياكل (Structures) والفصول (Classes). هناك نوعان رئيسيان من مؤثرات الوصول إلى الأعضاء:

1. النقطة (.)

مؤثر النقطة يُستخدم للوصول إلى أعضاء كائن معين. يتم استخدامه عندما يكون لديك متغير من النوع المناسب وترغب في الوصول إلى أحد أعضائه.

مثال:

```
struct Person {  
    string name;  
    int age;  
};  
Person person1;  
person1.name = "John";  
person1.age = 30;
```

في هذا المثال، يُستخدم المؤثر `.` للوصول إلى أعضاء `name` و `age` في كائن `person1` من النوع `Person`.

2. مؤثر السهم (`->`)

مؤثر السهم يُستخدم مع المؤشرات للوصول إلى أعضاء كائن مؤشر عليه. يُستخدم هذا المؤثر عندما يكون لديك مؤشر إلى كائن وترغب في الوصول إلى أحد أعضائه.

مثال:

```
struct Person {  
    string name;  
    int age;  
};  
Person person2;  
Person* ptrPerson = &person2;  
ptrPerson->name = "Jane";  
ptrPerson->age = 25;
```

في هذا المثال، `ptrPerson` هو مؤشر إلى كائن من النوع `Person`. يُستخدم مؤثر السهم `->` للوصول إلى أعضاء `name` و `age` في الكائن المؤشر عليه.

استخدامات مؤشرات الوصول إلى الأعضاء

- الوصول إلى متغيرات الكائنات والهياكل: تُستخدم للوصول إلى متغيرات الأعضاء مثل السلاسل والأعداد داخل الكائنات والهياكل.

- الوصول إلى دوال الكائنات: في حالة الفصول، يمكن استخدامها للوصول إلى الدوال المُعرفة داخل الكائنات والفصول.

مؤثرات الوصول إلى الأعضاء تُسهّل عملية التحكم في البيانات والعمليات داخل الهياكل والكائنات في ++C، وتساعد في جعل الكود أكثر تنظيماً وقابلية لإدارة الأعضاء بشكل صحيح وفعال.

9. مؤثرات المؤشرات (Pointer Operators):

مؤثرات المؤشرات (Pointer Operators) في لغة ++C تُستخدم للتعامل مع المؤشرات، وهي عبارة عن عناوين للذاكرة تحتوي على عناوين أو قيم أخرى. تتيح للمبرمجين الوصول إلى العناصر في الذاكرة مباشرة وتنفيذ العمليات عليها بشكل فعال. هنا هي المؤثرات الأساسية للمؤشرات في ++C:

1. `\*` (مؤشر إلى محتوى العنوان)

يُستخدم للوصول إلى المحتوى المخزن في العنوان المُشير إليه بواسطة المؤشر.

مثال:

```
int x = 10;

int* ptr = &x; // إنشاء مؤشر على متغير x

// باستخدام المؤشر x الوصول إلى المحتوى المخزن في
int value = *ptr;
```

2. `&` (عنوان المتغير)

يُستخدم للحصول على عنوان متغير معين.

مثال:

```
int y = 20;

int* ptrY = &y; // ptrY عنوان متغير y
```

استخدامات المؤثرات المؤشرية:

- تمرير المتغيرات إلى دوال باستخدام المؤشرات: يُستخدمون لتمرير عنوان متغير إلى دالة بدلاً من تمرير قيمته بالكامل.

- إدارة الذاكرة الحيوية (Dynamic Memory Management): المؤشرات تُستخدم عادةً لحجز ذاكرة مُتغيرة ديناميكياً باستخدام `new` و `delete`.

- تلافى نسخ البيانات الزائدة: باستخدام المؤشرات، يُمكن للمبرمجين تلافى نسخ البيانات الزائدة وتحسين أداء البرنامج.
- العمليات الرياضية المتقدمة: المؤشرات يمكن أن تُستخدم في العمليات الرياضية المتقدمة مثل الإشارة إلى مصفوفات ثنائية الأبعاد.
- هذه المؤثرات تُعتبر جزءًا أساسيًا من لغة ++C وتمكن المبرمجين من التحكم الكامل في الذاكرة والبيانات التي تخزن فيها، مما يُسهّل البرمجة على مستوى منخفض ويُعزز من قدرة البرنامج على إدارة الموارد بشكل فعال.

10. المؤثرات الأخرى:

تتضمن مؤثرات متنوعة تُستخدم لأغراض خاصة.

- `sizeof` (لحساب حجم الكائن أو النوع)

في لغة ++C، `sizeof` هو عامل (operator) يُستخدم لحساب حجم النوع أو الكائن في الذاكرة. يتم استخدامه للحصول على عدد البايتات التي يستغرقها تخزين نوع محدد أو متغير محدد في الذاكرة. تفاصيل استخدام `sizeof` تشمل:

استخدام `sizeof` مع الأنواع الأساسية:

`sizeof(int);` // في الذاكرة `int` حجم نوع

`sizeof(double);` // في الذاكرة `double` حجم نوع

`sizeof(char);` // في الذاكرة `char` حجم نوع

استخدام `sizeof` مع المتغيرات:

`int x = 10;`

`double y = 3.14;`

`char c = 'A';`

`sizeof(x);` // في الذاكرة `int` من نوع `x` حجم المتغير

`sizeof(y);` // في الذاكرة `double` من نوع `y` حجم المتغير

`sizeof(c);` // في الذاكرة `char` من نوع `c` حجم المتغير

استخدام `sizeof` مع الهياكل (Structures) والفصول (Classes):

```
struct Person {
```

```
    string name;
```

```
int age;  
  
};  
  
sizeof(Person); // في الذاكرة Person حجم الهيكل
```

استخدام `sizeof` مع المؤشرات:

```
int* ptr;  
  
sizeof(ptr); // في الذاكرة ptr حجم المؤشر
```

استخدامات `sizeof`:

1. تحديد حجم النوع: يمكن استخدام `sizeof` لمعرفة حجم النوع في الذاكرة، وهذا يمكن أن يكون مفيداً عندما يتعلق الأمر بتحديد الحجم الفعلي للمتغيرات والبيانات.
 2. تحديد حجم الهياكل والفصول: يساعد `sizeof` في تحديد حجم الهياكل والفصول، وهذا يمكن أن يكون مفيداً لتخطيط الذاكرة وإدارة الموارد.
 3. تحديد حجم المؤشرات: يمكن استخدام `sizeof` لمعرفة حجم المؤشرات، وهذا يساعد في حساب التغييرات في الذاكرة التي تطرأ عند استخدام المؤشرات.
- `sizeof` يُعد أداة قوية في ++C للحصول على معلومات دقيقة حول كيفية تخزين البيانات في الذاكرة، ويُستخدم بشكل شائع في البرمجة من أجل التحقق من الحجم الذي سيشغله كائن معين أو نوع في الذاكرة.

- `typeid` (المعرفة نوع الكائن في وقت التنفيذ)

`typeid` في ++C هو عامل (operator) يُستخدم لاسترجاع معلومات عن نوع (type) متغير معين أو قيمة. يتم استخدامه للحصول على معلومات حول النوع الفعلي لكائن في وقت التشغيل (run-time). يمكن أن يُفيد في إجراءات التحقق من النوع والتعامل مع الوراثة (inheritance).

`typeid` يُعيد كائن من النوع `std::type\_info`، والذي يحتوي على معلومات حول النوع مثل اسمه وما إذا كان مؤشراً أو مرجعاً.

استخدام `typeid` يتطلب تضمين ``.

- ``,`` (المؤثر الفاصلة)

مؤثر الفاصلة (`comma operator`) في ++C هو عامل يُستخدم لفصل بين تعبيرين أو أكثر في سياق اللغة البرمجية. يتم تقييم العبارة الأولى من الفاصلة، ثم العبارة الثانية، ويُعاد قيمة العبارة الثانية. يُستخدم غالباً في الحلقات لتحديد خطوات الحلقة، أو في التعبيرات التي تحتاج إلى تقييم عدة تعبيرات بترتيب محدد، مع تجنب تكرار استخدام فاصلة بنقطة واحدة في السطر.

- ``: (نطاق الحل)

نطاق الحل (``:) في ++C يُستخدم لتحديد النطاق الذي ينتمي إليه كل عنصر في البرنامج، سواء كان ذلك فصل (class)، فصل مشتق (derived class)، أو فضاء أسماء (namespace).

المؤثرات تعتبر جزءاً أساسياً من البرمجة في ++C، حيث تتيح للمبرمجين إجراء عمليات متنوعة ومعقدة على البيانات بشكل مباشر وسهل.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة من انواع المؤثرات المستخدمة في ++C مع ذكر مثال على كل نوع
- 2- اذكر ثلاثة من استخدامات المؤثرات البنئية
- 3- اذكر ثلاثة من استخدامات المؤثرات الشرطية
- 4- اذكر ثلاثة من استخدامات مؤثرات الوصول الى الاعضاء
- 5- اذكر ثلاثة من استخدامات sizeof

المحاضرة الثالثة عشر

الهياكل

في ++C، الهياكل (Structures) هي تعريفات للبيانات تسمح للمبرمجين بتجميع عدة أنواع مختلفة من البيانات تحت مجموعة واحدة. تُستخدم الهياكل لإنشاء نوع جديد من البيانات يتكون من متغيرات مختلفة تُعرف بأسماء وتخزن في ذاكرة النظام بشكل متتالي. الهياكل تُعتبر أحد أساسيات البرمجة الهيكلية في ++C وتوفر إمكانيات كبيرة لتنظيم البيانات بشكل منطقي وسهل الاستخدام.

خصائص الهياكل في ++C:

1. تعريف الهيكل:

يتم تعريف الهيكل باستخدام الكلمة المفتاحية `struct` متبوعة باسم الهيكل والحاضرة. يمكن تعريف المتغيرات داخل الهيكل كما لو كانت متغيرات عادية.

```
struct Person {  
    std::string name;  
    int age;  
    double height;  
};
```

2. إنشاء متغيرات من نوع الهيكل:

بعد تعريف الهيكل، يمكن إنشاء متغيرات جديدة من نوع الهيكل باستخدام اسم الهيكل متبوعاً بالاسم المختار للمتغير.

```
Person person1; // إنشاء متغير من نوع Person
```

3. وصول إلى أعضاء الهيكل:

يمكن الوصول إلى أعضاء الهيكل باستخدام مشغل النقطة (`.`) بعد اسم المتغير.

```
person1.name = "Ahmed";  
person1.age = 30;  
person1.height = 175.5;
```

4. المبادئ الأساسية:

- تسمح الهياكل بتجميع البيانات ذات الصلة تحت متغير واحد.
- يمكن تعريف واستخدام الهياكل لتمثيل هياكل بيانات معقدة مثل الأشخاص، الكتب، السيارات، إلخ.

الهياكل تُستخدم بشكل واسع في البرمجة لتنظيم البيانات وتحسين قابلية الصيانة والتعديل، وتُعد جزءاً أساسياً من البرمجة المنظمة والهيكلية في C++.

أنواع الهياكل

في C++، هناك عدة أنواع من الهياكل التي يمكن استخدامها، وتختلف بناءً على السياق واستخدامها. الأنواع الرئيسية للهياكل تشمل:

1. الهياكل الأساسية (Plain Structures):

الهياكل الأساسية: يتم تعريفها باستخدام الكلمة المفتاحية `struct` وتستخدم لتجميع البيانات المتنوعة في نوع بيانات واحد.

مثال:

```
struct Person {  
    std::string name;  
    int age;  
    double height;  
};
```

2. الهياكل المتداخلة (Nested Structures):

يمكن تعريف هياكل بداخل هياكل أخرى لتجميع البيانات المتعلقة ببعضها البعض في نمط متداخل.

مثال:

```
struct Address {  
    std::string city;  
    std::string state;  
    int zipCode;  
};  
  
struct Person {  
    std::string name;
```

```
int age;  
Address address;  
};
```

3. الهياكل المجهولة (Anonymous Structures):

تستخدم لتعريف متغيرات هيكلية بدون الحاجة إلى اسم الهيكل. تُستخدم عادةً في حالات مؤقتة أو محلية.

مثال:

```
struct {  
    int id;  
    std::string description;  
} item;
```

4. الهياكل ذات الأنواع المشتركة (Union Structures):

تسمح بتخزين أنواع مختلفة من البيانات في نفس موقع الذاكرة. تُستخدم في حالات تحتاج إلى استخدام نوع واحد فقط من البيانات في وقت معين.

مثال:

```
union Data {  
    int intValue;  
    float floatValue;  
    char charValue;  
};
```

5. الهياكل ذات التعداد (Enumerated Structures):

تسمح بتعريف القيم الثابتة المرتبطة داخل الهيكل.

مثال:

```
struct Car {
```

```
enum CarType { SEDAN, SUV, TRUCK } type;  
  
std::string brand;  
  
int year;  
  
};
```

6. الهياكل المُعرَّفة في فضاء الأسماء (Namespace Structures):

الهياكل المعرفة داخل نطاق معين: تُستخدم لتنظيم الهياكل داخل فضاء الأسماء لتجنب التضارب بين الأسماء.
مثال:

```
namespace Company {  
  
    struct Employee {  
  
        std::string name;  
  
        int id;  
  
        double salary;  
  
    };  
  
}
```

هذه الأنواع من الهياكل توفر مرونة كبيرة للمبرمجين في تنظيم البيانات وتعزيز القدرة على إدارة وتعريف أنواع البيانات بشكل منهجي وفعال.

أمثلة على استخدام الهياكل (structures) في C++:

المثال الأول: هيكل بسيط لتخزين بيانات شخص

```
#include <iostream>  
  
#include <string>  
  
struct Person {  
  
    std::string name;  
  
    int age;  
  
    double height;  
  
};  
  
int main() {
```

```
Person person1;  
// تعيين القيم لأعضاء الهيكل  
person1.name = "Ahmed";  
person1.age = 25;  
person1.height = 175.5;  
// طباعة القيم  
std::cout << "Name: " << person1.name << std::endl;  
std::cout << "Age: " << person1.age << std::endl;  
std::cout << "Height: " << person1.height << std::endl;  
return 0;  
}
```

الشرح:

- تعريف الهيكل `Person` لتخزين بيانات شخص مثل الاسم والعمر والطول.
- في الدالة `main`، تم إنشاء متغير من نوع `Person` يسمى `person1`.
- تم تعيين القيم لأعضاء الهيكل ثم طباعتها على الشاشة.

المثال الثاني: هيكل متداخل لتخزين بيانات عنوان وبيانات شخص

```
#include <iostream>  
#include <string>  
struct Address {  
    std::string city;  
    std::string state;  
    int zipCode;  
};  
struct Person {  
    std::string name;  
    int age;
```

```
Address address; // النوع من النوع Address

};

int main() {

    Person person2;
    // تعيين القيم لأعضاء الهيكل
    person2.name = "Sara";
    person2.age = 30;
    person2.address.city = "Cairo";
    person2.address.state = "Cairo Governorate";
    person2.address.zipCode = 12345;
    // طباعة القيم

    std::cout << "Name: " << person2.name << std::endl;
    std::cout << "Age: " << person2.age << std::endl;
    std::cout << "City: " << person2.address.city << std::endl;
    std::cout << "State: " << person2.address.state << std::endl;
    std::cout << "ZIP Code: " << person2.address.zipCode << std::endl;

    return 0;
}
```

الشرح:

- تعريف هيكل `Address` لتخزين بيانات العنوان.
- تعريف هيكل `Person` يتضمن عضوًا من نوع `Address`.
- في الدالة `main`، تم إنشاء متغير من نوع `Person` يسمى `person2`.
- تم تعيين القيم لأعضاء الهيكل `person2` بما في ذلك بيانات العنوان المتداخل، ثم طباعة هذه القيم على الشاشة.

المثال الثالث: هيكل لتخزين بيانات كتاب واستخدامه مع دالة

```
#include <iostream>
```

```
#include <string>

// تعريف الهيكل
struct Book {
    std::string title;
    std::string author;
    int year;
};

// دالة لطباعة بيانات الكتاب
void printBookInfo(const Book& book) {
    std::cout << "Title: " << book.title << std::endl;
    std::cout << "Author: " << book.author << std::endl;
    std::cout << "Year: " << book.year << std::endl;
}

int main() {
    Book book1;
    // تعيين القيم لأعضاء الهيكل
    book1.title = "The Great Gatsby";
    book1.author = "F. Scott Fitzgerald";
    book1.year = 1925;
    // استدعاء الدالة لطباعة بيانات الكتاب
    printBookInfo(book1);
    return 0;
}
```

الشرح:

- تعريف الهيكل `Book` لتخزين بيانات كتاب مثل العنوان والمؤلف والسنة.
- تعريف دالة `printBookInfo` التي تأخذ متغيراً من نوع `Book` كمعامل وتقوم بطباعة بيانات الكتاب.
- في الدالة `main`، تم إنشاء متغير من نوع `Book` يسمى `book1`.
- تم تعيين القيم لأعضاء الهيكل `book1`.
- استدعاء الدالة `printBookInfo` لطباعة بيانات الكتاب باستخدام المتغير `book1`.

هذه الأمثلة تظهر كيفية استخدام الهياكل في ++C لتجميع البيانات ذات الصلة وتنظيمها، وكيفية تمريرها إلى الدوال لتحسين تنظيم وإدارة البيانات في البرنامج.

استخدام الهياكل

الهياكل في ++C توفر وسيلة قوية ومنظمة لتجميع البيانات المتنوعة في نوع واحد. تُستخدم الهياكل في العديد من السيناريوهات لتسهيل إدارة البيانات والعمل معها. إليك بعض الاستخدامات الرئيسية للهياكل:

1. تجميع البيانات المتعلقة ببعضها:

- تجميع بيانات معقدة: يمكن للهياكل تجميع بيانات متعددة الأنواع ذات صلة ببعضها في وحدة واحدة، مما يسهل الوصول إليها وإدارتها.
- مثال: تجميع بيانات الشخص مثل الاسم، العمر، والعنوان.

2. تنظيم البيانات في البرمجة:

- تحسين قابلية القراءة والصيانة: تنظيم البيانات في هياكل يجعل الكود أكثر وضوحاً وأسهل في الصيانة.
- مثال: إنشاء هيكل لتمثيل خصائص السيارة مثل الطراز، السنة، واللون.

3. تسهيل تمرير البيانات للدوال:

- تمرير مجموعات بيانات للدوال: يمكن تمرير متغير هيكل لدالة بدلاً من تمرير كل عنصر على حدة، مما يقلل من التعقيد.
- مثال: تمرير هيكل يحتوي على بيانات العميل إلى دالة لمعالجة الطلب.

4. إنشاء مجموعات بيانات معقدة:

- الهياكل المتداخلة: يمكن استخدام الهياكل داخل هياكل أخرى لإنشاء مجموعات بيانات معقدة ومنظمة.
- مثال: هيكل كتاب يحتوي على عنوان، مؤلف، وناشر، حيث الناشر هو هيكل آخر يحتوي على اسمه وعنوانه.

5. إدارة البيانات في التطبيقات الكبيرة:

- تسهيل إدارة البيانات في البرامج الكبيرة: الهياكل تجعل من السهل تتبع البيانات المعقدة والمتعددة في التطبيقات الكبيرة.
- مثال: استخدام هيكل لتمثيل بيانات موظفي الشركة في نظام إدارة الموارد البشرية.

6. تطوير الألعاب والتطبيقات الرسومية:

- تخزين خصائص الكائنات: يمكن استخدام الهياكل لتخزين خصائص الكائنات في الألعاب مثل الموضع، السرعة، والصحة.
- مثال: هيكل يمثل كائن لعبة يحتوي على إحداثيات الموضع، السرعة، والنقاط الصحية.

7. التعامل مع الملفات:

- تنظيم البيانات المقروءة والمكتوبة من/إلى الملفات: يمكن استخدام الهياكل لتنسيق البيانات المقروءة من الملفات أو المكتوبة إليها، مما يسهل عمليات القراءة والكتابة.
- مثال: هيكل لتخزين بيانات سجل يحتوي على حقول مثل الاسم، العنوان، ورقم الهاتف.

8. تطوير الأنظمة المدمجة:

- إدارة موارد الأجهزة: تُستخدم الهياكل لتنظيم البيانات التي تتحكم في موارد الأجهزة المختلفة في الأنظمة المدمجة.
- مثال: هيكل يمثل سجل تحكم لمصفح الجهاز يحتوي على عدة بتات تحكم.

باختصار، الهياكل تُستخدم على نطاق واسع في C++ لتنظيم البيانات وتسهيل العمل معها، مما يجعلها أداة أساسية في تطوير البرامج المعقدة والمشاريع الكبيرة.

الايخطاء التي يمكن ان تحدث عند استخدام الهياكل

استخدام الهياكل في C++ يمكن أن يؤدي إلى بعض الأخطاء الشائعة التي يجب على المبرمجين الانتباه لها. من بين هذه الأخطاء:

1. نسيان تعريف أعضاء الهيكل:

- الوصف: عند تعريف هيكل، يجب أن يتم تعريف كافة الأعضاء المكونة له بشكل صحيح.

- التأثير: قد يؤدي نسيان تعريف عضو أو استخدام عضو غير معرف إلى أخطاء في وقت الترجمة (compilation errors).

2. استخدام الذاكرة غير المهيأة:

- الوصف: عند إنشاء متغير هيكل، الأعضاء الداخلية قد تحتوي على قيم غير مهيأة (garbage values) إذا لم يتم تهيئتها بشكل صحيح.

- التأثير: استخدام الأعضاء غير المهيأة يمكن أن يؤدي إلى نتائج غير متوقعة أو أخطاء في وقت التشغيل.

3. نسيان استخدام مشغل النقطة (.) للوصول إلى الأعضاء:

- الوصف: للوصول إلى أعضاء الهيكل، يجب استخدام مشغل النقطة (.) بعد اسم متغير الهيكل.

- التأثير: نسيان استخدام مشغل النقطة يمكن أن يؤدي إلى أخطاء في وقت الترجمة.

4. الخلط بين الهياكل والفصول (classes):

- الوصف: على الرغم من أن الهياكل والفصول يتشابهان، هناك بعض الاختلافات الدقيقة، مثل أن أعضاء الهيكل عامة (public) بشكل افتراضي بينما أعضاء الفصل خاصة (private) بشكل افتراضي.

- التأثير: قد يؤدي عدم الوعي بهذه الفروق إلى أخطاء في تصميم الكود واستخدامه.

5. نسيان تضمين مكتبة <string> عند استخدام std::string:

- الوصف: عند استخدام سلاسل النصوص (strings) داخل الهياكل، يجب تضمين مكتبة <string>.

- التأثير: نسيان تضمين المكتبة يؤدي إلى أخطاء في وقت الترجمة عند محاولة استخدام std::string.

6. عدم تهيئة الهياكل الديناميكية بشكل صحيح:

- الوصف: عند استخدام الهياكل في الذاكرة الديناميكية باستخدام new، يجب التأكد من تهيئة الأعضاء بشكل صحيح وإدارة الذاكرة (بما في ذلك استخدام delete).

- التأثير: يمكن أن يؤدي ذلك إلى تسرب الذاكرة (memory leaks) أو أخطاء في الوصول إلى الذاكرة.

7. نسيان كتابة المشغلات (operators) عند الحاجة:

- الوصف: قد تحتاج إلى كتابة مشغلات مثل مشغل المساواة (equality operator) أو مشغل النسخ (copy operator) بشكل يدوي للهياكل المعقدة.

- التأثير: يمكن أن يؤدي عدم وجود هذه المشغلات إلى سلوك غير متوقع أو أخطاء عند محاولة نسخ أو مقارنة الهياكل.

8. مشاكل في توافق الذاكرة (Memory Alignment):

- الوصف: ترتيب الأعضاء في الهيكل يمكن أن يؤدي إلى مشاكل في توافق الذاكرة، خاصة عند التعامل مع بنى بيانات معقدة أو عند نقل البيانات عبر الشبكات.

- التأثير: يمكن أن يؤدي ذلك إلى أداء غير فعال أو أخطاء في الوصول إلى البيانات.

9. نسيان التحقق من الأحجام (sizes) بشكل صحيح:

- الوصف: عند إرسال أو استلام هياكل عبر الشبكات أو كتابة/قراءة الهياكل إلى/من الملفات، يجب التأكد من توافق حجم الهيكل بين النظامين.

- التأثير: يمكن أن يؤدي ذلك إلى مشاكل توافق البيانات أو فقدان البيانات.

معرفة هذه الأخطاء الشائعة يمكن أن تساعد المبرمجين على تجنبها وتطوير برامج أكثر موثوقية وصيانة.

المحاضرة الرابعة عشر

الملفات في C++

في لغة البرمجة C++، تُستخدم الملفات (Files) لتخزين البيانات بشكل دائم على القرص الصلب أو وسائط تخزين أخرى بدلاً من الذاكرة المؤقتة (RAM). يمكن للبرامج أن تقرأ من الملفات أو تكتب إليها، مما يتيح تخزين البيانات بين جلسات تشغيل البرنامج. التعامل مع الملفات في C++ يتم عادةً باستخدام مكتبة الإدخال والإخراج القياسية (Standard Input/Output Library) والتي تُعرف بـ `<fstream>`.

في C++، يتم التعامل مع نوعين رئيسيين من الملفات:

1. ملفات النصوص (Text Files):

- تحتوي على بيانات نصية يمكن قراءتها وتحريرها بواسطة برامج تحرير النصوص.

- تُستخدم لكتابة البيانات في شكل نصوص مفصولة بأسطر (مثل ملفات `.txt`).

2. الملفات الثنائية (Binary Files):

- تحتوي على بيانات في شكل ثنائي، وهي غير قابلة للقراءة بواسطة برامج تحرير النصوص العادية.

- تُستخدم لتخزين البيانات في شكلها الأصلي دون تحويلها إلى نصوص (مثل ملفات الصور، ملفات الفيديو، وملفات البرامج التنفيذية).

الاختلافات الرئيسية بين ملفات النصوص والملفات الثنائية:

- تخزين البيانات:

- ملفات النصوص: تخزن البيانات على شكل نصوص حيث تكون كل بايتات البيانات تمثل رموزًا قابلة للقراءة.
- الملفات الثنائية: تخزن البيانات في شكلها الخام، أي كما هي دون أي تحويل، مما يجعلها أكثر كفاءة من حيث المساحة والأداء.

- القراءة والكتابة:

- ملفات النصوص: تستخدم دوال مثل `getline()` للقراءة و `>>` للكتابة.
- الملفات الثنائية: تستخدم دوال مثل `read()` للقراءة و `write()` للكتابة، حيث يتم التعامل مع البيانات ككتل من البايتات.

مكتبات الإدخال والإخراج في ++C:

++C توفر مكتبات للتعامل مع الملفات والتي تتضمن:

- `<fstream>`: مكتبة تشمل `ifstream`، `ofstream`، و `fstream` للتعامل مع كل من ملفات النصوص والملفات الثنائية.

- `<iostream>`: مكتبة تشمل دوال الإدخال والإخراج القياسية مثل `cin` و `cout`.

التعامل مع الملفات في ++C

1. كتابة إلى ملف نصي:

```
#include <iostream>

#include <fstream>

int main() {

    فتح ملف للكتابة // std::ofstream outFile("example.txt");
```

```
if (outFile.is_open()) {  
    outFile << "Hello, World!" << std::endl; // كتابة نص إلى الملف  
    outFile.close(); // إغلاق الملف  
} else {  
    std::cerr << "Unable to open file";  
}  
return 0;  
}
```

2. قراءة من ملف نصي:

```
#include <iostream>  
#include <fstream>  
#include <string>  
int main() {  
    std::ifstream inFile("example.txt"); // فتح ملف للقراءة  
    std::string line;  
    if (inFile.is_open()) {  
        while (getline(inFile, line)) { // قراءة كل سطر من الملف  
            std::cout << line << std::endl; // طباعة السطر  
        }  
        inFile.close(); // إغلاق الملف  
    } else {  
        std::cerr << "Unable to open file";  
    }  
    return 0;  
}
```

3. كتابة إلى ملف ثنائي:

```
#include <iostream>

#include <fstream>

int main() {

    std::ofstream outFile("example.bin", std::ios::binary); // فتح ملف ثنائي للكتابة
    if (outFile.is_open()) {

        int data = 12345;

        outFile.write(reinterpret_cast<char*>(&data), sizeof(data)); // كتابة البيانات الثنائية
        outFile.close(); // إغلاق الملف
    } else {

        std::cerr << "Unable to open file";

    }

    return 0;

}
```

4. قراءة من ملف ثنائي:

```
#include <iostream>

#include <fstream>

int main() {

    std::ifstream inFile("example.bin", std::ios::binary); // فتح ملف ثنائي للقراءة
    if (inFile.is_open()) {

        int data;

        inFile.read(reinterpret_cast<char*>(&data), sizeof(data)); // قراءة البيانات الثنائية
        std::cout << "Data: " << data << std::endl; // طباعة البيانات
        inFile.close(); // إغلاق الملف
    } else {

        std::cerr << "Unable to open file";

    }

}
```

```
return 0;
```

```
}
```

هذه الأمثلة توضح كيفية التعامل مع الملفات النصية والثنائية في C++. يجب دائماً التحقق من أن الملف تم فتحه بنجاح قبل محاولة القراءة منه أو الكتابة إليه لتجنب الأخطاء.

استخدامات الملفات في C++

استخدامات الملفات في C++ متعددة وتغطي العديد من السيناريوهات التي تحتاج إلى تخزين واسترجاع البيانات بشكل دائم. فيما يلي بعض الاستخدامات الشائعة للملفات في C++:

1. تخزين البيانات بين جلسات تشغيل البرنامج:

يُستخدم لتخزين البيانات التي تحتاج إلى الاحتفاظ بها بين عمليات تشغيل البرنامج مثل إعدادات المستخدم، نتائج اللعبة، سجلات النشاط، إلخ.

2. إعداد التقارير وتوليد الملفات النصية:

يمكن استخدام الملفات لإنشاء تقارير نصية تحتوي على نتائج العمليات، السجلات، والتحليلات التي يمكن مشاركتها أو حفظها للمراجعة المستقبلية.

3. القراءة من مصادر بيانات ثابتة:

تستخدم الملفات لقراءة البيانات الثابتة مثل ملفات التكوين (Configuration Files)، ملفات البيانات، ملفات القواميس، إلخ، التي يحتاج البرنامج إلى قراءتها أثناء التشغيل.

4. التعامل مع قواعد البيانات البسيطة:

يمكن استخدام الملفات لتخزين البيانات المنظمة بطريقة تشبه قواعد البيانات البسيطة، مثل ملفات CSV أو JSON، التي تُستخدم لتخزين جداول بيانات.

(ملفات Comma-Separated Values CSV هي نوع من الملفات النصية التي تُستخدم لتخزين البيانات بشكل منظم يشبه الجداول. كل سطر في ملف CSV يمثل سجلاً واحداً، وكل قيمة في السجل تُفصل بفاصلة. تُستخدم ملفات CSV بشكل واسع لتبادل البيانات بين التطبيقات المختلفة، خصوصاً في البيانات التي تتعامل مع البيانات الجدولية مثل قواعد البيانات وجداول البيانات) مثل Excel.

خصائص ملفات CSV

تنظيم البيانات:

البيانات تُنظم على شكل جداول، حيث يمثل كل سطر سجلاً جديداً وكل قيمة في السجل تُفصل بفاصلة.

سهولة القراءة:

يمكن فتح ملفات CSV بواسطة أي محرر نصوص، كما يمكن استيرادها إلى برامج جداول البيانات مثل Microsoft Excel وGoogle Sheets.

البساطة:

ملفات CSV بسيطة وسهلة الإنشاء والتحليل، مما يجعلها مناسبة لنقل البيانات بين الأنظمة المختلفة.

بنية ملف CSV

عادةً ما يحتوي ملف CSV على سطر أول يُعرف باسم رأس الجدول (Header) ، الذي يصف أسماء الأعمدة. يلي ذلك أسطر البيانات.

ملفات JSON (JavaScript Object Notation) هي تنسيق خفيف لتبادل البيانات، يمكن قراءته وكتابته بسهولة من قبل الإنسان وآلات البرمجة. يستخدم JSON بشكل واسع لتبادل البيانات بين الخادم والعميل في تطبيقات الويب.

خصائص ملفات JSON

تنسيق سهل القراءة:

البيانات تُنظم بطريقة تسهل قراءتها وفهمها، حتى من قبل غير المبرمجين.

قابلية النقل:

JSON خفيف الوزن ويمكن نقله عبر الشبكة بسهولة، مما يجعله مناسبًا لتطبيقات الويب والخدمات السحابية.

دعم واسع:

JSON مدعوم من قبل معظم لغات البرمجة، مما يجعله خيارًا مرئيًا لتبادل البيانات بين الأنظمة المختلفة.

بنية JSON

ملفات JSON تتكون من أزواج مفتاح-قيمة. يمكن أن تحتوي البيانات على كائنات (objects) أو مصفوفات (arrays).

5. تخزين وتحميل البيانات الكبيرة:

تُستخدم الملفات لتخزين البيانات الكبيرة التي لا يمكن تحميلها في الذاكرة دفعة واحدة. يمكن تقسيم البيانات إلى أجزاء وتحميلها عند الحاجة.

6. التعامل مع الملفات الثنائية:

يمكن استخدام الملفات لتخزين البيانات الثنائية مثل الصور، الفيديو، الملفات الصوتية، وملفات البرامج التنفيذية. يتم قراءة وكتابة البيانات كما هي دون تحويلها إلى نصوص.

7. النسخ الاحتياطي واستعادة البيانات:

يمكن استخدام الملفات لإنشاء نسخ احتياطية من البيانات الهامة واستعادتها عند الحاجة.

8. التعامل مع بيانات الإدخال/الإخراج:

يمكن استخدام الملفات لقراءة البيانات من أجهزة الإدخال أو الكتابة إلى أجهزة الإخراج مثل الطابعات، أجهزة التخزين الخارجية، إلخ.

أمثلة على استخدامات الملفات في C++

مثال على تخزين إعدادات المستخدم

هذا المثال يوضح كيفية تخزين واسترجاع إعدادات المستخدم (مثل اسم المستخدم وأعلى درجة حققها) باستخدام ملفات نصية.

1. كتابة إعدادات المستخدم إلى ملف نصي

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
void saveSettings(const std::string &username, int highScore) {
```

```
    std::ofstream outFile("settings.txt");
```

```
    if (outFile.is_open()) {
```

```
        outFile << username << std::endl;
```

```
        outFile << highScore << std::endl;
```

```
        outFile.close();
```

```
    } else {
```

```
std::cerr << "Unable to open file for writing";
}
}
```

- `<include <iostream#``: مكتبة الإدخال والإخراج القياسية.

- `<include <fstream#``: مكتبة الإدخال والإخراج من الملفات.

- `<include <string#``: مكتبة التعامل مع النصوص.

- `void saveSettings(const std::string &username, int highScore)`: دالة لتخزين الإعدادات في ملف.

- `std::ofstream outFile("settings.txt")`: إنشاء كائن من نوع `ofstream` وفتح ملف نصي للكتابة.

- `outFile.is_open()`: التحقق من أن الملف تم فتحه بنجاح.

- `outFile << username << std::endl`: كتابة اسم المستخدم إلى الملف.

- `outFile << highScore << std::endl`: كتابة أعلى درجة حققها المستخدم إلى الملف.

- `outFile.close()`: إغلاق الملف بعد الانتهاء من الكتابة.

2. قراءة إعدادات المستخدم من ملف نصي

```
void loadSettings(std::string &username, int &highScore) {
    std::ifstream inFile("settings.txt");
    if (inFile.is_open()) {
        getline(inFile, username);
        inFile >> highScore;
        inFile.close();
    } else {
        std::cerr << "Unable to open file for reading";
    }
}
```

- `std::ifstream inFile("settings.txt")`: إنشاء كائن من نوع `ifstream` وفتح ملف نصي للقراءة.

- `getline(inFile, username)`: قراءة اسم المستخدم من الملف.

- `inFile >> highScore`: قراءة أعلى درجة من الملف.
- `inFile.close();`: إغلاق الملف بعد الانتهاء من القراءة.

3. البرنامج الرئيسي

```
int main() {  
    std::string username;  
    int highScore;  
  
    // Save settings  
    saveSettings("Player1", 100);  
  
    // Load settings  
    loadSettings(username, highScore);  
  
    std::cout << "Username: " << username << std::endl;  
    std::cout << "High Score: " << highScore << std::endl;  
  
    return 0;  
}
```

- `int main()`: الدالة الرئيسية للبرنامج.
- `saveSettings("Player1", 100)`: حفظ إعدادات المستخدم.
- `loadSettings(username, highScore)`: تحميل إعدادات المستخدم.
- `std::cout << "Username: " << username << std::endl`: طباعة اسم المستخدم.
- `std::cout << "High Score: " << highScore << std::endl`: طباعة أعلى درجة.

مثال على التعامل مع ملفات CSV

هذا المثال يوضح كيفية التعامل مع ملفات CSV (قيم مفصولة بفواصل) لكتابة وقراءة بيانات أشخاص.

1. كتابة بيانات إلى ملف CSV

```
#include <iostream>

#include <fstream>

#include <sstream>

#include <vector>

struct Person {

    std::string name;

    int age;

    std::string city;

};

void writeCSV(const std::vector<Person> &people, const std::string &filename) {

    std::ofstream outFile(filename);

    if (outFile.is_open()) {

        outFile << "Name,Age,City" << std::endl;

        for (const auto &person : people) {

            outFile << person.name << "," << person.age << "," << person.city << std::endl;

        }

        outFile.close();

    } else {

        std::cerr << "Unable to open file for writing";

    }

}
```

- struct Person : هيكل لتمثيل بيانات الشخص.

- `void writeCSV(const std::vector<Person> &people, const std::string &filename)` : دالة لكتابة بيانات الأشخاص إلى ملف CSV.

- `std::ofstream outFile(filename)` : إنشاء كائن من نوع `ofstream` وفتح ملف CSV للكتابة.

- `outFile << "Name, Age, City" << std::endl` : كتابة رأس الجدول إلى الملف.

- `for (const auto &person : people)` : حلقة لكتابة بيانات كل شخص في الجدول.

- `outFile << person.name << "," << person.age << "," << person.city << std::endl` : كتابة بيانات الشخص إلى الملف.

2. قراءة بيانات من ملف CSV

```
std::vector<Person> readCSV(const std::string &filename) {  
    std::vector<Person> people;  
    std::ifstream inFile(filename);  
    if (inFile.is_open()) {  
        std::string line;  
        getline(inFile, line); // Skip header  
        while (getline(inFile, line)) {  
            std::stringstream ss(line);  
            std::string name, ageStr, city;  
            getline(ss, name, ',');  
            getline(ss, ageStr, ',');  
            getline(ss, city, ',');  
            people.push_back({name, std::stoi(ageStr), city});  
        }  
        inFile.close();  
    } else {  
        std::cerr << "Unable to open file for reading";  
    }  
    return people;  
}
```

}

- `std::vector<Person> readCSV(const std::string &filename)` - دالة لقراءة بيانات الأشخاص من ملف CSV.

- `std::ifstream inFile(filename)` : إنشاء كائن من نوع `ifstream` وفتح ملف CSV للقراءة.

- `getline(inFile, line)` : قراءة السطر وتخطي رأس الجدول.

- `while (getline(inFile, line))` : حلقة لقراءة كل سطر من الملف.

- `std::stringstream ss(line)` : تحويل السطر إلى دفق نصي لمعالجة البيانات.

- `getline(ss, name, ',')` : قراءة اسم الشخص.

- `getline(ss, ageStr, ',')` : قراءة عمر الشخص كقيمة نصية.

- `getline(ss, city, ',')` : قراءة مدينة الشخص.

- `people.push_back({name, std::stoi(ageStr), city})` : إضافة بيانات الشخص إلى القائمة بعد تحويل العمر إلى عدد صحيح.

3. البرنامج الرئيسي

```
int main() {
```

```
    std::vector<Person> people = {"Alice", 30, "New York"}, {"Bob", 25, "Los Angeles"};
```

```
    // Write CSV
```

```
    writeCSV(people, "people.csv");
```

```
    // Read CSV
```

```
    people = readCSV("people.csv");
```

```
    for (const auto &person : people) {
```

```
        std::cout << "Name: " << person.name << ", Age: " << person.age << ", City: " <<
        person.city << std::endl;
```

```
    }
```

```
return 0;  
}
```

```
- int main(): الدالة الرئيسية للبرنامج.  
- std::vector<Person> people = {"Alice", 30, "New York"}, {"Bob", 25, "Los Angeles"} : إنشاء قائمة بأشخاص.  
- writeCSV(people, "people.csv") : كتابة بيانات الأشخاص إلى ملف CSV.  
- people = readCSV("people.csv") : قراءة بيانات الأشخاص من ملف CSV.  
- for (const auto &person : people) : حلقة لعرض بيانات كل شخص.  
- std::cout << "Name: " << person.name << ", Age: " << person.age << ", City: " << person.city << std::endl : طباعة بيانات الشخص.
```

هذه الأمثلة توضح كيفية التعامل مع الملفات النصية وملفات CSV في C++ لتخزين واسترجاع البيانات بشكل منظم وفعال.

الأسئلة الاسترشادية

- 1- اذكر ثلاثة من استخدامات الهياكل Structures في C++
- 2- اذكر الكلمة المفتاحية المستخدمة لتعريف الهياكل
- 3- اذكر ثلاثة من الأخطاء التي يمكن ان تحدث عند استخدام الهياكل
- 4- اذكر الأنواع الرئيسية للملفات في C++
- 5- اذكر الفرق في القراءة والكتابة بين نوعي الملفات في C++
- 6- اذكر ثلاثة من استخدامات الملفات في C++
- 7- اذكر ثلاثة من خصائص ملفات CSV في C++
- 8- اذكر ثلاثة من خصائص ملفات JSON في C++

المحاضرة الرابعة عشر

السلاسل النصية (Strings) في لغة C++

في لغة C++ ، تُعد السلاسل النصية (Strings) من أهم أنواع البيانات المستخدمة لتمثيل ومعالجة النصوص. وتوفر C++ طريقتين رئيسيتين للتعامل مع السلاسل النصية:

أولاً: السلاسل النصية التقليدية (C-Style Strings)

وهي سلاسل نصية مبنية على مصفوفة من الأحرف من نوع char وتنتهي بمحرف فارغ (null character) '\0'.

مثال:

```
#include <iostream>
using namespace std;

int main() {
    char str[] = "Hello, World!";
    cout << str << endl;
    return 0;
}
```

خصائص:

- تعتمد على مصفوفات الأحرف.
- تتطلب إدارة يدوية للذاكرة.
- تُستخدم دوال من مكتبة <cstring> لمعالجة السلاسل مثل strlen(), strcpy(), strcat(), وغيرها.

ثانياً: كائن السلسلة النصية (std::string)

وهو نوع بيانات حديث ومريح يُؤقره الـ C++ ضمن مكتبة <string> يتميز بسهولة الاستخدام والتعامل مع النصوص دون الحاجة لإدارة الذاكرة يدوياً.

مثال:

```
#include <iostream>
#include <string>
using namespace std;
```



```
int main() {
    string str = "Hello, C++!";
    cout << str << endl;
    return 0;
}
```

مزايا std::string:

- دعم كامل للعمليات على السلاسل (النسخ، الإضافة، المقارنة، البحث...).
- لا حاجة لإدارة المحارف الفارغة. '\0'.
- يمكن استخدام المشغلات مثل + و == مباشرة.
- توفر توابع جاهزة مثل:
 - length() أو size() للحصول على الطول.
 - substr() لاستخراج جزء من السلسلة.
 - find() للبحث داخل السلسلة.
 - append() و insert() و erase() وغيرها.

مقارنة بين char[] و std::string:

الخاصية	char[]	std::string
إدارة الذاكرة	يدوية	آلية تلقائية
سهولة الاستخدام	أقل	أعلى
دعم العمليات النصية	محدود	متقدم
الأمان من أخطاء التجاوز	أقل	أعلى

أمثلة إضافية:

دمج سلسلتين:

```
string first = "Hello";
string second = "World";
string result = first + " " + second;
cout << result; // Hello World
```

استخدام دالة:

```
string str = "programming";  
size_t index = str.find("gram");  
if (index != string::npos) {  
    cout << "Found at index: " << index << endl;  
}
```

الأخطاء التي تحدث عند التعامل مع السلاسل النصية

عند التعامل مع السلاسل النصية (Strings) في لغة C++، سواء باستخدام النمط التقليدي (char[]) أو باستخدام كائن std::string، قد يقع المبرمج في عدد من الأخطاء الشائعة. وسنستعرض فيما يلي هذه الأخطاء مع توضيح أمثلة لها وطرق تجنبها:

أولاً: الأخطاء عند استخدام السلاسل النصية بنمط C التقليدي (char[])

1- نسيان المحرف الفارغ ('\\0') null character

السلاسل من نوع char[] يجب أن تنتهي بـ '\\0' لتحديد نهاية السلسلة.

الصحيح:

```
char str[] = {'H', 'e', 'l', 'l', 'o', '\\0'};
```

الخطأ:

```
char str[] = {'H', 'e', 'l', 'l', 'o'}; // '\\0' لا يوجد  
قد => ' لا يوجد '\\0' // يؤدي إلى سلوك غير متوقع
```

2- الوصول خارج حدود المصفوفة

تجاوز حدود المصفوفة يؤدي إلى undefined behavior سلوك غير معرّف

مثال:

```
char str[5] = "Hello"; // فيها 5 محارف + '\\0' "Hello" تحتاج إلى 6 خانات لأن
```

الحل:

```
char str[6] = "Hello";
```

3- الخلط بين تخصيص المصفوفات والتعامل مع المؤشرات

استخدام المؤشرات مع سلاسل C قد يسبب مشاكل إذا تم تعديل محتوى سلسلة ثابتة.

مثال خاطئ:

```
char* str = "Hello"; // سلسلة ثابتة (literal)
```

```
str[0] = 'h'; // خطأ: محاولة التعديل على سلسلة غير قابلة للتغيير
```

الصحيح:

```
char str[] = "Hello"; // نسخة قابلة للتعديل
```

```
str[0] = 'h';
```

4- عدم استخدام دوال النسخ الآمن

استخدام `strcpy()` أو `strcat()` بدون التأكد من سعة المصفوفة قد يؤدي إلى تجاوز الذاكرة (Buffer overflow).

يفضل استخدام `strncpy()` أو `strncat()` مع تحديد الحجم.

ثانيًا: الأخطاء عند استخدام `std::string`

1- نسيان تضمين المكتبة `<string>`

خطأ شائع:

```
string s = "Hello"; // خطأ في الترجمة إن لم يتم تضمين
```

الصحيح:

```
#include <string>
using namespace std;
```

2- محاولة استخدام `std::string` كـ `char[]`

مثال خاطئ:

```
string s = "Hello";
char c = s[100]; // الوصول إلى موقع غير موجود في السلسلة
```

الحل:

- تحقق من الحجم باستخدام `s.size()` أو `s.length()`
- أو استخدم `at()` لأنها ترمي استثناء عند الخروج عن النطاق:

```
char c = s.at(100); // سترمي std::out_of_range
```

3- سوء استخدام التكرار مع السلاسل

مثال خاطئ:

```
string s = "test";
for (int i = 0; i <= s.length(); i++) // خطأ: <= يؤدي للوصول
    لمحرف غير موجود
    cout << s[i];
```

الصحيح:

```
for (int i = 0; i < s.length(); i++)
    cout << s[i];
```

الخلط بين `std::string` و `char*`

مثال خاطئ:

```
string s = "Hello";
someCFunction(s); // char* وليس string تتطلب دالة
```

الحل:

`someCFunction(s.c_str()); // تحويل string إلى const char*`

نصائح لتفادي الأخطاء:

التوضيح	التوصية
لأنه أكثر أماناً وأسهل	استخدم <code>std::string</code> بدلاً من <code>char[]</code> إن أمكن
لتفادي الخروج عن النطاق	تحقق دائماً من حدود السلاسل قبل الوصول لمعرف معين
عند تمرير <code>std::string</code> لدوال C القديمة	استخدم <code>c_str()</code> عند الحاجة إلى <code>char*</code>
لأنها غالباً موجودة في جزء غير قابل للكتابة من الذاكرة	تجنب تعديل سلاسل ثابتة

الأسئلة الاسترشادية

- 1- اذكر انواع السلاسل النصية
- 2- اذكر خصائص السلاسل النصية التقليدية
- 3- اذكر ثلاثة من الأخطاء التي يمكن ان تحدث عند استخدام السلاسل النصية



1. المراجع

2. **The C++ Programming Language** – Bjarne Stroustrup
3. **Programming: Principles and Practice Using C++** – Bjarne Stroustrup
4. **C++ Primer (5th Edition)** – Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
5. **Effective Modern C++** – Scott Meyers
6. **More Effective C++** – Scott Meyers
7. **Effective C++ (3rd Edition)** – Scott Meyers
8. **Accelerated C++** – Andrew Koenig, Barbara E. Moo
9. **The C++ Standard Library: A Tutorial and Reference** – Nicolai M. Josuttis
10. **Modern C++ Design** – Andrei Alexandrescu
11. **C++ Concurrency in Action** – Anthony Williams
12. **C++ Templates: The Complete Guide** – David Vandevoorde, Nicolai M. Josuttis, Douglas Gregor
13. **A Tour of C++** – Bjarne Stroustrup